

Facultade de Informática
UNIVERSIDADE DA CORUÑA

TRABALLO FIN DE GRAO
GRAO EN ENXEÑARÍA INFORMÁTICA
MENCIÓN EN ENXEÑARÍA DO SOFTWARE




# Vodeo: Diseño y desarrollo de una plataforma de streaming VOD basada en arquitectura de microservicios

**Estudante:** Alexandre Fernández Fernández

**Dirección:** Miguel Rodríguez Penabad

A Coruña, septiembre de 2026.

*A mis padres y hermanos*

**Agradecimientos**

Quiero agradecer a mi familia y a Gabi por apoyarme incondicionalmente durante estos años. En especial a mi madre por siempre confiar en mí. A mis amigos por acompañarme durante este camino. También agradecer a mi tutor, Miguel, por haberme guiado durante el desarrollo de este proyecto.

**Resumen**

En la actualidad, el consumo de contenido audiovisual a través de internet es una parte fundamental del ocio, lo que exige plataformas capaces de distribuir vídeos de forma eficiente y escalable. Sin embargo, las aplicaciones monolíticas tradicionales presentan problemas de rendimiento y escalabilidad cuando manejan tareas pesadas, como el procesamiento de vídeo, afectando a la experiencia global del usuario.

El objetivo de este trabajo de fin de grado es el diseño y desarrollo de "Vodeo", una plataforma de streaming de vídeo bajo demanda (VOD) basada en una arquitectura de microservicios, que proporciona a los usuarios un sistema robusto para el consumo de contenido multimedia con reproducción adaptativa en tiempo real según las condiciones de red.

Para lograr este objetivo, se empezó por diseñar un ecosistema de servicios independientes aislados mediante contenedores Docker. Posteriormente, se implementó un motor de transcodificación asíncrono utilizando FFmpeg y colas de mensajería (RabbitMQ) para convertir los archivos de vídeo al protocolo HLS. Finalmente, se desarrolló una interfaz de usuario completa que integra un reproductor adaptativo capaz de ajustar la calidad del vídeo en tiempo real evitando interrupciones durante la reproducción.

En cuanto a la arquitectura y desarrollo, se diseñó un sistema distribuido compuesto por un backend de microservicios desarrollados en lenguaje Java mediante el framework Spring Boot, utilizando PostgreSQL como base de datos relacional para la persistencia y Redis para la gestión de la caché. En la capa de presentación (frontend), se utilizó la librería React para construir la interfaz de usuario. El proyecto se desarrolló siguiendo una metodología ágil basada en Kanban.

**Abstract**

Nowadays, the consumption of audiovisual content over the internet is a fundamental part of leisure, demanding platforms capable of distributing videos in an efficient and scalable manner. However, traditional monolithic applications present performance and scalability issues when handling heavy tasks, such as video processing, affecting the overall user experience.

The objective of this final degree project is the design and development of "Vodeo", a Video on Demand (VOD) streaming platform based on a microservices architecture, which provides users with a robust system for multimedia consumption with adaptive real-time playback according to network conditions.

To achieve this goal, the process began by designing an ecosystem of independent services isolated using Docker containers. Subsequently, an asynchronous transcoding engine was

implemented using FFmpeg and message queues (RabbitMQ) to convert video files to the HLS protocol. Finally, a comprehensive user interface was developed that integrates an adaptive player capable of adjusting video quality in real-time, thereby avoiding interruptions during playback.

Regarding the architecture and development, a distributed system was designed, consisting of a microservices backend developed in Java using Spring Boot framework, utilizing PostgreSQL as a relational database for persistence and Redis for cache management. On the presentation layer (frontend), the React library was used to build the user interface. The project was developed following an agile methodology based on Kanban.

**Palabras clave:**

- Vídeo bajo demanda (VOD)
- Arquitectura de microservicios
- Transcodificación asíncrona
- HTTP Live Streaming (HLS)
- Reproducción adaptativa

**Keywords:**

- Video on demand (VOD)
- Microservices architecture
- Asynchronous transcoding
- HTTP Live Streaming (HLS)
- Adaptive Bitrate Streaming (ABS)

# Índice general


**1 Introducción** **1**

1.1 Contexto y motivaciones . . . 1

1.2 Objetivos . . . 2

1.3 Estructura de la memoria . . . 2

**2 Fundamentos tecnológicos** **4**

2.1 Estado del arte . . . 4

2.2 Tecnologías utilizadas . . . 6

2.2.1 Capa de servicios (Backend) . . . 6

2.2.2 Capa de presentación (Frontend) . . . 7

2.2.3 Almacenamiento de datos . . . 7

2.3 Herramientas de desarrollo . . . 7

**3 Metodología y planificación** **8**

3.1 Metodología de desarrollo . . . 8

3.1.1 Flujo de trabajo . . . 9

3.2 Planificación . . . 10

3.2.1 Recursos . . . 10

3.2.2 Fases del proyecto planificadas . . . 11

3.2.3 Diagrama de Gantt inicial . . . 11

3.2.4 Estimación de costes inicial . . . 12

3.3 Seguimiento . . . 12

3.3.1 Desviaciones . . . 12

3.3.2 Diagrama de Gantt real . . . 13

3.3.3 Impacto en costes . . . 13

**4 Análisis** **15**

4.1 Requisitos . . . 15

4.1.1 Actores del sistema . . . 15
4.1.2 Requisitos funcionales . . . 15
4.1.3 Casos de uso . . . 17
4.1.4 Requisitos no funcionales . . . 19
4.2 Arquitectura del sistema . . . 20
4.3 Interfaz de usuario . . . 21
4.4 Modelo conceptual de datos . . . 24
4.4.1 Entidades y relaciones del sistema . . . 25
4.5 Interfaz de los servicios . . . 26

**5 Diseño** **28**
5.1 Arquitectura tecnológica del sistema . . . 28
5.2 Diseño del Backend . . . 29
5.2.1 Diagrama de paquetes . . . 30
5.2.2 Diseño de componentes: fachadas y flujo de dependencias . . . 31
5.2.3 Comunicación asíncrona: arquitectura orientada a eventos . . . 33
5.3 Modelo lógico de datos . . . 34
5.3.1 Tablas del *users-service* . . . 35
5.3.2 Tablas del *catalog-service* . . . 37
5.4 Diseño del Frontend . . . 43
5.4.1 Estructura de componentes y enrutamiento . . . 44
5.4.2 Gestión de sesión y comunicación con el backend . . . 45
5.4.3 Patrones de diseño en la capa de presentación . . . 45
5.5 Comportamiento dinámico (Diagramas de secuencia) . . . 47
5.6 Trazabilidad . . . 49

**6 Implementación y pruebas** **51**
6.1 Implementación . . . 51
6.1.1 Entrega fragmentada de vídeo (*Chunked Streaming*) . . . 51
6.1.2 Autenticación JWT en las peticiones de `hls.js` . . . 52
6.1.3 Corrección de consultas N+1 y `MultipleBagFetchException` 53
6.1.4 Búsqueda difusa con `pg_trgm` . . . 54
6.1.5 Transcodificación adaptativa y control del subproceso FFmpeg . . . 55
6.1.6 Caché distribuida con Redis: patrón *Cache-Aside* . . . 57
6.1.7 Reproducción continua entre episodios (*Autoplay*) . . . 58
6.1.8 Composición y post-procesado de datos en el cliente . . . 59
6.1.9 Propagación de errores internos con Spring Security 6 . . . 61
6.1.10 Despliegue . . . 62

6.2 Pruebas . . . 62
6.2.1 Pruebas unitarias . . . 63
6.2.2 Pruebas de integración automatizadas . . . 66
6.2.3 Pruebas manuales de API con Postman . . . 67
6.2.4 Pruebas de aceptación . . . 68

**7 Solución desarrollada** **69**
7.1 Registro e inicio de sesión . . . 69
7.2 Catálogo . . . 69
7.3 Ficha de contenido y reproducción . . . 70
7.4 Reseñas . . . 72
7.5 Mi Lista . . . 73
7.6 Seguir viendo y recomendaciones . . . 74
7.7 Panel de administración . . . 74
7.8 Panel de gestión de usuarios y analítica . . . 76
7.9 Gestión del perfil . . . 77

**8 Conclusiones y trabajo futuro** **78**
8.1 Grado de cumplimiento de los objetivos . . . 78
8.2 Aportación personal y profesional . . . 78
8.3 Trabajo futuro . . . 79

**A Material adicional** **82**
A.1 Prototipos de interfaz de usuario . . . 82

**Bibliografía** **92**

# Índice de figuras


2.1 Interfaz web de usuario del catálogo de Netflix. . . . . 4
2.2 Interfaz web de usuario del catálogo de Amazon Prime Video. . . . . 5
2.3 Interfaz web de usuario del catálogo de Plex. . . . . 5
2.4 Interfaz web de usuario del catálogo de Jellyfin. . . . . 6

3.1 Ejemplo teórico de un tablero de Kanban. . . . . 8
3.2 Tablero Kanban del proyecto Vodeo en GitHub Projects. . . . . 10
3.3 Diagrama de Gantt con la planificación inicial estimada. . . . . 12
3.4 Diagrama de Gantt con la distribución real de las fases del proyecto. . . . . 13

4.1 Arquitectura de alto nivel del sistema Vodeo. . . . . 21
4.2 Mapa de navegación de la interfaz de usuario de Vodeo. . . . . 23
4.3 Prototipo de la pantalla de Catálogo. . . . . 23
4.4 Prototipo de la pantalla de Detalle de contenido (Serie). . . . . 24
4.5 Diagrama entidad-relación de la plataforma Vodeo. . . . . 26

5.1 Arquitectura tecnológica de la plataforma Vodeo. . . . . 29
5.2 Organización en capas de los microservicios de backend. . . . . 31
5.3 Diagrama de clases simplificado del *catalog-service*: fachada, orquestación y patrón fallback. . . . . 33
5.4 Modelo lógico de datos de Vodeo, distribuido entre *users-service* y *catalog-service*. 35
5.5 Diagrama de componentes y enrutamiento de la SPA de Vodeo. . . . . 45
5.6 Diagrama de secuencia de CU-01: subida y procesamiento asíncrono de vídeo. 47
5.7 Diagrama de secuencia de CU-02: reproducción adaptativa mediante HLS. . . 48

6.1 Resultado de la ejecución de las pruebas unitarias del backend. . . . . 66
6.2 Autenticación de un usuario: la respuesta incluye el `AuthResponse` con el JWT. . . . . 68

6.3 Intento de borrado de una película con un token de rol USER: el servidor responde `403 Forbidden`. . . . . 68

7.1 Pantallas de registro e inicio de sesión de Vodeo. . . . . 69
7.2 Vista principal del catálogo. . . . . 70
7.3 Resultado de una búsqueda de contenido en el catálogo. . . . . 70
7.4 Ficha de una película, con su información y el botón de reproducción. . . . . 71
7.5 Ficha de una serie con la navegación por temporadas y episodios. . . . . 71
7.6 Reproducción de una película. . . . . 72
7.7 Aviso de reproducción automática del siguiente episodio. . . . . 72
7.8 Sección de reseñas de un contenido. . . . . 73
7.9 Vista de "Mi Lista" con contenido guardado por el usuario. . . . . 73
7.10 Carrusel de "Seguir viendo" y de recomendaciones en la vista principal del catálogo. . . . . 74
7.11 Formulario de alta de una película, con importación automática de metadatos. 75
7.12 Formulario de alta de una serie, con importación automática de metadatos. . . 75
7.13 Gestión de temporadas y episodios desde la ficha de una serie. . . . . 76
7.14 Panel de gestión de usuarios. . . . . 76
7.15 Panel analítico con métricas de la plataforma. . . . . 77
7.16 Panel de gestión del perfil de usuario. . . . . 77

A.1 Prototipo de la pantalla de inicio de sesión. . . . . 83
A.2 Prototipo de la pantalla de registro. . . . . 83
A.3 Prototipo de la pantalla principal del catálogo. . . . . 84
A.4 Prototipo de la pantalla de detalle de película. . . . . 85
A.5 Prototipo de la pantalla de detalle de serie. . . . . 86
A.6 Prototipo de la pantalla del reproductor multimedia. . . . . 87
A.7 Prototipo de la pantalla de lista de favoritos. . . . . 87
A.8 Prototipo de la pantalla de perfil de usuario. . . . . 88
A.9 Prototipo del panel de administración: subida de contenido. . . . . 89
A.10 Prototipo del panel de administración: estadísticas. . . . . 89
A.11 Prototipo del panel de administración: gestión de usuarios. . . . . 90

# Índice de tablas


3.1 Comparativa de costes planificados y reales del proyecto. . . . . 13

4.1 Grupos funcionales expuestos por la interfaz de servicios de Vodeo. . . . . 27

5.1 Diccionario de datos de la tabla `users`. . . . . 36
5.2 Diccionario de datos de la tabla `watchlist`. Restricción adicional a nivel de tabla: UNIQUE`(user_id, media_id, media_type)`, para impedir duplicados del mismo contenido en la lista de un mismo usuario. . . . . 36
5.3 Diccionario de datos de la tabla `playback_history`. Restricción adicional: UNIQUE`(user_id, media_id, media_type)`, un único registro de progreso por usuario y contenido. . . . . 37
5.4 Diccionario de datos de la tabla `movies`. . . . . 38
5.5 Diccionario de datos de la tabla `series`. . . . . 39
5.6 Diccionario de datos de la tabla `genres`. . . . . 39
5.7 Diccionario de datos de las tablas intermedias `movie_genres` y `series_genres`. . . . . 40
5.8 Diccionario de datos de la tabla `seasons`. . . . . 40
5.9 Diccionario de datos de la tabla `episodes`. . . . . 41
5.10 Diccionario de datos de la tabla `subtitles`. `movie_id` y `episode_id` son mutuamente excluyentes en la práctica (uno de los dos es siempre NULL), aunque esta exclusión se controla a nivel de aplicación y no mediante una restricción CHECK física. . . . . 42
5.11 Diccionario de datos de la tabla `reviews`. Al igual que en `watchlist`, `media_id`/`media_type` es una referencia polimórfica: no hay clave foránea real porque la reseña y el contenido valorado viven en microservicios distintos. . . . . 43
5.12 Trazabilidad entre requisitos funcionales y componentes de la interfaz. . . . . 49
5.13 Trazabilidad entre casos de uso y microservicios del backend. . . . . 50

6.1 Clases de prueba unitaria desarrolladas para el backend de Vodeo. . . . . 65
6.2 Pruebas de integración automatizadas, una clase por cada repositorio del proyecto. 67

Capítulo 1

# Introducción

## 1.1 Contexto y motivaciones

En la actualidad, el consumo de contenido audiovisual a través de internet ha transformado por completo la industria del entretenimiento. Más allá de los gigantes comerciales como Netflix o Amazon Prime Video, existe una creciente comunidad de usuarios que optan por gestionar sus propias bibliotecas multimedia mediante soluciones autoalojadas (self-hosted). Herramientas como Plex se han convertido en referentes de este sector, permitiendo a los usuarios montar sus propios servidores de streaming en casa.

El uso habitual de plataformas como Plex despertó el interés por comprender la complejidad técnica que oculta la reproducción de vídeo bajo demanda (VOD). Tareas que parecen invisibles para el usuario, como la transcodificación de archivos multimedia o la adaptación automática de la calidad de imagen a la condiciones de red, representan verdaderos desafíos de ingeniería de software que estas herramientas resuelven sin exponer su arquitectura interna.

Sin embargo, al tratarse en su mayoría de soluciones privadas, el usuario carece de control sobre cómo se distribuyen los recursos internos. Al ejecutarse todas la funcionalidades en un mismo servidor centralizado, los procesos intesivos como la transcodificación un vídeo pesado pueden saturar el sistema completo, degradando la experiencia para el resto de usuarios. Este proyecto nace del interés por construir una plataforma VOD propia, aplicando un enfoque moderno basado en microservicios que resuelva estos problemas de escalabilidad aislando los procesos críticos.

El objetivo principal de este Trabajo de Fin de Grado es el diseño y desarrollo de “Vodeo”, una plataforma de streaming construida bajo una arquitectura de microservicios, capaz de gestionar contenido multimedia de forma eficiente y adaptando dinámicamente la calidad de reproducción a las condiciones de red del usuario.

## 1.2 Objetivos

Para alcanzar el objetivo principal del proyecto, es necesario dividir el trabajo en metas más concretas. El desarollo abarca dos partes principales: la creación de toda la infraestructura distribuida (backend) y el diseño de la interfaz web para el usuario. Por lo tanto, el proyecto se divide en los siguientes objetivos específicos:

- Diseñar una arquitectura de microservicios que separe las responsabilidades del sistema (usuarios, catálogo, streaming, transcodificación) en servicios independientes, centralizando las peticiones a través de un punto de entrada único para mejorar la seguridad.
- Implementar un sistema de autenticación seguro, con dos roles de usuario: administrador (gestor de la plataforma) y usuario estándar (consumidor de contenido).
- Construir un catálogo de contenido multimedia que permita a los administradores subir películas y series, con importación automática de metadatos (título, sinopsis, duración, carteles, etc.) desde bases de datos cinematográficas externas.
- Desarrollar un motor de procesamiento de vídeo en segundo plano. Este sistema convertirá los archivos para permitir una reproducción fluida a través de internet, ajustando la calidad de imagen en tiempo real dependiendo de la conexión del espectador.
- Implementar funcionalidades características de las plataformas comerciales: lista de favoritos, historial de reproducción para continuar donde se dejó (Seguir viendo), sistema de reseñas, soporte de subtítulos.
- Construir una aplicación web interactiva que ofrezca al usuario una navegación intuitiva y una buena experiencia visual.
- Desplegar el sistema completo de forma automatizada y replicable mediante contenedores para garantizar su portabilidad.
- Validar la lógica de negocio del sistema mediante pruebas automatizadas.

## 1.3 Estructura de la memoria

Este apartado describe la organización del contenido presente en esta memoria, proporcionando una visión general de los capítulos que la componen:

- **1. Introducción:** Se presenta el contexto general del proyecto, las motivaciones que impulsan su desarrollo y los objetivos principales y específicos que se pretenden alcanzar.

- **2. Estado del arte y fundamentos:** Se analizan las soluciones y plataformas VOD ya existentes y se detallan los lenguajes de programación, frameworks y herramientas empleadas para la construcción del sistema.
- **3. Metodología:** Se describe la metodología de trabajo seguida durante el desarrollo del proyecto y las herramientas de gestión y control de versiones empleadas.
- **4. Análisis:** Se detallan los requisitos funcionales y no funcionales del sistema, así como los casos de uso que definen su comportamiento.
- **5. Diseño:** Se presenta la arquitectura de microservicios del sistema, el modelo de datos de cada servicio y los patrones de diseño aplicados tanto en el backend como en el frontend.
- **6. Implementación:** Se explican los detalles técnicos más relevantes del desarrollo, así como las pruebas realizadas para validar la lógica de negocio del sistema.
- **7. Solución desarrollada:** Se muestra el resultado final de la aplicación mediante una visita guiada por sus principales funcionalidades.
- **8. Conclusiones y trabajo futuro:** Se valora el grado de cumplimiento de los objetivos planteados, la aportación personal del proyecto y las posibles líneas de ampliación futuras.

Capítulo 2

# Fundamentos tecnológicos

## 2.1 Estado del arte

El mercado actual de las plataformas de vídeo bajo demanda (VOD) se divide principalmente en dos grandes paradigmas: los servicios comerciales de suscripción global y las soluciones autoalojadas para la gestión de bibliotecas privadas. A continuación, se analizan las herramientas más representativas de ambos sectores:

**Netflix:** Es el pionero y líder indiscutible del mercado comercial del streaming. Fundado en 1997 como servicio de alquiler de DVDs evolucionó hasta convertirse en la referencia mundial del vídeo bajo demanda. Destaca por ofrecer un inmenso catálogo y un reproductor altamente optimizado. A nivel técnico, Netflix es un gran referente mundial en la adopción de arquitecturas orientadas a microservicios, su infraestructura se compone de cientos de servicios independientes que le permiten escalar de forma masiva y dar servicio a millones de usuarios concurrentes por todo el mundo sin que el sistema colapse. Sin embargo al tratarse de un servicio completamente cerrado, no permite al usuario gestionar su propio contenido local.

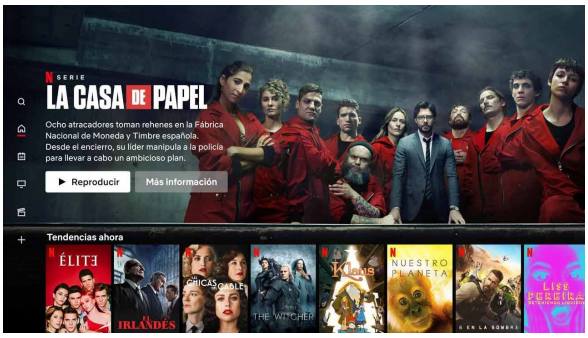



Figura 2.1: Interfaz web de usuario del catálogo de Netflix.

**Amazon Prime Video** es otro de los grandes referentes del sector, disponible en más de 240 países y territorios. Opera sobre la infraestructura en la nube de Amazon (AWS), lo

que le permite escalar de forma masiva y gestionar millones de reproducciones simultáneas. Ofrece contenido propio y de terceros, y permite la descarga de títulos para su reproducción sin conexión. Al igual que Netflix, es un servicio completamente cerrado que no permite al usuario gestionar su propio contenido local.

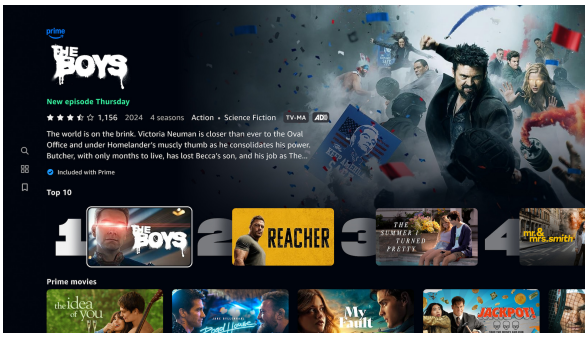



Figura 2.2: Interfaz web de usuario del catálogo de Amazon Prime Video.

**Plex:** En el ámbito de las soluciones autoalojadas (self-hosted), Plex es la plataforma más popular. Permite a los usuarios montar su propio servidor en casa utilizando sus archivos locales, lo cual da mucha flexibilidad al usuario permitiéndole seleccionar su contenido multimedia preferido. Ofrece una experiencia de usuario pulida y automatiza la descarga de metadatos (carátulas, sinopsis). Sin embargo, su código es privado y propiedad de Plex Inc., lo que impide que los usuarios puedan conocer su funcionamiento o modificar su arquitectura

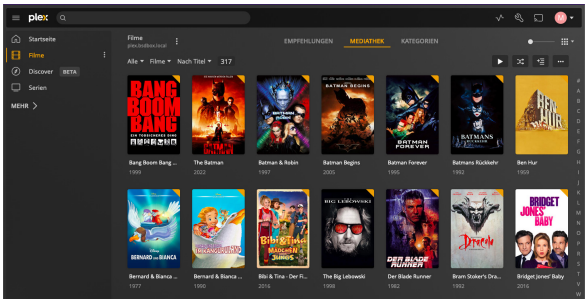



Figura 2.3: Interfaz web de usuario del catálogo de Plex.

**Jellyfin:** Surgió como una alternativa de código abierto cuando Emby (otro servidor de streaming) dejó de ser de código abierto. Resuelve el problema de la transparencia de Plex al ser totalmente libre, permitiendo a la comunidad revisar y modificar el código. Sin embargo, su diseño sigue siendo monolítico y centralizado.

**Comparativa con Vodeo:** Las herramientas analizadas representan los extremos del mercado. Los servicios comerciales como Netflix o Prime Video disponen de arquitecturas altamente escalables, pero son ecosistemas cerrados que no permiten al usuario gestionar su propio contenido local. Las soluciones autoalojadas como Plex o Jellyfin resuelven la gestión de

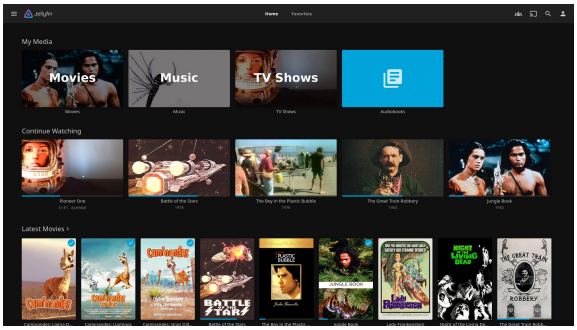



Figura 2.4: Interfaz web de usuario del catálogo de Jellyfin.

bibliotecas privadas e incluyen soporte de transcodificación de vídeo y reproducción adaptativa, pero dependen de arquitecturas monolíticas: al centralizar todas las operaciones en un único proceso, una tarea de transcodificación intensiva puede degradar el rendimiento global del sistema para el resto de usuarios.

Vodeo no pretende competir en funcionalidades con estas plataformas, sino abordar un objetivo distinto: construir desde cero una plataforma autoalojada cuya arquitectura de microservicios aísle el procesamiento de vídeo en un servicio independiente y desacoplado mediante un sistema de colas asíncrono. De este modo, la transcodificación de un archivo pesado no penaliza la navegación del resto de usuarios, algo que las arquitecturas monolíticas de Plex y Jellyfin no garantizan por diseño.

## 2.2 Tecnologías utilizadas

En el desarrollo de Vodeo se utilizaron las siguientes tecnologías:

### 2.2.1 Capa de servicios (Backend)

- **Java** [1]. Lenguaje de programación orientado a objetos, multiplataforma y de tipado estático, utilizado como lenguaje principal para el desarrollo de la lógica de negocio de los microservicios.
- **Spring Boot** [2]: Framework basado en Java utilizado como núcleo del backend. Integra módulos clave como Spring Security para la autenticación sin estado (JWT) y Spring Cloud Gateway para implemetar el API Gateway.
- **RabbitMQ** [3]: Intermediario de mensajes utilizado para orquestar la comunicación asíncrona, aislando la recepción de archivos multimedia de su posterior transcodificación.
- **FFmpeg** [4]: Aplicación multimedia de línea de comandos empleado en los workers asíncronos para transcodificar los vídeos a múltiples resoluciones bajo el protocolo HLS,

permitiendo la adaptación dinámica de la calidad (Adaptive Bitrate Streaming).

### 2.2.2 Capa de presentación (Frontend)

- **React y Vite** [5]: Herramientas para la construcción de la interfaz de usuario (SPA). React gestiona la reactividad y el estado, mientras que Vite optimiza drásticamente los tiempos de empaquetado y el servidor de desarrollo.
- **Tailwind CSS** [6]: Librería de estilos que permite diseñar elementos HTML aplicando clases cortas directamente, evitando tener que crear archivos CSS externos.

### 2.2.3 Almacenamiento de datos

- **PostgreSQL** [7]: Sistema de gestión de bases de datos relacional (SGBD) de código abierto. Cumple con el estándar SQL y permite realizar búsquedas de texto aproximadas o difusas mediante la extensión *pg_trgm*.
- **Redis** [8]: Motor de base de datos en memoria implementado como sistema de caché distribuida para reducir la latencia y la carga del servidor en las peticiones de lectura recurrentes del catálogo.

## 2.3 Herramientas de desarrollo

Se utilizaron diversas herramientas para facilitar la codificación, las pruebas y el despliegue del sistema:

- **Docker** [9]: Plataforma de contenerización empleada, junto con Docker Compose, para empaquetar cada microservicio en entornos aislados, garantizando la portabilidad y la reproducibilidad del despliegue en cualquier sistema.
- **Git** [10]: Sistema de control de versiones distribuido para la gestión del código fuente.
- **Postman** [11]: Herramienta para el diseño, depuración y validación de los endpoints de los microservicios.
- **Visual Studio Code** [12]: Editor de código fuente ligero y multiplataforma desarrollado por Microsoft. Destaca por su alta extensibilidad mediante el uso de extensiones y su integración nativa con sistemas de control de versiones.
- **JUnit 5 y Mockito** [13]: Frameworks utilizados para el diseño y ejecución de pruebas unitarias automatizadas sobre la lógica de negocio. Mockito se utiliza para simular dependencias externas (bases de datos, servicios), permitiendo probar cada servicio de forma aislada.

Capítulo 3

# Metodología y planificación

## 3.1 Metodología de desarrollo

Al construir una arquitectura de microservicios desde cero, es muy común que la estructura y los requisitos del sistema cambien a medida que avanza el proyecto, por lo que se descartó el uso de metodologías secuenciales como el modelo en cascada, cuya rigidez no permite adaptarse bien a los imprevistos. En su lugar, se utilizó una metodología de desarrollo ágil basada en el marco de trabajo **Kanban** [14].

Kanban es un método de gestión visual del trabajo basado en principios de entrega continua. Su principal característica no es trabajar mediante iteraciones estructuradas, sino la organización de las tareas en un tablero y la limitación del trabajo en curso (WIP, *Work In Progress*) [14]. Esto permite identificar y resolver cuellos de botella a tiempo. Para un proyecto individual como este TFG, resultó ser la opción ideal debido a que reduce la carga organizativa y aporta una enorme flexibilidad para adaptarse a los imprevistos y decisiones técnicas que surgen durante el ciclo de vida del desarrollo.

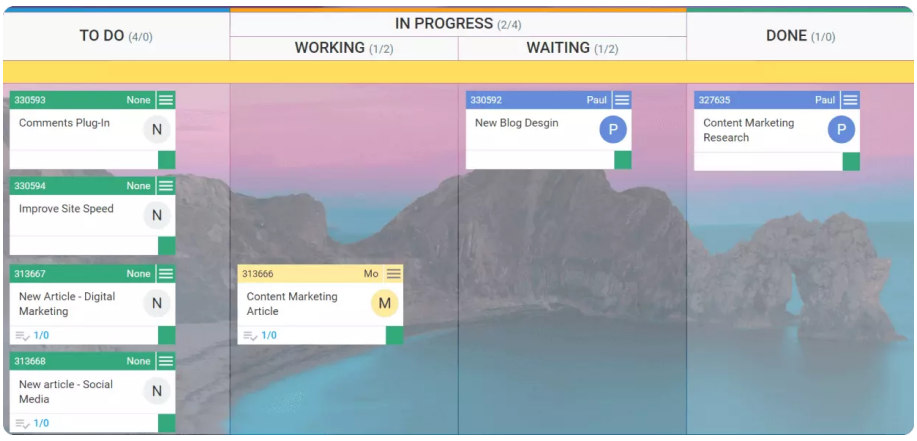



Figura 3.1: Ejemplo teórico de un tablero de Kanban.

### 3.1.1 Flujo de trabajo

La aplicación práctica de esta metodología se organizó mediante un flujo de trabajo estructurado. Cada nueva funcionalidad se desarrollaba siguiendo el mismo proceso:

Antes de escribir código, se utilizó **Draw.io** como herramienta de apoyo visual para diseñar los esquemas de bases de datos y la comunicación entre microservicios en cada módulo complejo. Esta fase de diseño previa facilitó la comprensión del sistema y el desarrollo posterior.

Una vez definido este diseño, cada necesidad del sistema se registraba como un Issue en GitHub, con un título que describía el requisito concreto (por ejemplo, "feat: implement autoplay next episode functionality for series"). Gracias a GitHub Projects, al crear un Issue, este se sincronizaba y aparecía automáticamente en la columna inicial (Backlog) del tablero de Kanban.

Cuando se decidía comenzar a trabajar en una tarea, la tarjeta correspondiente se trasladaba manualmente a la columna In Progress. Después se creaba una rama de desarrollo específica para ese Issue en Git, aislada de la rama principal (main), aplicando la estrategia de Feature Branching. Cuando varias tareas presentaban una fuerte dependencia entre sí o formaban parte de una misma funcionalidad, se desarrollaban dentro de una misma rama para optimizar el desarrollo.

Durante el proceso de desarrollo, cada cambio relevante en el código se registró siguiendo la convención **Conventional Commits** [15]. El uso de prefijos como feat para las nuevas funcionalidades, fix para correcciones de errores o refactor para modificaciones estructurales de código permitió mantener un historial de cambios ordenado y coherente.

Al redactar los commits, se incluía la referencia al identificador del Issue (por ejemplo, #15). Gracias a esta práctica, una vez comprobada la funcionalidad, al integrar la rama en main mediante una Pull Request, la tarjeta se movía de forma automática a la columna Done del tablero si el commit cerraba el Issue con la palabra clave "Closes".

Posteriormente, la tarea finalizaba su ciclo al ser desplazada a la columna Documented. Este estado indicaba que los detalles técnicos, los problemas resueltos y las decisiones tomadas en el Issue habían sido recogidos en un documento interno de seguimiento ("Memoria sobre issues"). Como resultado de la aplicación de este flujo de trabajo, se gestionaron un total de 56 issues y 37 Pull Requests a lo largo del proyecto.

Para visualizar el estado del proyecto en todo momento, las columnas del tablero de Kanban se estructuraron de la siguiente manera:

- **Backlog:** Tareas pendientes de iniciar, incorporadas automáticamente al crear los issues.
- **In Progress:** Tareas en fase de implementación.
- **Done:** Tareas finalizadas e integradas en la rama principal.

- **Documented:** Tareas completadas cuya explicación técnica ha sido escrita en la memoria de trabajo y quedan archivadas.

La Figura 3.2 muestra el estado final del tablero al cierre del proyecto.

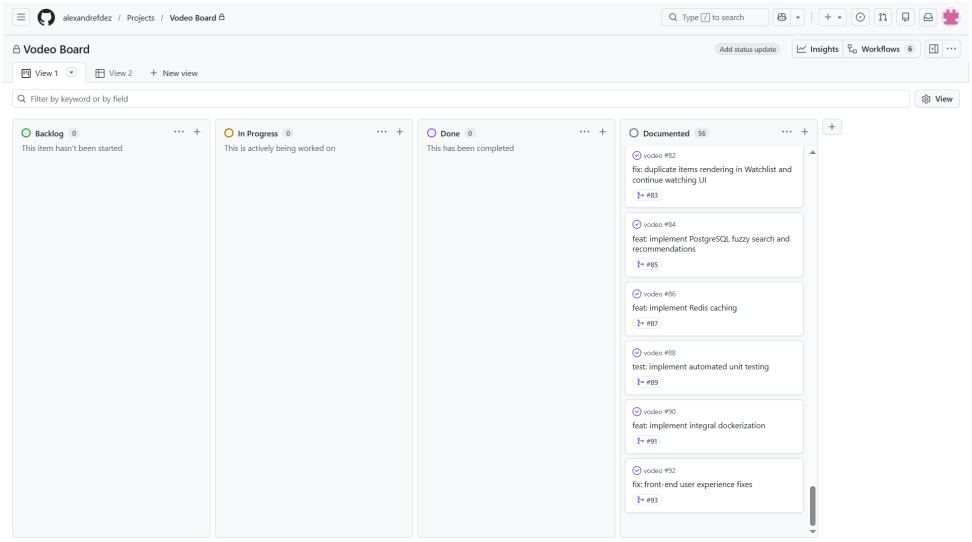



Figura 3.2: Tablero Kanban del proyecto Vodeo en GitHub Projects.

## 3.2 Planificación

Para abordar el desarrollo de la plataforma, se dividió el desarrollo en varios bloques funcionales que permitieron avanzar de forma ordenada y hacer un seguimiento claro del progreso.

### 3.2.1 Recursos

El proyecto fue desarrollado por un único desarrollador, que ha asumido los roles de analista, arquitecto de sistemas y desarrollador Full-Stack, así como las tareas de pruebas y despliegue. Tomando como referencia el salario medio de un ingeniero de software *junior* en España (20 €/hora) y una dedicación estimada de 250 horas, el coste asociado al personal se fijó en 5.000 € [16].

Para el desarrollo de la plataforma se ha utilizado un ordenador portátil Lenovo IdeaPad Gaming 3 15IMH05, cuyo valor total de mercado es de 800 €. A nivel de software, se ha apostado por un ecosistema de tecnologías *Open Source*, lo que evitó la dependencia de licencias comerciales (0 €) y reduce los costes del proyecto.

### 3.2.2 Fases del proyecto planificadas

El desarrollo del proyecto se planificó inicialmente en seis fases principales, antes de comenzar el proyecto:

- **Fase 1. Diseño y planificación** (28 abril–13 mayo): Definición de la arquitectura, selección del stack tecnológico, esquematización de la base de datos, preparación del entorno de desarrollo y planificación de tareas (fase previa a la escritura de código).
- **Fase 2. Servicios base del backend** (14–19 mayo): Implementación del servicio de usuarios (*users-service*) con autenticación JWT, el catálogo (*catalog-service*) con integración de APIs externas de metadatos (TMDB/OMDb), junto con la infraestructura inicial del servicio de streaming (*streaming-service*) con chunked streaming.
- **Fase 3. Frontend y experiencia del usuario base** (20-23 mayo): Construcción de la interfaz web principal (Single Page Application), conectando el sistema de autenticación, el catálogo, el reproductor multimedia y el panel de administración inicial.
- **Fase 4. Funcionalidades avanzadas** (24 mayo–28 mayo): Desarrollo de la gestión de perfiles de usuario, lista de favoritos (watchlist), soporte completo para series de televisión, sistema de reseñas, historial de reproducción y filtros de búsqueda en el catálogo.
- **Fase 5. Motor asíncrono HLS/ABS** (29 mayo–3 junio): Desarrollo del procesamiento de vídeo para la transmisión adaptativa a múltiples calidades, junto con soporte de subtítulos.
- **Fase 6. Optimización y cierre** (4–6 junio): Implementación de caché distribuida con Redis, búsqueda difusa con `pg_trgm`, pruebas unitarias automatizadas, dockerización integral y documentación de la API.

### 3.2.3 Diagrama de Gantt inicial

La Figura 3.3 muestra la planificación inicial estimada antes de comenzar el desarrollo del proyecto.

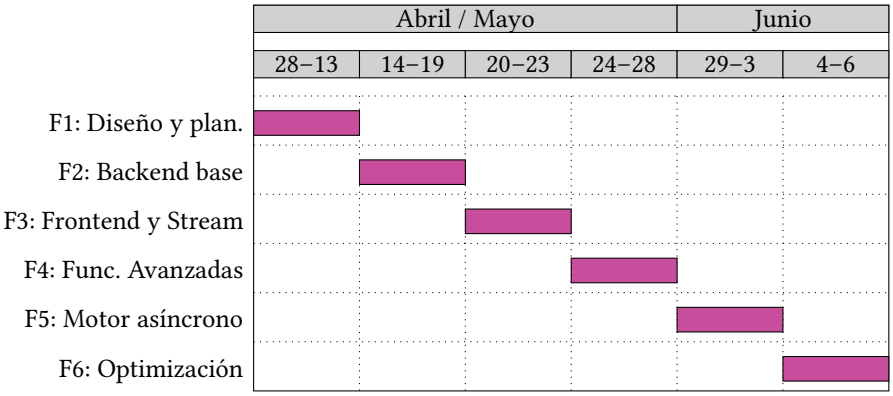



Figura 3.3: Diagrama de Gantt con la planificación inicial estimada.

### 3.2.4 Estimación de costes inicial

Con esta planificación de 40 días naturales y una dedicación media estimada de 6,25 horas diarias, se proyectaron **250 horas** de trabajo que, como se señalo en la sección de recursos, equivalen a un coste de personal de **5.000 €**.

En cuanto a los recursos materiales, se contabilizó el ordenador portátil Lenovo IdeaPad Gaming 3 15IMH05, valorado en 800 €. A efectos de esta estimación, se ha calculado un coste de amortización de 35 € proporcional al tiempo de desarrollo del proyecto.

Sumando el coste de personal (5.000 €) y la amortización del equipo (35 €), el presupuesto inicial total del proyecto se situó en **5.035 €**.

## 3.3 Seguimiento

Durante el desarrollo real del proyecto se produjeron dos desviaciones respecto a la planificación inicial, ambas gestionadas con éxito gracias a la flexibilidad de la metodología Kanban.

### 3.3.1 Desviaciones

**Desviación 1 — Fase 2 (retraso de 7 días).** Durante la segunda quincena de mayo, el ritmo de desarrollo se redujo por la coincidencia con el período de exámenes finales del grado. La fase, planificada para concluir el 19 de mayo, se extendió hasta el 26 de mayo.

**Desviación 2 — Fase 5 (retraso de 4 días).** En el diseño inicial, la transmisión de vídeo se había planificado mediante *chunked streaming* síncrono. Sin embargo, al evolucionar hacia un sistema de *Adaptive Bitrate Streaming* (ABS), se comprobó que transcodificar archivos pesados de forma síncrona bloqueaba los hilos del servidor. Fue necesario integrar RabbitMQ

como infraestructura de mensajería y aislar la transcodificación en un microservicio *worker* independiente, lo que amplió la duración de esta fase en 4 días de desarrollo adicionales.

Como consecuencia de ambas desviaciones, el proyecto se retrasó un total de **11 días** sobre la estimación inicial, finalizando el 17 de junio en lugar del 6 de junio previsto.

### 3.3.2 Diagrama de Gantt real

La Figura 3.4 muestra la distribución temporal real del proyecto, incorporando ambas desviaciones.

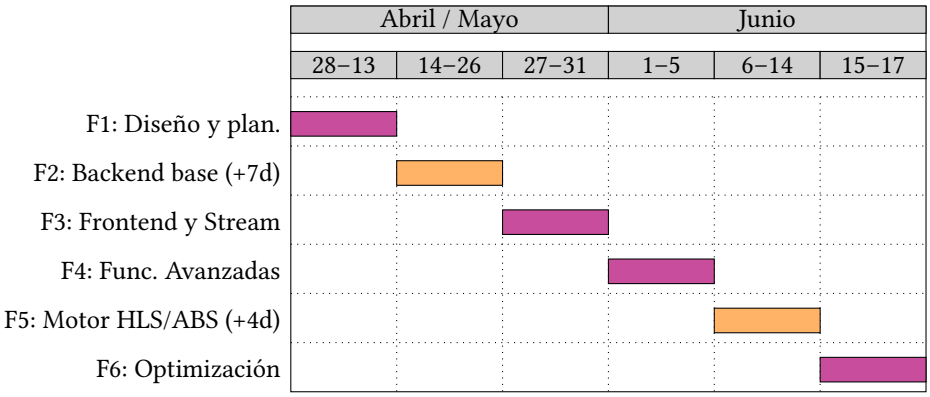



Figura 3.4: Diagrama de Gantt con la distribución real de las fases del proyecto.

### 3.3.3 Impacto en costes

Hay que diferenciar el tiempo de calendario del impacto en el esfuerzo. El proyecto se llevó a cabo a lo largo de un total de 51 días naturales. La **Desviación 1** (7 días) consistió en una pausa académica temporal que no sumó horas de trabajo.

Sin embargo, el esfuerzo técnico adicional requerido por la **Desviación 2** (infraestructura asíncrona) elevó la dedicación real a **275 horas**, frente a las 250 horas operativas planificadas.

La Tabla 3.1 recoge el impacto económico de estas nuevas 25 horas de desarrollo.

| **Concepto** | **Horas** | **Planificado (€)** | **Real (€)** |
|---|---|---|---|
| Desarrollador | 250 h / 275 h | 5.000,00 | 5.500,00 |
| Hardware | — | 35,00 | 35,00 |
| Licencias y software | — | 0,00 | 0,00 |
| **Total** | — | **5.035,00** | **5.535,00** |

Tabla 3.1: Comparativa de costes planificados y reales del proyecto.

La desviación total supuso un incremento de **500 €** sobre el presupuesto inicial.

# Capítulo 4

# Análisis

Este capítulo detalla el proceso de análisis del sistema Vodeo. En primer lugar, se definen los requisitos funcionales y no funcionales que la plataforma debe satisfacer para cumplir con sus objetivos. A continuación, se presenta la arquitectura del sistema, se diseña la interfaz de usuario y se establece el modelo de datos que soportará la persistencia de la información. Finalmente, se describe la interfaz de los servicios web que ofrece el sistema.

## 4.1 Requisitos

### 4.1.1 Actores del sistema

En la plataforma interactúan tres actores principales: dos son usuarios del sistema y el tercero es un servicio externo que proporciona metadatos sobre el contenido multimedia.

- **Usuario:** Consumidor final de la plataforma. Debe estar registrado en el sistema y puede navegar por el catálogo, buscar contenido, reproducir vídeos, gestionar su lista de favoritos, publicar reseñas y continuar visualizaciones pendientes desde su historial.
- **Administrador:** Gestor de la plataforma. Posee todos los privilegios del Usuario estándar, además de permisos exclusivos para acceder al panel de control del administrador, subir y gestionar contenido multimedia, y administrar las cuentas del resto de usuarios.
- **Sistemas externos (TMDB/OMDb):** Servicios que proporcionan metadatos sobre el contenido multimedia (sinopsis, carátulas, géneros, duración, reparto).

### 4.1.2 Requisitos funcionales

#### Gestión de usuarios y seguridad

- **RF-01. Registro y autenticación:** El sistema debe permitir a los usuarios registrarse con nombre de usuario, correo electrónico y contraseña, e iniciar sesión mediante credenciales. Las contraseñas se almacenarán cifradas.
- **RF-02. Control de acceso por roles:** El sistema debe distinguir entre los roles *USER* y *ADMIN*, restringiendo las funcionalidades de administración exclusivamente al segundo.
- **RF-03. Gestión de perfil:** Un usuario autenticado debe poder actualizar su nombre de usuario y su contraseña, previa verificación de la contraseña actual.
- **RF-04. Gestión de usuarios (administración):** El administrador debe poder consultar el listado completo de usuarios registrados y eliminar cuentas. La eliminación de un usuario debe borrar en cascada sus datos privados (favoritos, historial), preservando de forma anónima sus reseñas publicadas en el catálogo.

**Gestión del catálogo multimedia**

- **RF-05. Subida de contenido:** El administrador debe poder subir archivos de vídeo correspondientes tanto a películas como a episodios de series.
- **RF-06. Sincronización de metadatos:** El sistema debe permitir importar automáticamente los metadatos del contenido consultando APIs externas de bases de datos cinematográficas, con un mecanismo de respaldo entre ambas fuentes ante fallos de disponibilidad. El administrador debe poder sobrescribir manualmente cualquier metadato importado.
- **RF-07. Organización jerárquica de series:** El sistema debe permitir estructurar el contenido de series en temporadas y episodios, cada uno con su propio archivo de vídeo y metadatos independientes.
- **RF-08. Exploración y búsqueda:** El usuario debe poder ver el contenido disponible de forma paginada, filtrarlo por género y realizar búsquedas por título mediante coincidencias aproximadas.
- **RF-09. Sistema de reseñas:** El usuario debe poder publicar una valoración y un comentario sobre el contenido. El administrador puede eliminar cualquier reseña.
- **RF-10. Panel de estadísticas:** El administrador debe disponer de un panel con métricas sobre la cantidad de contenido y usuarios registrados en el sistema.

**Reproducción y streaming**

- **RF-11. Transcodificación asíncrona:** Al subir un nuevo vídeo, el sistema debe procesarlo en segundo plano sin bloquear la disponibilidad de la plataforma, generando múltiples variantes de calidad.
- **RF-12. Reproducción adaptativa:** El sistema debe ofrecer un reproductor web que ajuste automáticamente la calidad del vídeo en tiempo real según las condiciones de la red.
- **RF-13. Soporte de subtítulos:** El reproductor debe permitir activar, desactivar y seleccionar pistas de subtítulos en distintos idiomas, tanto en películas como en episodios.
- **RF-14. Reproducción continua:** En el caso de las series, el sistema debe ofrecer reproducción automática del siguiente episodio al finalizar el actual.

**Interacción del usuario**

- **RF-15. Lista de favoritos:** El usuario debe poder añadir y eliminar tanto películas como series de su lista personal (*watchlist*).
- **RF-16. Historial de reproducción:** El sistema debe registrar el progreso de reproducción de cada usuario, permitiendo reanudar la visualización desde el punto exacto donde se dejó, y mostrar un listado de los contenidos que tiene pendientes.
- **RF-17. Recomendaciones:** El sistema debe sugerir contenido relacionado a partir del historial de visualización del usuario, excluyendo el contenido ya visualizado.

### 4.1.3 Casos de uso

A continuación se detallan dos de los casos de uso más representativos del sistema, correspondientes a los flujos de mayor complejidad técnica de la plataforma.

#### CU-01. Subir y procesar vídeo

**Descripción:** Permite al administrador subir un archivo de vídeo asociado a una película o episodio, desencadenando su procesamiento asíncrono para generar múltiples variantes de calidad.

**Precondición:** El administrador debe estar autenticado con el rol *ADMIN*. La película o episodio de destino debe existir previamente en el catálogo.

**Secuencia normal:**

1. El administrador selecciona un archivo de vídeo desde el panel de administración.
2. El sistema recibe el archivo y lo almacena de forma secuencial.

3. El sistema asigna un identificador único al archivo y devuelve su ruta física al panel de administración.
4. El sistema publica un evento en la cola de mensajería, indicando que hay un nuevo archivo pendiente de procesar.
5. El servicio de transcodificación procesa el evento de forma asíncrona y ejecuta el motor de conversión de vídeo.
6. El sistema genera tres variantes de calidad (1080p, 720p y 480p) empaquetadas bajo un índice de reproducción adaptativo.
7. Una vez completado el procesamiento, el archivo original se elimina y las variantes generadas quedan disponibles para su reproducción.

**Excepciones:**

- El servicio de transcodificación no encuentra el archivo de origen en la ruta esperada → el evento se descarta y el incidente queda registrado en los logs sin afectar a la disponibilidad del resto del sistema.
- El proceso de transcodificación falla → el archivo original se conserva en disco y el incidente queda registrado en el sistema, sin afectar a la disponibilidad del resto de la plataforma.

**Postcondición:** El contenido queda disponible para su reproducción en el catálogo.

**CU-02. Reproducir contenido con calidad adaptativa**

**Descripción:** Permite a un usuario reproducir una película o episodio, ajustando automáticamente la calidad del vídeo según las condiciones de su red.

**Precondición:** El usuario debe estar autenticado. El contenido solicitado debe haber completado su procesamiento.

**Secuencia normal:**

1. El usuario accede a la ficha de una película o episodio y pulsa el botón de reproducción.
2. El reproductor solicita el índice de reproducción adaptativo del contenido al servicio de streaming.
3. El sistema valida el acceso y responde con el índice correspondiente.
4. El reproductor estima el ancho de banda disponible y selecciona automáticamente la variante de calidad más adecuada.

5. El contenido se transmite por fragmentos, permitiendo que la reproducción comience sin necesidad de descargar el archivo completo.

6. Si las condiciones de red cambian durante la reproducción, el reproductor cambia de variante de calidad sin que el usuario tenga que intervenir.

**Excepciones:**

- El token de acceso no es válido o ha caducado → el sistema rechaza la petición y el usuario es redirigido a la pantalla de inicio de sesión.
- El archivo físico no se encuentra disponible en el servidor → el reproductor muestra un mensaje de error indicando que el contenido no puede reproducirse en ese momento.

**Postcondición:** El usuario visualiza el contenido con la calidad de vídeo óptima para su conexión.

### 4.1.4 Requisitos no funcionales

A continuación se definen los requisitos no funcionales del sistema, orientados a garantizar que la plataforma de streaming sea robusta, eficiente, segura y mantenible a largo plazo:

- **RNF-01. Escalabilidad:** La arquitectura debe permitir que los procesos intensivos, como la transcodificación de vídeo, se ejecuten de forma aislada, sin penalizar la navegación del resto de usuarios concurrentes.
- **RNF-02. Rendimiento:** Las consultas de lectura frecuentes sobre el catálogo deben responder con baja latencia, apoyándose en una capa de caché en memoria, y evitar problemas de rendimiento derivados de un número excesivo de consultas a la base de datos.
- **RNF-03. Seguridad:** Las contraseñas deben almacenarse cifradas y toda comunicación con rutas protegidas debe autenticarse mediante tokens de seguridad (JWT). Además, el tráfico externo debe canalizarse a través de un único punto de entrada que oculte la estructura interna del sistema.
- **RNF-04. Fiabilidad:** La eliminación de una entidad en un servicio debe propagar de forma fiable la eliminación de sus recursos asociados en otros servicios, incluso ante fallos temporales de disponibilidad de estos.
- **RNF-05. Portabilidad:** El sistema completo debe poder desplegarse en cualquier entorno mediante un único comando, de forma independiente, sin necesidad de instalar bases de datos o programas externos, basándose únicamente en el código fuente del proyecto.

- **RNF-06. Mantenibilidad:** El código debe organizarse en capas independientes por responsabilidad, con un tratamiento de errores uniforme en toda la plataforma.
- **RNF-07. Usabilidad:** La interfaz web debe funcionar de forma fluida y responsiva, adaptándose tanto a dispositivos de escritorio como móviles, sin recargas de página durante la navegación.

## 4.2 Arquitectura del sistema

Vodeo adopta una **arquitectura basada en microservicios**, en la que cada responsabilidad del sistema se encapsula en un componente independiente con su propia lógica de negocio y capa de persistencia. Esta decisión permite escalar, desplegar y mantener cada servicio de forma autónoma, evitando el fuerte acoplamiento propio de las arquitecturas monolíticas.

El sistema se estructura en tres niveles principales:

- **Nivel de presentación (Frontend):** Formado por una *Single Page Application* (SPA). Toda la comunicación de esta aplicación con el servidor pasa por un *API Gateway*. Este componente actúa como un proxy inverso y punto de entrada único; recibe las peticiones del cliente y las redirige al microservicio correspondiente, mejorando la seguridad y centralizando el acceso.
- **Nivel de servicios (Backend):** Es el núcleo de la plataforma, compuesto por cuatro microservicios independientes. Cada uno gestiona un dominio concreto:
  - *Servicio de Usuarios*: Gestiona la autenticación, las cuentas de usuario y sus preferencias (favoritos e historial).
  - *Servicio de Catálogo*: Gestiona la información multimedia, los metadatos y las reseñas.
  - *Servicio de Streaming*: Se encarga de la subida de los archivos y de transmitir el vídeo al reproductor del usuario y gestionar el almacenamiento físico.
  - *Servicio de Transcodificación*: Realiza la conversión de vídeo a múltiples calidades en segundo plano.
- **Nivel de persistencia:** Se emplea un único servidor de base de datos relacional que es compartido por los servicios del backend. Sin embargo, para mantener el desacoplamiento propio de los microservicios, se aplica un aislamiento lógico a nivel de tablas: el *Servicio de Usuarios* y el *Servicio de Catálogo* gestionan sus propias entidades de forma independiente y no existen consultas cruzadas ni claves foráneas entre ambos dominios. Por su parte, el *Servicio de Streaming* y el *Servicio de Transcodificación* guardan los archivos físicos

directamente en el disco. Además, se utiliza una base de datos en memoria como caché distribuida para agilizar las operaciones de lectura más frecuentes del catálogo.

Para que los componentes se comuniquen de forma eficiente, la arquitectura utiliza dos estrategias principales.

En primer lugar, se emplea la **comunicación asíncrona mediante colas de mensajería**. Para tareas pesadas como el procesamiento de vídeo, el *Servicio de Streaming* publica un evento en un intermediario de mensajes. Esto permite que el *Servicio de Transcodificación* recoja este evento y procese el archivo en segundo plano sin bloquear las peticiones HTTP del resto de usuarios.

En segundo lugar, se utiliza el patrón de **composición de API en el cliente (*API Composition*)**. Dado que los datos de usuarios y catálogo están aislados lógicamente en la persistencia, las funcionalidades que requieren cruzar información (como cargar la lista de favoritos) se resuelven en el nivel de presentación. La aplicación web asume el rol de orquestador, realizando las peticiones asíncronas necesarias a cada microservicio y uniendo la información final en la interfaz del usuario.

La Figura 4.1 ilustra la comunicación general entre los distintos componentes del sistema.

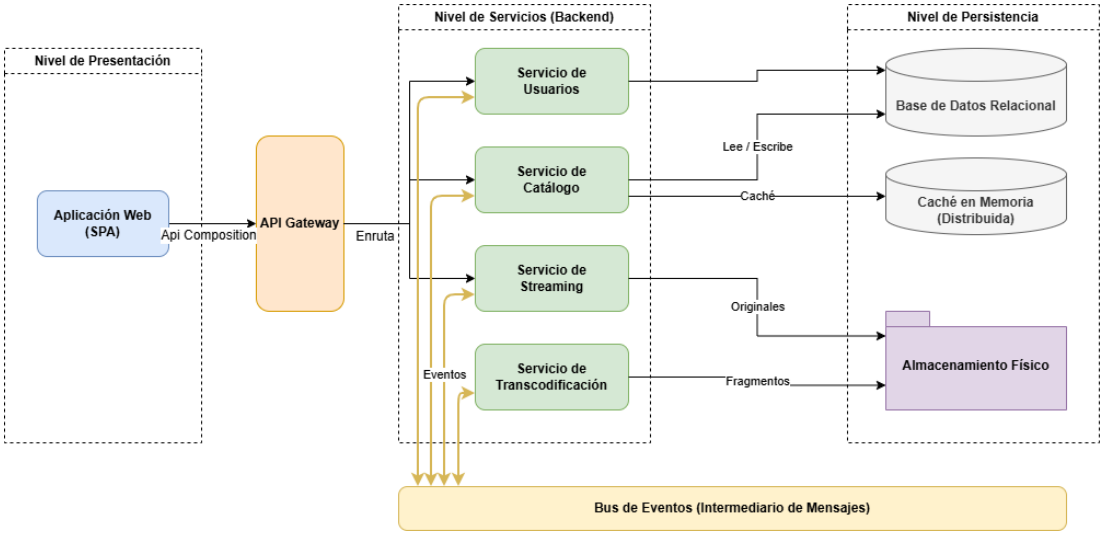



Figura 4.1: Arquitectura de alto nivel del sistema Vodeo.

## 4.3 Interfaz de usuario

La interfaz de usuario de Vodeo se concibe como una aplicación web de página única, en la que la navegación entre pantallas no implica recargas completas del navegador. La estructura de pantallas y su flujo de navegación se describen a continuación.

- **Login y Registro**: Pantallas públicas de acceso a la plataforma. Un usuario no autenticado que intente a acceder a cualquier otra pantalla es redirigido automáticamente aquí.
- **Catálogo:** Pantalla principal tras el inicio de sesión. Presenta el contenido disponible organizado en una cuadrícula visual, con controles de búsqueda, filtrado por categoría y navegación paginada. Incluye además carruseles de contenido recomendado y de continuación de reproducción.
- **Detalle de contenido:** Ficha ampliada de una película o serie concreta, con su información descriptiva, las reseñas de otros usuarios y el acceso a la reproducción. En el caso de las series, incluye además la navegación por temporadas y episodios. Adicionalmente, esta es la pantalla desde la cual el rol administrador sube los archivos de vídeo individuales de cada episodio de las series.
- **Reproductor:** Pantalla de reproducción multimedia, accesible tanto para películas como para episodios, con selección de subtítulos y reproducción automática del siguiente episodio en el caso de series.
- **Mi Lista:** Pantalla con el contenido guardado por el usuario en su lista de favoritos personal, combinando películas y series.
- **Perfil:** Pantalla de gestión de los datos personales del usuario autenticado.
- **Panel de administración:** Conjunto de pantallas accesibles exclusivamente para el rol *ADMIN*, que incluyen la subida de nuevo contenido al catálogo (con importación automática de metadatos), la gestión de usuarios registrados y un panel con métricas agregadas del sistema.

La navegación entre estas pantallas parte del Catálogo como punto central, desde el cual se accede al resto de funcionalidades. El acceso a las pantallas del Panel de administración está protegido mediante un control de rutas que verifica el rol del usuario, y resulta inaccesible para cuentas sin privilegios de administrador.

Para ilustrar este flujo, la Figura 4.2 detalla el árbol de navegación completo del sistema.

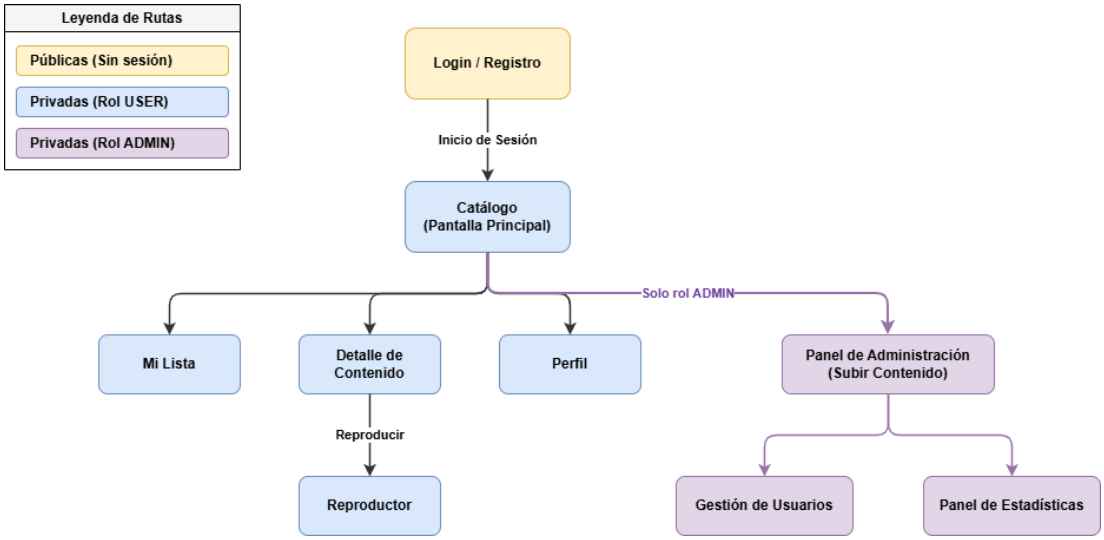



Figura 4.2: Mapa de navegación de la interfaz de usuario de Vodeo.

A modo de ejemplo visual de la plataforma, se incluyen a continuación los prototipos correspondientes al Catálogo y al Detalle de contenido. Los prototipos detallados del resto de pantallas de la aplicación se incluyen en el Anexo A.

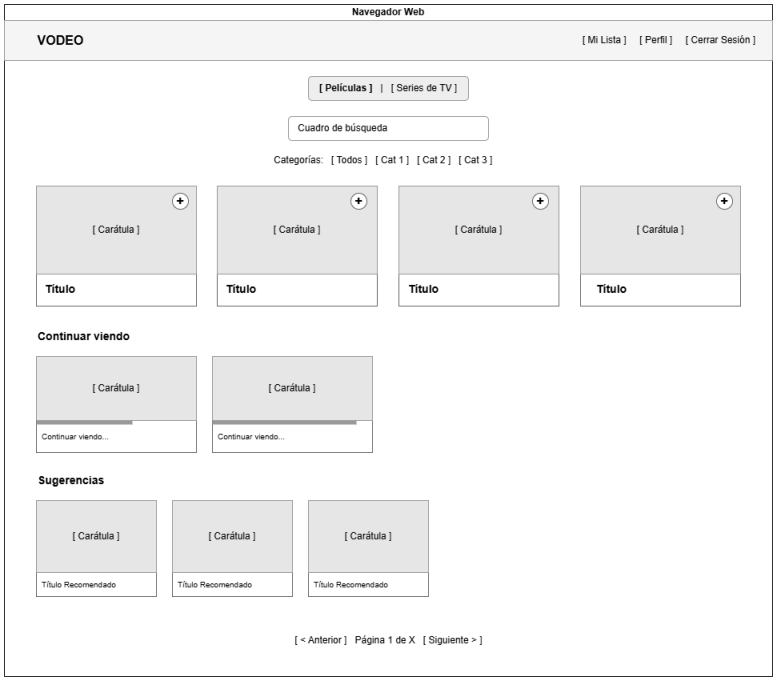



Figura 4.3: Prototipo de la pantalla de Catálogo.

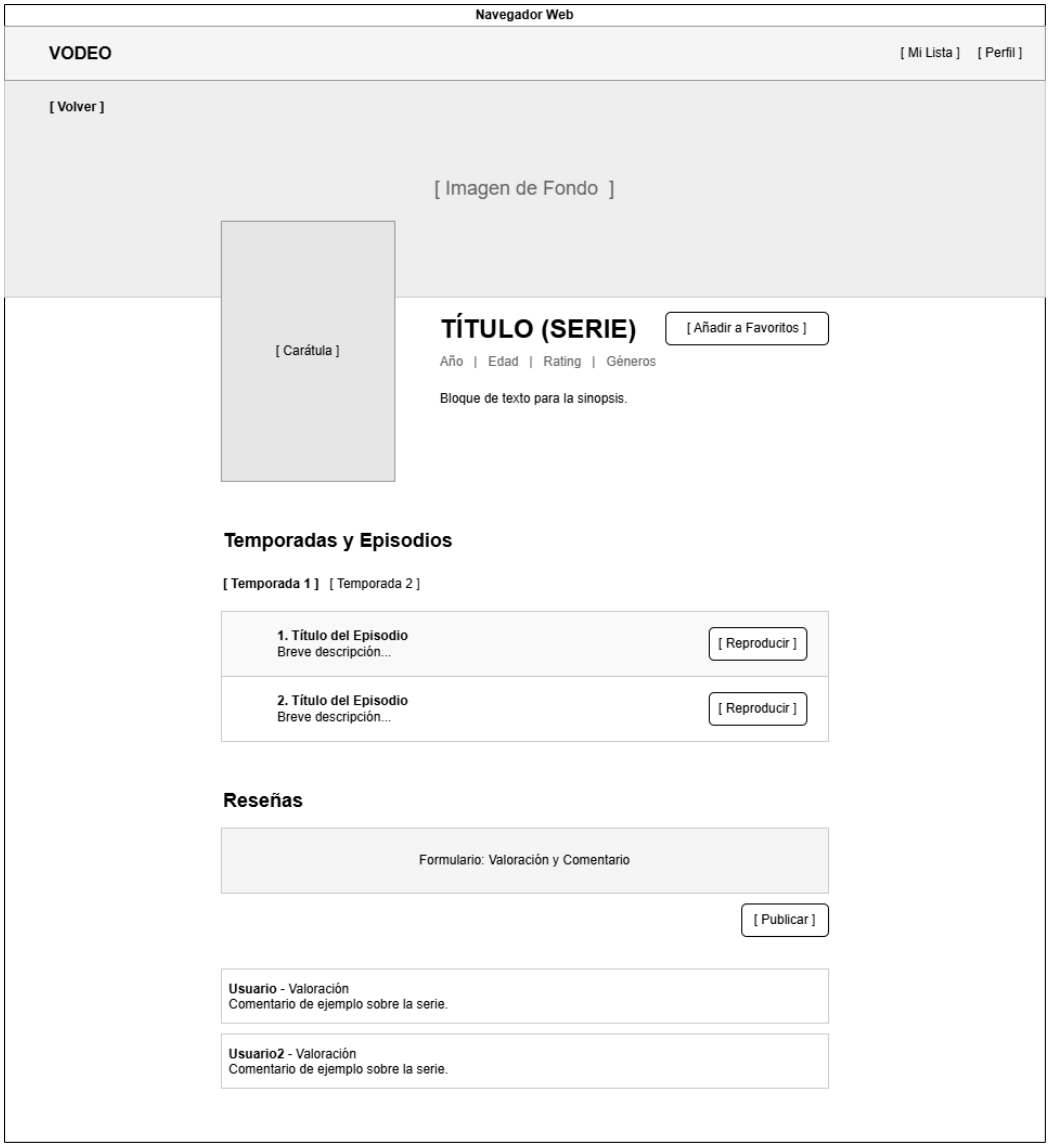



Figura 4.4: Prototipo de la pantalla de Detalle de contenido (Serie).

## 4.4 Modelo conceptual de datos

El modelo conceptual de datos define de manera estructurada y formal la información que gestiona el sistema, abstrayendo los detalles de implementación física. Para este propósito, se ha diseñado un diagrama Entidad-Relación (E-R), donde se representan las entidades, sus atributos y las interacciones entre ellas.

### 4.4.1 Entidades y relaciones del sistema

El dominio de datos de la plataforma se organiza a través de las siguientes entidades principales y sus relaciones:

- **User**: Almacena la información de los usuarios del sistema, incluyendo sus credenciales de acceso, correo electrónico y rol. **Relaciones:**
  - *WatchlistItem*: Relación 1:N (*Has*). Un usuario puede tener múltiples elementos guardados en su lista de seguimiento.
  - *PlaybackHistory*: Relación 1:N (*Logs*). Un usuario genera múltiples registros en su historial.
  - *Review*: Relación 1:N (*Writes*). Un usuario puede escribir múltiples reseñas.
- **Movie** y **Series**: Representan el contenido audiovisual principal del catálogo. **Relaciones:**
  - *Genre*: Relación M:N (*Classifies*). El contenido se asocia a múltiples géneros, y un género agrupa múltiples contenidos.
  - *Review*: Relación 1:N (*Rates*). El contenido puede recibir múltiples reseñas por parte de los usuarios.
- **WatchlistItem** y **PlaybackHistory**: Actúan como conexión entre el usuario y el catálogo. *WatchlistItem* representa un elemento individual guardado para ver más tarde, guardando la referencia al contenido (*Movie* o *Series*) mediante una relación N:1 (*Saves*). Por su parte, *PlaybackHistory* registra el progreso temporal sobre un contenido (*Movie* o *Episode*) mediante una relación N:1 (*Watches*).
- **Review**: Almacena las valoraciones y los comentarios asociados a las películas y series. Se relaciona con el usuario que escribe reseñas mediante una relación 1:N *Writes* y con el contenido a valorar mediante una relación N:1 *Rates*.
- **Season**: Representa las temporadas en las que se estructura una serie. Posee una relación de dependencia existencial y participación total respecto a la entidad **Series** mediante una relación 1:N (*Composes*).
- **Episode**: Representa los capítulos individuales. Posee una dependencia existencial y participación total respecto a **Season** mediante una relación 1:N (*Contains*).
- **Subtitle**: Almacena las pistas de texto en distintos idiomas. Se asocia a una película o a un episodio mediante una relación 1:N (*Translates*).

La Figura 4.5 muestra el diagrama entidad-relación del sistema.

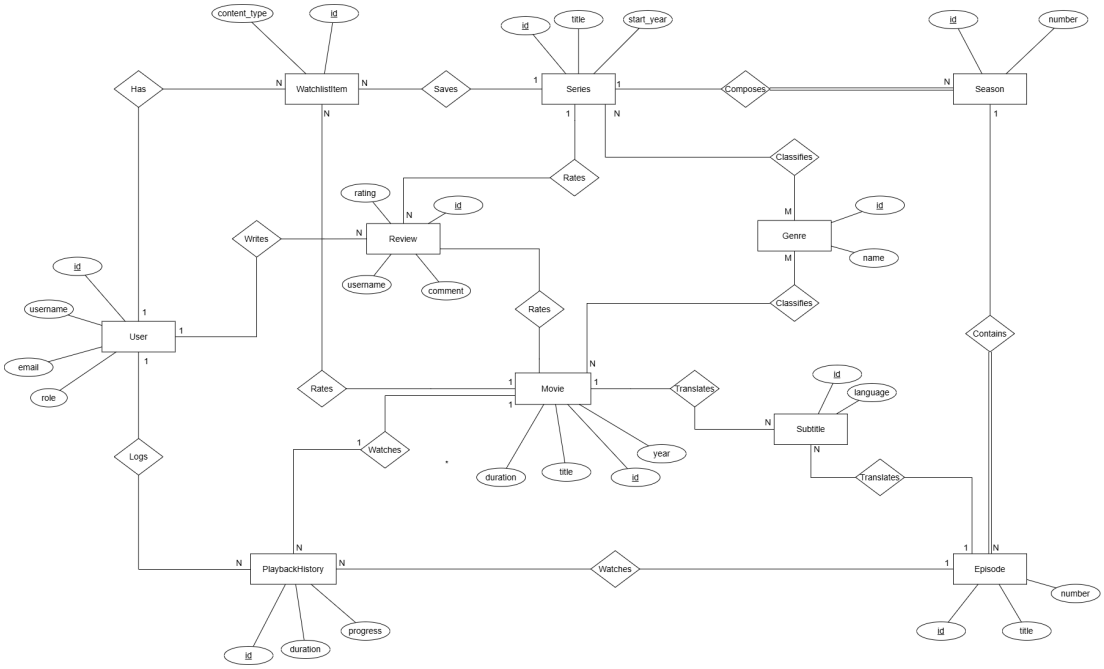



Figura 4.5: Diagrama entidad-relación de la plataforma Vodeo.

## 4.5 Interfaz de los servicios

Dado que la arquitectura del sistema se fundamenta en un modelo de microservicios, la comunicación entre la aplicación cliente y el *backend* se realiza a través de interfaces web de tipo REST. Estas interfaces definen un contrato de comunicación claro y uniforme que establece los límites de interacción con el sistema.

Para ello, todas las peticiones del cliente se canalizan a través de un punto de entrada único (*API Gateway*) que actúa como proxy inverso y enruta cada solicitud al servicio correspondiente según el dominio funcional al que pertenezca.

La Tabla 4.1 resume los grupos funcionales expuestos mediante la API y el nivel de acceso requerido para consumirlos.

| Grupo funcional | Operaciones expuestas | Nivel de acceso |
|---|---|---|
| Autenticación | Registro de cuentas e inicio de sesión. | Público |
| Perfil de usuario | Consulta de datos y actualización de credenciales. | Usuario registrado |
| Catálogo (lectura) | Listado paginado, búsqueda por similitud y detalles. | Usuario registrado |
| Catálogo (escritura) | Publicación, eliminación e importación de metadatos. | Administrador |
| Estructura de series | Gestión jerárquica de temporadas y episodios. | Administrador |
| Lista de favoritos | Agregación, eliminación y consulta de *watchlist*. | Usuario registrado |
| Historial de visionado | Registro de progreso temporal y consulta de estado. | Usuario registrado |
| Reseñas y valoraciones | Publicación, consulta y moderación (eliminación). | Usuario / Admin. |
| Ingesta multimedia | Recepción de archivos de vídeo de gran tamaño. | Administrador |
| Transmisión (Streaming) | Entrega de fragmentos de vídeo y pistas de subtítulos. | Usuario registrado |
| Gestión de usuarios | Listado de cuentas registradas y eliminación. | Administrador |
| Panel analítico | Consulta de métricas globales del sistema. | Administrador |

Tabla 4.1: Grupos funcionales expuestos por la interfaz de servicios de Vodeo.

En cuanto a la seguridad de la interfaz, las operaciones que no son públicas exigen incluir un token JWT en la cabecera de la petición HTTP. Por otro lado, las respuestas del sistema siguen siempre el mismo formato: si la petición es correcta, se devuelven los datos solicitados en formato JSON; si falla, se devuelve un mensaje de error junto con el código de estado HTTP correspondiente. Por último, la documentación de la API se genera de forma automática utilizando el estándar OpenAPI.

# Capítulo 5

# Diseño

Este capítulo describe las decisiones de diseño más relevantes adoptadas durante el desarrollo de Vodeo, estableciendo la trazabilidad entre los requisitos y casos de uso definidos en el Capítulo 4 y su implementación concreta. Se detalla la arquitectura tecnológica del sistema, la organización interna del backend y del frontend, el modelo lógico de datos, el comportamiento dinámico de los casos de uso más complejos y la trazabilidad entre los requisitos de análisis y los elementos de diseño que los resuelven.

## 5.1 Arquitectura tecnológica del sistema

La arquitectura de Vodeo se implementa como un conjunto de cinco microservicios independientes, cada uno desarrollado como un proyecto Spring Boot autónomo. Todos se despliegan como contenedores Docker orquestados mediante Docker Compose, comunicándose a través de una red interna que oculta la estructura del sistema al exterior.

La Figura 5.1 muestra la arquitectura tecnológica completa del sistema, detallando las tecnologías empleadas en cada componente.

Cada componente emplea las siguientes tecnologías:

- **Capa de Presentación:** Es una *Single Page Application* desarrollada con **React** [5] y empaquetada con **Vite** [17]. Los estilos se implementan con **Tailwind CSS** [6]. La reproducción HLS se gestiona con **hls.js** y la navegación entre vistas con **React Router**.
- **API Gateway:** Implementado con **Spring Cloud Gateway** en modo reactivo (*WebFlux*). Actúa como punto de entrada único, gestionando el enrutamiento hacia los microservicios y las políticas de CORS de forma centralizada.
- **Microservicio de Usuarios:** Gestiona la identidad, las listas de seguimiento (*Watchlist*) y el historial de reproducción. Implementa autenticación **JWT** mediante la librería

**JJWT**, cifrado de contraseñas con **BCrypt** y persistencia con **Spring Data JPA** sobre **PostgreSQL** [7].

- **Microservicio de Catálogo:** Gestiona el contenido multimedia y la integración con APIs externas (TMDB y OMDb). Utiliza **Redis** como caché distribuida (Spring Cache) y la extensión `pg_trgm` de **PostgreSQL** para búsquedas difusas (*Fuzzy Search*).
- **Microservicio de Streaming:** Gestiona la recepción de archivos multimedia, su almacenamiento físico, la entrega de contenido HLS y la eliminación de vídeos. Publica eventos asíncronos en **RabbitMQ** [3] para delegar el procesamiento de vídeo.
- **Worker de Transcodificación:** Microservicio *headless* que consume eventos de **RabbitMQ** y ejecuta **FFmpeg** [4] para generar las variantes HLS de múltiples calidades mediante *Adaptive Bitrate Streaming* (ABS).

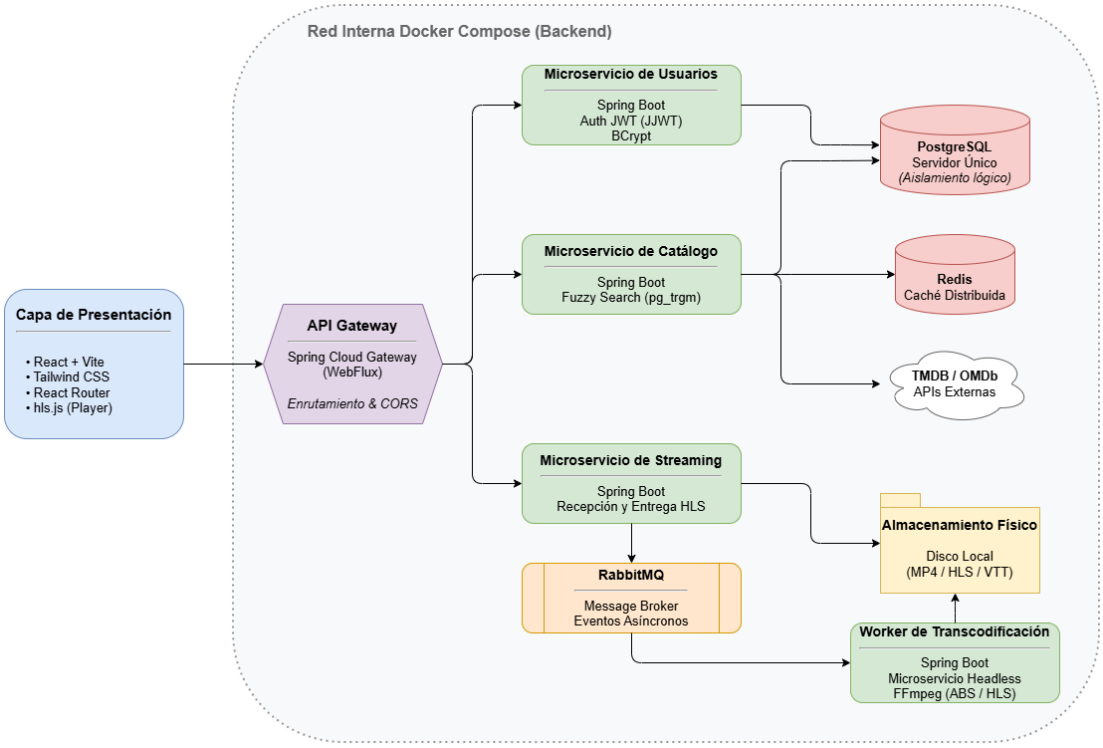



Figura 5.1: Arquitectura tecnológica de la plataforma Vodeo.

## 5.2 Diseño del Backend

Esta sección describe la organización interna del código fuente de los microservicios de *backend* y establece la trazabilidad entre los requisitos definidos en el Capítulo 4 y su implementación concreta.

### 5.2.1 Diagrama de paquetes

Todos los microservicios de *backend* (*users-service*, *catalog-service*, *streaming-service* y *transcoding-service*) siguen una organización por capas homogénea, siguiendo el patrón habitual de las aplicaciones Spring Boot. Cada microservicio se estructura, con matices según sus responsabilidades, en los siguientes paquetes:

- `controller`: expone los endpoints REST y traduce las peticiones HTTP en llamadas a la capa de servicio (p. ej. `UserController`, `CatalogController`, `VideoController`, `StreamingController`).
- `service`: contiene la lógica de negocio (p. ej. `UserService`, `CatalogService`, `VideoService`, `WatchlistService`).
- `repository`: interfaces `JpaRepository` que dan acceso a la persistencia en base de datos(p. ej. `MovieRepository`, `SeriesRepository`, `SeasonRepository`, `EpisodeRepository`).
- `model`: las clases anotadas con `@Entity` que representan el modelo de datos (`User`, `Movie`, `Series`, `Season`, `Episode`, `Review`, `Watchlist`, `PlaybackHistory`, `Subtitle`...).
- `dto`: objetos de transferencia de datos que desacoplan la API REST del modelo de persistencia (p. ej. `RegisterRequest`, `UserResponse`, `MovieRequest`, `MovieResponse`, `AuthResponse`).
- `exception`: jerarquía de excepciones de dominio con tipado fuerte ( `ResourceNotFoundException`, `ConflictException`) y un `GlobalExceptionHandler` anotado con `@RestControllerAdvice` que centraliza su traducción al DTO `ErrorResponse`, evitando el uso de excepciones genéricas de Java y garantizando respuestas HTTP uniformes.
- `config`: configuración de infraestructura de cada microservicio, incluyendo la seguridad (`SecurityConfig`, `JwtAuthFilter`, `JwtService`), la mensajería asíncrona (`RabbitMQConfig`) y la caché distribuida (configuración de **Redis**).

Los clientes de integración con APIs externas (`TmdbClient`, `OmdbClient`) y los *listeners* de eventos asíncronos (`UserCleanupListener`, `MediaCleanupListener`, `FileCleanupListener`) se ubican también dentro del paquete `service`, al tratarse de lógica de negocio o de orquestación externa más que de infraestructura.

Los DTO que representan eventos de dominio publicados en RabbitMQ (`VideoEvent`, `MediaDeletedEvent`, `UserDeletedEvent`) se encuentran en el paquete `dto`, ya que actúan como el contrato de datos serializado que viaja por la cola de mensajería.

La Figura 5.2 muestra esta organización por capas para uno de los microservicios, siendo extrapolable al resto salvo matices puntuales (por ejemplo, *streaming-service* y *transcoding-service* no tienen el paquete `model`, al no necesitar persistencia relacional propia).

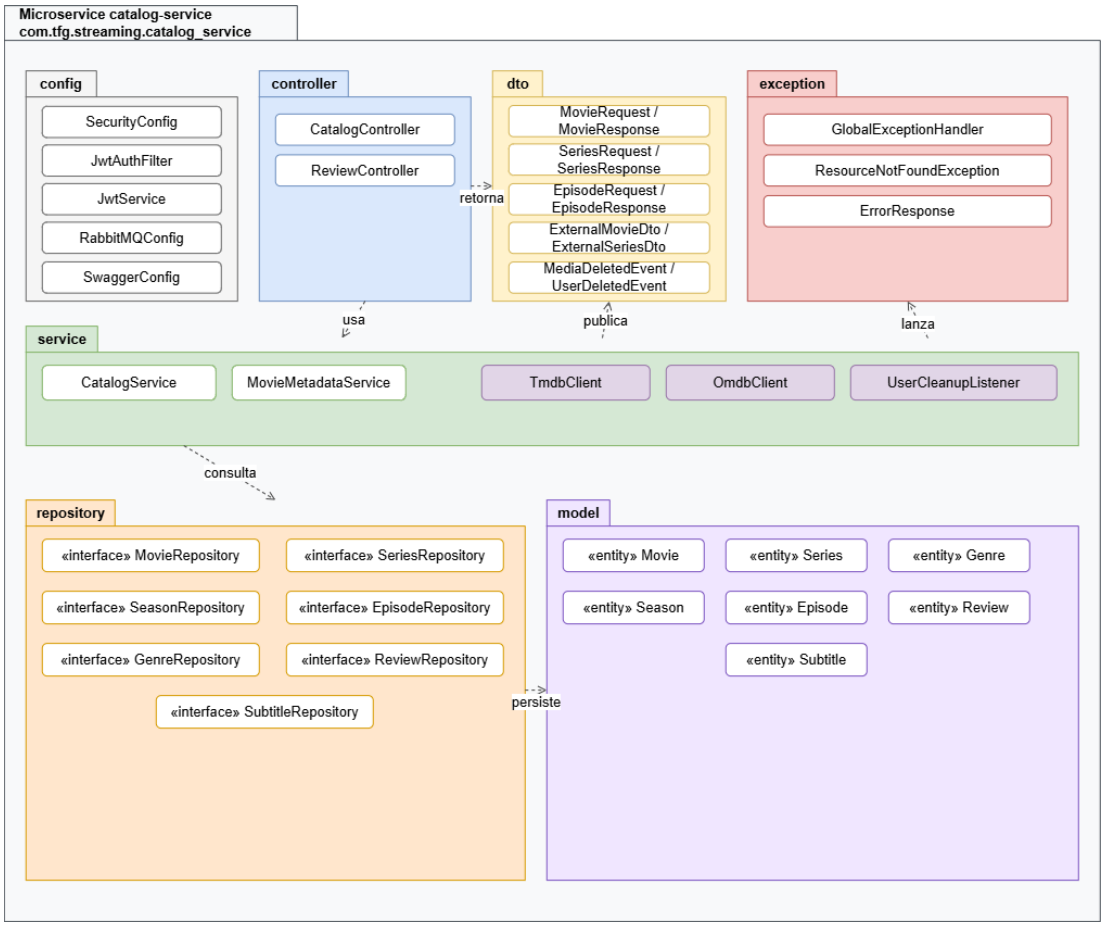



Figura 5.2: Organización en capas de los microservicios de backend.

### 5.2.2 Diseño de componentes: fachadas y flujo de dependencias

Sobre esta base de paquetes, todos los microservicios comparten un mismo patrón de diseño en capas, en el que el *controller* actúa como fachada de entrada, delegando toda la lógica en el *service* correspondiente, que a su vez se apoya en el *repository* para acceder a la persistencia. La Figura 5.3 ilustra este flujo para el *catalog-service*, tomado como ejemplo representativo al ser el que presenta un mayor número de colaboraciones entre componentes. `CatalogController` expone los endpoints de lectura y escritura del catálogo y delega

en `CatalogService`, que orquesta `MovieRepository`, `GenreRepository` y `SeriesRepository` (además de `SeasonRepository`, `EpisodeRepository`, `SubtitleRepository` y `ReviewRepository`, omitidos en la figura por simplicidad).

`CatalogService` colabora, a su vez, con `MovieMetadataService` para la obtención de metadatos externos mediante el patrón *Fallback* entre `TmdbClient` y `OmdbClient`: se solicitan primero los datos a `TmdbClient` y solo se recurre a `OmdbClient` si el primero no devuelve resultados. Además, `CatalogService` inyecta un `RabbitTemplate` para publicar de forma asíncrona los eventos de borrado físico de vídeo (véase la Apartado 5.2.3 sobre la Arquitectura Orientada a Eventos), de modo que operaciones como `deletePhysicalVideo` no acceden al disco directamente, sino que delegan la eliminación real del fichero en el *streaming-service* mediante RabbitMQ.

El *users-service* no concentra toda la gestión del usuario en un único componente. `UserController` se limita a las operaciones propias de identidad: registro, login, consulta y edición del perfil, y conteo de usuarios; delega exclusivamente en `UserService`, que a su vez solo depende de `UserRepository`, del cifrador `BCryptPasswordEncoder` y de `JwtService` para la emisión de tokens.Las operaciones exclusivas de administración de cuentas (listado y borrado de usuarios) se separan en `AdminUserController` y `AdminUserService`, que orquesta el borrado en cascada de los datos privados del usuario y la publicación del evento `UserDeletedEvent` descrito en la Apartado 5.2.3.

Las listas de seguimiento y el historial de reproducción se implementan, en cambio, como componentes independientes dentro del mismo microservicio, cada uno con su propia entidad y tabla:

```
WatchlistController          PlaybackHistoryController
        ↓                            ↓
  WatchlistService           PlaybackHistoryService
        ↓                            ↓
WatchlistRepository        PlaybackHistoryRepository
```

Esta separación evita una clase de servicio monolítica y mantiene acotada la responsabilidad de cada componente.

En el *streaming-service*, `VideoController` delega en `VideoService`, responsable de la escritura en disco del contenido multimedia y de la publicación de eventos en RabbitMQ tras la subida, mientras que `StreamingController` delega en `StreamingService` para la lectura fragmentada del vídeo, el borrado físico y la entrega de subtítulos.

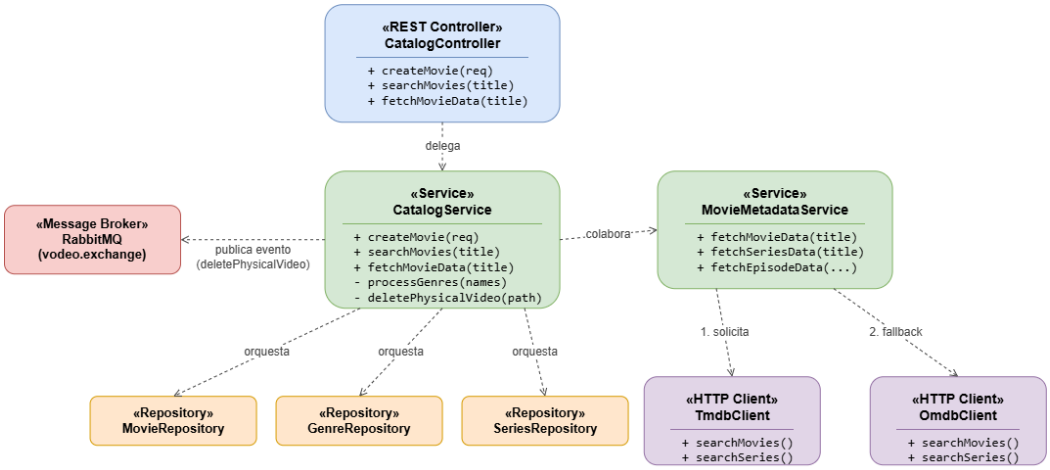



Figura 5.3: Diagrama de clases simplificado del *catalog-service*: fachada, orquestación y patrón fallback.

### 5.2.3 Comunicación asíncrona: arquitectura orientada a eventos

Determinadas operaciones del sistema no encajan bien en el modelo síncrono petición-respuesta que usa REST, bien porque son tareas pesadas que no deben bloquear los hilos del servidor, bien porque requieren mantener la consistencia de datos entre varios microservicios sin acoplarlos mediante llamadas HTTP directas. Para resolver ambos casos se incorporó **RabbitMQ** como *message broker*, configurando en cada microservicio implicado un `TopicExchange` y colas duraderas, con `Jackson2JsonMessageConverter` como convertidor de mensajes para garantizar que el JSON publicado por un microservicio pueda ser interpretado correctamente por el resto. Esta arquitectura orientada a eventos (*Event-Driven Architecture*, EDA) se aplica en Vodeo en dos flujos claramente diferenciados.

**Desacoplo de la subida y transcodificación de vídeo.** La subida de un archivo de vídeo es una operación que puede ser lenta y pesada. Si el *streaming-service* tuviera que transcodificar el vídeo a HLS antes de responder al cliente, el hilo HTTP quedaría bloqueado durante minutos, afectando al rendimiento del servidor ante peticiones concurrentes. Para evitarlo, `VideoService` actúa como *Publisher*: tras escribir el archivo crudo en disco, publica un evento `VideoEvent` con la ruta física del fichero y devuelve inmediatamente una respuesta HTTP 201 Created al cliente, sin esperar a que el vídeo esté procesado. El *transcoding-service*, un microservicio *headless* sin interfaz REST propia, actúa como *Consumer* mediante un método anotado con `@RabbitListener` suscrito a esa cola; al recibir el evento, invoca **FFmpeg** a través de `ProcessBuilder` para generar las variantes HLS en 1080p, 720p y 480p. Este desacoplo permite que la recepción del archivo y su procesamiento que consume mucha CPU escalen de forma independiente.

**Consistencia distribuida ante operaciones de borrado.** El aislamiento de bases de datos propio de la arquitectura de microservicios plantea un problema de consistencia de datos cuando se elimina una entidad: borrar una película en *catalog-service* no elimina automáticamente sus ficheros `.ts`/`.vtt` en *streaming-service*, y borrar un usuario en *users-service* no modifica sus reseñas en *catalog-service*. Resolver esto mediante llamadas HTTP síncronas acoplaría fuertemente los microservicios entre sí. En su lugar, *catalog-service* y *users-service* publican eventos de dominio (`MediaDeletedEvent`, `UserDeletedEvent`, `FileDeletedEvent`) que son interceptados de forma asíncrona por *listeners* transaccionales dedicados (`MediaCleanupListener`, `UserCleanupListener`, `FileCleanupListener`). Estos listeners son responsables de eliminar en cascada los archivos estáticos HLS/VTT del disco del *streaming-service* y de depurar los metadatos obsoletos asociados, además de aplicar una estrategia de anonimización (*soft-delete*) sobre las reseñas del usuario eliminado en lugar de borrarlas, para conservar la integridad del histórico de valoraciones cumpliendo al mismo tiempo con la normativa de protección de datos (RGPD).

Este segundo flujo es el mismo que está detrás de la publicación mencionada en la Figura 5.3: cuando `CatalogService` necesita eliminar el vídeo físico asociado a una película borrada, no accede al disco del *streaming-service* directamente, sino que publica el evento correspondiente y delega su procesamiento en el listener adecuado.

## 5.3 Modelo lógico de datos

El Capítulo 4 presenta el modelo conceptual de datos del sistema mediante un diagrama Entidad-Relación abstracto (Figura 4.5). Esta sección completa esa visión con el **diccionario de datos** de su modelo lógico: las tablas relacionales realmente generadas por Hibernate a partir de las entidades JPA de *users-service* y *catalog-service*, con el tipo, el significado y las restricciones de cada atributo.

Como ya se indicó en el Capítulo 4, ambos microservicios comparten servidor de base de datos pero mantienen un aislamiento lógico estricto: no existen claves foráneas entre las tablas de *users-service* y las de *catalog-service*. Las referencias entre dominios (por ejemplo, qué película guardó un usuario en su *watchlist*) se resuelven mediante un identificador polimórfico (`media_id`, `media_type`) sin una clave foránea real que las conecte: su consistencia se delega en el mecanismo de eventos asíncronos descrito en la Apartado 5.2.3.

La Figura 5.4 muestra el modelo lógico completo derivado de las entidades JPA de ambos microservicios, con sus atributos tipados y las relaciones (claves foráneas, tablas intermedias) que Hibernate genera a partir de las anotaciones `@ManyToOne`, `@OneToMany` y `@ManyToMany`. El diccionario de datos que se detalla a continuación desarrolla, tabla por tabla, la información que resume este diagrama.

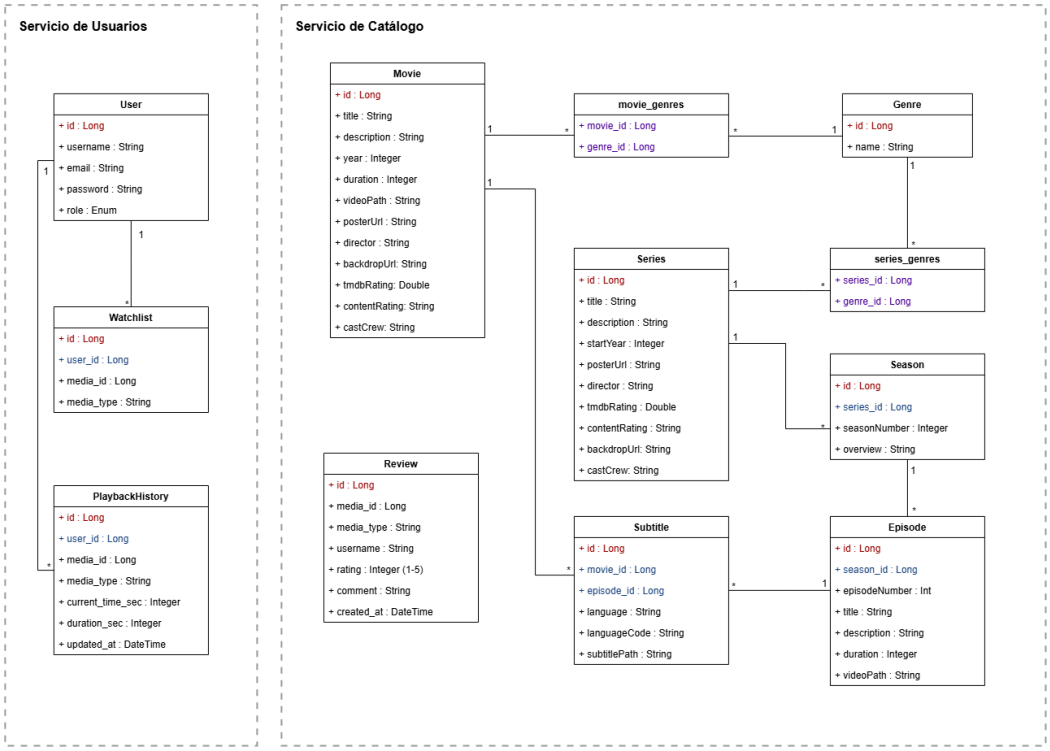



Figura 5.4: Modelo lógico de datos de Vodeo, distribuido entre *users-service* y *catalog-service*.

### 5.3.1 Tablas del *users-service*

**Tabla `users`.** Almacena las cuentas de la plataforma.

| Atributo | Tipo | Descripción | Restricciones |
|---|---|---|---|
| `id` | BIGINT | Identificador único del usuario. | PK, autoincremental |
| `username` | VARCHAR | Nombre visible del usuario en la plataforma. | NOT NULL, UNIQUE |
| `email` | VARCHAR | Correo electrónico, usado como identificador de acceso. | NOT NULL, UNIQUE |
| `password` | VARCHAR | *Hash* BCrypt de la contraseña. | NOT NULL |
| `role` | VARCHAR | Rol del usuario (`USER` / `ADMIN`), almacenado como texto. | NOT NULL, por defecto `USER` |

Tabla 5.1: Diccionario de datos de la tabla `users`.

**Tabla `watchlist`.** Guarda las referencias al contenido marcado como favorito por cada usuario.

| Atributo | Tipo | Descripción | Restricciones |
|---|---|---|---|
| `id` | BIGINT | Identificador único del registro. | PK, autoincremental |
| `user_id` | BIGINT | Usuario propietario de la entrada. | NOT NULL, FK → `users(id)` |
| `media_id` | BIGINT | Identificador del contenido en *catalog-service*. | NOT NULL |
| `media_type` | VARCHAR | Tipo de contenido (`"movie"` / `"series"`). | NOT NULL |

Tabla 5.2: Diccionario de datos de la tabla `watchlist`. Restricción adicional a nivel de tabla: `UNIQUE(user_id, media_id, media_type)`, para impedir duplicados del mismo contenido en la lista de un mismo usuario.

**Tabla `playback_history`.** Registra el progreso de reproducción de cada usuario sobre cada contenido.

| Atributo | Tipo | Descripción | Restricciones |
|---|---|---|---|
| `id` | BIGINT | Identificador único del registro. | PK, autoincremental |
| `user_id` | BIGINT | Usuario al que pertenece el historial. | NOT NULL, FK → `users(id)` |
| `media_id` | BIGINT | Identificador del contenido en *catalog-service*. | NOT NULL |
| `media_type` | VARCHAR | Tipo de contenido (`"movie"` / `"episode"`). | NOT NULL |
| `current_time_sec` | INTEGER | Segundo exacto de reproducción donde se dejó el contenido. | NOT NULL |
| `duration_sec` | INTEGER | Duración total del contenido, usada para calcular el % de progreso. | NULL permitido |
| `updated_at` | TIMESTAMP | Instante de la última actualización del progreso. | NOT NULL |

Tabla 5.3: Diccionario de datos de la tabla `playback_history`. Restricción adicional: UNIQUE`(user_id, media_id, media_type)`, un único registro de progreso por usuario y contenido.

### 5.3.2 Tablas del *catalog-service*

**Tabla `movies`.** Almacena las películas del catálogo.

| Atributo | Tipo | Descripción | Restricciones |
|---|---|---|---|
| `id` | BIGINT | Identificador único de la película. | PK, autoincremental |
| `title` | VARCHAR | Título de la película. | NOT NULL |
| `description` | VARCHAR(2000) | Sinopsis del contenido. | NULL permitido |
| `year` | INTEGER | Año de estreno. | NULL permitido |
| `duration` | INTEGER | Duración en minutos. | NULL permitido |
| `video_path` | VARCHAR | Ruta física (UUID) del archivo de vídeo asociado. | NULL permitido hasta la subida |
| `poster_url` | VARCHAR(500) | URL del póster. | NULL permitido |
| `backdrop_url` | VARCHAR(500) | URL de la imagen de fondo de la ficha. | NULL permitido |
| `tmdb_rating` | DOUBLE | Valoración media importada de TMDB. | NULL permitido |
| `content_rating` | VARCHAR(50) | Clasificación por edades. | NULL permitido |
| `director` | VARCHAR | Director de la película. | NULL permitido |
| `cast_crew` | VARCHAR(1000) | Reparto principal. | NULL permitido |

Tabla 5.4: Diccionario de datos de la tabla `movies`.

**Tabla `series`.** Almacena las series del catálogo. Comparte prácticamente los mismos atributos descriptivos que `movies`, sustituyendo la duración y el año de estreno por el año de inicio de emisión.

| Atributo | Tipo | Descripción | Restricciones |
|---|---|---|---|
| `id` | BIGINT | Identificador único de la serie. | PK, autoincremental |
| `title` | VARCHAR | Título de la serie. | NOT NULL |
| `description` | VARCHAR(2000) | Sinopsis del contenido. | NULL permitido |
| `start_year` | INTEGER | Año de inicio de emisión. | NULL permitido |
| `poster_url` | VARCHAR(500) | URL del póster. | NULL permitido |
| `backdrop_url` | VARCHAR(500) | URL de la imagen de fondo de la ficha. | NULL permitido |
| `tmdb_rating` | DOUBLE | Valoración media importada de TMDB. | NULL permitido |
| `content_rating` | VARCHAR(50) | Clasificación por edades. | NULL permitido |
| `director` | VARCHAR | Creador(es) de la serie. | NULL permitido |
| `cast_crew` | VARCHAR(1000) | Reparto principal. | NULL permitido |

Tabla 5.5: Diccionario de datos de la tabla `series`.

**Tabla `genres`.** Catálogo de géneros cinematográficos, compartido entre películas y series.

| Atributo | Tipo | Descripción | Restricciones |
|---|---|---|---|
| `id` | BIGINT | Identificador único del género. | PK, autoincremental |
| `name` | VARCHAR | Nombre del género (p. ej. *Drama*, *Comedia*). | NOT NULL, UNIQUE |

Tabla 5.6: Diccionario de datos de la tabla `genres`.

**Tablas `movie_genres` y `series_genres`.** Tablas intermedias generadas automáticamente por Hibernate a partir de las relaciones `@ManyToMany` de `Movie` y `Series` con `Genre`. Ninguna de las dos tiene entidad Java propia, ambas siguen la misma estructura.

| Atributo | Tipo | Descripción | Restricciones |
|---|---|---|---|
| `movie_id` / `series_id` | BIGINT | Referencia a la película o serie clasificada. | PK compuesta, FK → `movies(id)` / `series(id)` |
| `genre_id` | BIGINT | Referencia al género asignado. | PK compuesta, FK → `genres(id)` |

Tabla 5.7: Diccionario de datos de las tablas intermedias `movie_genres` y `series_genres`.

**Tabla `seasons`.** Temporadas que componen una serie.

| Atributo | Tipo | Descripción | Restricciones |
|---|---|---|---|
| `id` | BIGINT | Identificador único de la temporada. | PK, autoincremental |
| `season_number` | INTEGER | Número de orden de la temporada dentro de la serie. | NOT NULL |
| `overview` | VARCHAR(1000) | Sinopsis de la temporada. | NULL permitido |
| `series_id` | BIGINT | Serie a la que pertenece. | NOT NULL, FK → `series(id)` |

Tabla 5.8: Diccionario de datos de la tabla `seasons`.

**Tabla `episodes`.** Capítulos individuales de cada temporada.

| Atributo | Tipo | Descripción | Restricciones |
|---|---|---|---|
| `id` | BIGINT | Identificador único del episodio. | PK, autoincremental |
| `episode_number` | INTEGER | Número de orden del episodio dentro de la temporada. | NOT NULL |
| `title` | VARCHAR | Título del episodio. | NOT NULL |
| `description` | VARCHAR(2000) | Sinopsis del episodio. | NULL permitido |
| `duration` | INTEGER | Duración en minutos. | NULL permitido |
| `video_path` | VARCHAR | Ruta física (UUID) del archivo de vídeo asociado. | NULL permitido hasta la subida |
| `season_id` | BIGINT | Temporada a la que pertenece. | NOT NULL, FK → `seasons(id)` |

Tabla 5.9: Diccionario de datos de la tabla `episodes`.

**Tabla `subtitles`.** Pistas de subtítulos en distintos idiomas, asociadas de forma exclusiva a una película o a un episodio.

| Atributo | Tipo | Descripción | Restricciones |
|---|---|---|---|
| `id` | BIGINT | Identificador único del subtítulo. | PK, autoincremental |
| `language` | VARCHAR | Nombre del idioma (p. ej. *Español*). | NOT NULL |
| `language_code` | VARCHAR | Código del idioma (p. ej. `es`, `en`). | NOT NULL |
| `subtitle_path` | VARCHAR | Ruta física del fichero `.vtt`. | NOT NULL |
| `movie_id` | BIGINT | Película asociada, si el subtítulo pertenece a una. | NULL permitido, FK → `movies(id)` |
| `episode_id` | BIGINT | Episodio asociado, si el subtítulo pertenece a uno. | NULL permitido, FK → `episodes(id)` |

Tabla 5.10: Diccionario de datos de la tabla `subtitles`. `movie_id` y `episode_id` son mutuamente excluyentes en la práctica (uno de los dos es siempre NULL), aunque esta exclusión se controla a nivel de aplicación y no mediante una restricción CHECK física.

**Tabla `reviews`.** Valoraciones y comentarios publicados por los usuarios sobre una película o serie.

| Atributo | Tipo | Descripción | Restricciones |
|---|---|---|---|
| `id` | BIGINT | Identificador único de la reseña. | PK, autoincremental |
| `media_id` | BIGINT | Identificador de la película o serie valorada. | NOT NULL |
| `media_type` | VARCHAR | Tipo de contenido (`"movie"` / `"series"`). | NOT NULL |
| `username` | VARCHAR | Nombre del autor, desnormalizado desde el JWT (*Token Enrichment*). | NOT NULL |
| `rating` | INTEGER | Puntuación otorgada, de 1 a 5. | NOT NULL, rango validado en la aplicación |
| `comment` | VARCHAR(1000) | Comentario textual de la reseña. | NULL permitido |
| `created_at` | TIMESTAMP | Instante de publicación de la reseña. | NOT NULL, por defecto el instante de creación |

Tabla 5.11: Diccionario de datos de la tabla `reviews`. Al igual que en `watchlist`, `media_id`/`media_type` es una referencia polimórfica: no hay clave foránea real porque la reseña y el contenido valorado viven en microservicios distintos.

Como puede observarse, las relaciones dentro de un mismo microservicio (`seasons` → `series`, `episodes` → `seasons`, `subtitles` → `movies`/`episodes`, `watchlist`/`playback_history` → `users`) sí tienen clave foránea real. Las que cruzan el límite entre *users-service* y *catalog-service* (`watchlist.media_id`, `playback_history.media_id`, `reviews.media_id`) no: ahí solo hay un identificador polimórfico, sin ninguna clave foránea real que las conecte, siguiendo el mismo aislamiento entre microservicios ya explicado en el Capítulo 4.

## 5.4 Diseño del Frontend

Esta sección describe la organización interna de la *Single Page Application* (SPA) desarrollada en React que constituye la capa de presentación de Vodeo, así como los patrones de diseño

empleados para resolver los problemas propios de una interfaz cliente que consume una API compuesta por varios microservicios independientes.

### 5.4.1 Estructura de componentes y enrutamiento

Para mantener un código estructurado y escalable, el proyecto se organiza en cuatro directorios principales dentro de `src/`:

- `pages/`: vistas completas asociadas a una ruta de la aplicación (`Login`, `Register`, `Catalog`, `MovieDetails`, `SeriesDetails`, `Player`, `Watchlist`, `Profile`, `Admin`, `AdminDashboard`, `AdminUsers`).
- `components/`: piezas de interfaz reutilizables que se emplean en distintas vistas (`PrivateRoute`, `AdminRoute`, `ContinueWatching`, `Recommendations`, `ReviewSection`).
- `context/`: estado global de la aplicación mediante la *Context API* de React (`AuthContext`).
- `api/`: configuración centralizada del cliente HTTP (`axiosConfig.js`).

El enrutamiento se gestiona con **React Router** desde el componente raíz `App.jsx`, que declara todas las rutas de la SPA. Las rutas que requieren sesión iniciada se envuelven en el componente de orden superior `PrivateRoute`, que consulta el `token` expuesto por `AuthContext` y redirige a `/login` mediante `<Navigate replace>` si no existe. Las rutas exclusivas de administración (`/admin`, `/admin/dashboard`, `/admin/users`) se envuelven además en `AdminRoute`, que exige adicionalmente que el `role` decodificado del JWT sea `ADMIN`, redirigiendo en caso contrario a `/catalog` en lugar de a `/login`, ya que el usuario sí está autenticado, simplemente no autorizado para esa vista. Esta separación en dos guardias de ruta evita duplicar la lógica de comprobación de sesión en cada página protegida.

La Figura 5.5 muestra la organización de componentes de la SPA y el árbol de rutas resultante de esta protección en dos niveles.

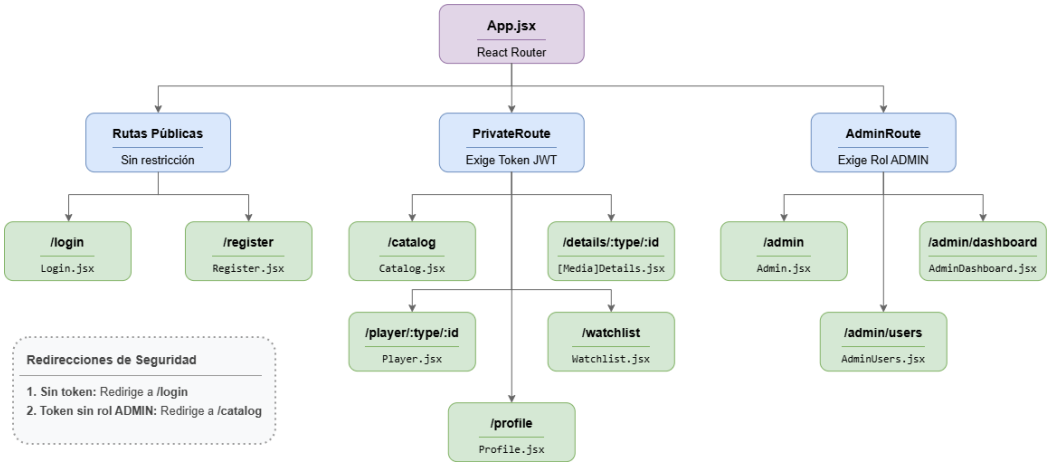



Figura 5.5: Diagrama de componentes y enrutamiento de la SPA de Vodeo.

### 5.4.2 Gestión de sesión y comunicación con el backend

La sesión de usuario se gestiona mediante el patrón *Provider*, evitando el problema de *prop drilling* que supondría propagar el JWT manualmente a través del árbol de componentes. El componente `AuthContext` almacena el token en `localStorage` para persistir la sesión entre recargas de página y, mediante un `useEffect`, lo decodifica en cada cambio con la librería `jwt-decode` para derivar el objeto `user` (correo y rol) sin necesidad de una petición adicional al servidor. En esa misma comprobación se comprueba la fecha de expiración (`exp`) del token: si ha caducado o el token es inválido, se ejecuta un *logout* que limpia tanto el estado de React como el `localStorage`, evitando que la aplicación quede en un estado de sesión inconsistente.

La comunicación con el backend se centraliza en `axiosConfig.js`, que define tres instancias independientes de `axios` (`authApi`, `catalogApi` y `streamingApi`), una por cada microservicio con el que interactúa el cliente, con su `baseURL` correspondiente inyectada desde variables de entorno. Las tres instancias comparten un mismo *interceptor* de petición que añade automáticamente la cabecera `Authorization: Bearer` con el token leído de `localStorage`, de modo que ningún componente necesita gestionar el token de forma manual en sus llamadas a la API.

### 5.4.3 Patrones de diseño en la capa de presentación

Además de la estructura general, varias vistas de Vodeo resuelven problemas concretos mediante patrones de diseño específicos, derivados de la división del backend en microservicios y de la gestión multimedia.

**Composición de API (*API Composition* / *Double-Fetch*).** Al estar los datos de usuario y los metadatos de contenido en microservicios distintos, ninguno de los dos servicios puede resolver por sí mismo una vista que combine ambos. `Watchlist.jsx` resuelve este problema en dos fases: primero solicita a `users-service` una página de identificadores (`mediaId`, `mediaType`) ordenada por fecha de guardado, y a continuación separa esos identificadores por tipo y lanza en paralelo, mediante `Promise.all`, dos peticiones en lote (`/movies/batch` y `/series/batch`) hacia el `catalog-service` para recuperar la información completa. El resultado combinado se reordena localmente para respetar el orden cronológico original, ya que la unión de ambas respuestas no lo conserva. El mismo patrón se reutiliza en `ContinueWatching.jsx` y `Recommendations.jsx` para cruzar el historial de reproducción del usuario con los metadatos del catálogo.

**Interfaz optimista (*Optimistic UI*).** Para que la acción de marcar o desmarcar una película como favorita se perciba como inmediata, la interfaz actualiza el estado local en el momento del clic, mientras la petición HTTP real se envía en segundo plano hacia el `users-service`. Si la petición falla, se deshace el cambio visual y avisa del error mediante `react-hot-toast`. Permite no tener que esperar a que el servidor responda para ver el resultado.

***Throttling* del seguimiento de reproducción.** Guardar la posición de reproducción en cada evento `timeupdate` del elemento `<video>` (que se dispara varias veces por segundo) saturaría innecesariamente a `users-service`. `Player.jsx` evita esto mediante una referencia mutable (`useRef`) que almacena el último instante guardado, y solo emite una petición de actualización cuando la diferencia respecto al tiempo actual supera los 10 segundos, además de hacerlo de forma explícita al pausar el vídeo o al cerrar el reproductor. Además, descarta cualquier progreso inferior a 30 segundos, para no llenar el historial con reproducciones accidentales.

***Retry* de inicialización para evitar condiciones de carrera.** Para que la librería `hls.js` funcione correctamente, el nodo `<video>` debe estar ya montado en el DOM, algo que no está garantizado si la inicialización se dispara desde un `useEffect` estándar nada más resolverse los metadatos del vídeo. `Player.jsx` resuelve esto dentro de la propia función `initHls`: si `videoRef.current` todavía no apunta a ningún nodo, la función se reprograma a sí misma mediante `setTimeout(initHls, 100)` y vuelve a comprobarlo, repitiendo este ciclo hasta que el nodo exista realmente en el DOM. Este mismo mecanismo de reintento se reutiliza más adelante para recuperarse de errores de red al cargar el manifiesto HLS.

## 5.5 Comportamiento dinámico (Diagramas de secuencia)

Para comprender cómo colaboran los distintos microservicios, a continuación se detalla el comportamiento dinámico de los dos casos de uso más complejos del sistema: la subida y procesamiento asíncrono de un vídeo (CU-01) y su reproducción con calidad adaptativa (CU-02).

### CU-01. Subir y procesar vídeo

El flujo comienza cuando el administrador selecciona un archivo de vídeo desde el panel de administración (`Admin.jsx`), que lo envía al *streaming-service* mediante una petición `multipart/form-data`. `VideoController` delega en `VideoService`, que persiste el archivo en disco de forma secuencial mediante `Files.copy()` para evitar cargarlo íntegramente en memoria RAM, y le asigna un identificador único (UUID) que preserva la extensión original. Inmediatamente después, `VideoService` publica un evento `VideoEvent` en RabbitMQ con la ruta física del fichero y devuelve la respuesta HTTP al cliente sin esperar a que el vídeo esté procesado.

De forma asíncrona y desacoplada del ciclo petición-respuesta, el *transcoding-service* consume el evento mediante un método anotado con `@RabbitListener`, invoca **FFmpeg** a través de `ProcessBuilder` y genera las tres variantes de calidad (1080p, 720p y 480p) empaquetadas bajo un índice de reproducción adaptativo (`master.m3u8`). Una vez completado el procesamiento, el archivo original se elimina del disco y las variantes generadas quedan disponibles para su reproducción. La Figura 5.6 recoge este flujo, delimitando mediante un marco la parte que se ejecuta de forma asíncrona.

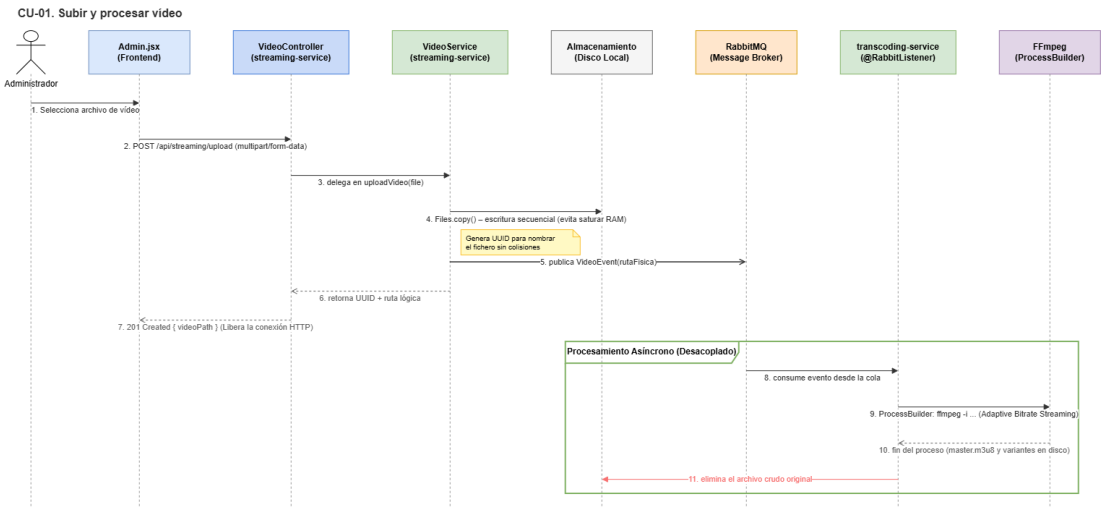



Figura 5.6: Diagrama de secuencia de CU-01: subida y procesamiento asíncrono de vídeo.

### CU-02. Reproducir contenido con calidad adaptativa

Al pulsar el botón de reproducción, `Player.jsx` solicita primero al *catalog-service* los metadatos y la ruta lógica del contenido. A continuación, instancia la librería **hls.js** mediante el mecanismo de *Retry* descrito en la sección de patrones de diseño del *frontend* (evitando así la condición de carrera con el montaje del nodo `<video>`), indicándole la URL del índice HLS en el *streaming-service*. Dado que las etiquetas `<video>` de HTML5 no permiten inyectar cabeceras `Authorization` personalizadas, el JWT se transmite como parámetro de consulta (`?token=`) en dicha URL, siendo validado por `JwtAuthFilter` de forma equivalente a como se hace con la cabecera estándar.

Una vez descargado el índice adaptativo, **hls.js** estima el ancho de banda disponible y selecciona automáticamente la variante de calidad más adecuada, solicitando los fragmentos de vídeo correspondientes al *streaming-service*, que responde con código HTTP 206 (*Partial Content*). Si las condiciones de red cambian durante la reproducción, **hls.js** cambia de variante de forma transparente para el usuario, sin interrumpir la reproducción. En paralelo, y de forma no bloqueante, `Player.jsx` guarda periódicamente el progreso de visionado en el *users-service* mediante un patrón de *throttling* (una actualización cada 10 segundos, o de forma explícita al pausar o cerrar el reproductor), detallado en la sección de patrones de diseño del *frontend*. La Figura 5.7 ilustra este flujo completo.

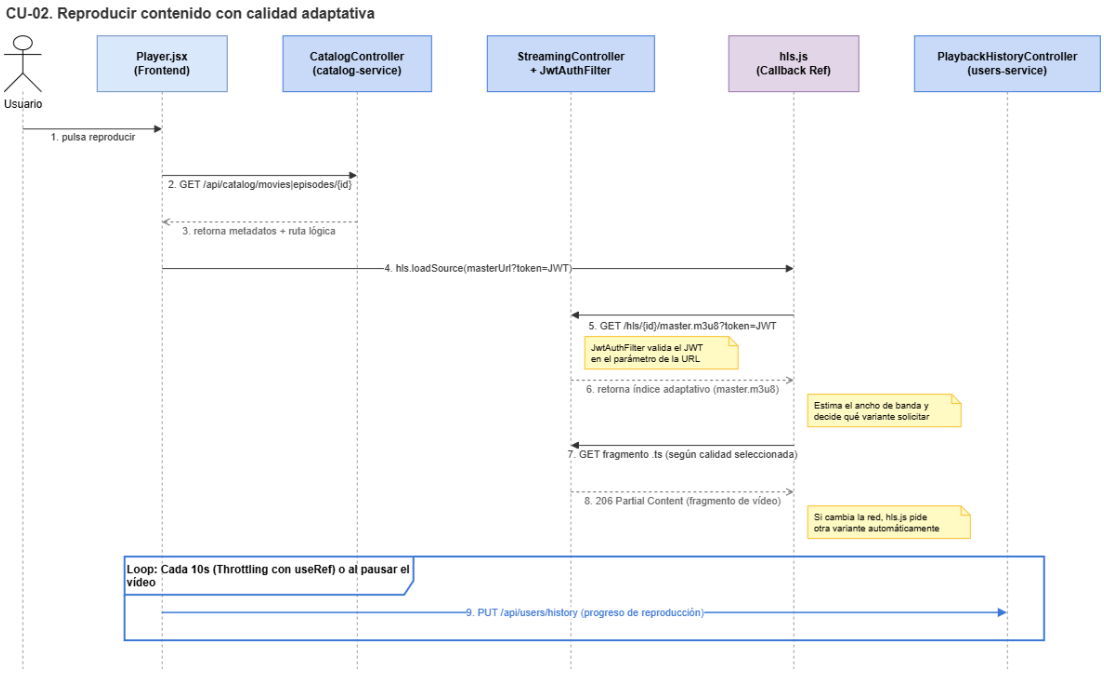



Figura 5.7: Diagrama de secuencia de CU-02: reproducción adaptativa mediante HLS.

## 5.6 Trazabilidad

Para garantizar que el diseño técnico responde a las necesidades identificadas en la fase de análisis (Capítulo 4), en esta sección se establece la trazabilidad entre los requisitos del sistema y su implementación concreta en el código fuente.

En primer lugar, la Tabla 5.12 relaciona los Requisitos Funcionales (RF) con las vistas (*pages*) y componentes de la capa de presentación desarrollada en React, permitiendo identificar qué archivos físicos resuelven la interfaz de cada funcionalidad.

| **Requisito Funcional (RF)** | **Componente Frontend (`src/pages` y `components`)** |
|---|---|
| RF-01, RF-02. Registro, autenticación y control de roles | `Login.jsx`, `Register.jsx`, `PrivateRoute.jsx`, `AdminRoute.jsx` |
| RF-03. Gestión de perfil | `Profile.jsx` |
| RF-08. Exploración y búsqueda | `Catalog.jsx` |
| RF-07, RF-09. Detalle de series/películas y sistema de reseñas | `MovieDetails.jsx`, `SeriesDetails.jsx`, `ReviewSection.jsx` |
| RF-12, RF-13, RF-14. Reproducción adaptativa, subtítulos y reproducción continua | `Player.jsx` |
| RF-15, RF-16, RF-17. Favoritos, historial y recomendaciones | `Watchlist.jsx`, `ContinueWatching.jsx`, `Recommendations.jsx` |
| RF-04, RF-05, RF-06, RF-10. Administración, subida y métricas | `Admin.jsx`, `AdminUsers.jsx`, `AdminDashboard.jsx` |

Tabla 5.12: Trazabilidad entre requisitos funcionales y componentes de la interfaz.

Por otro lado, la Tabla 5.13 mapea los casos de uso principales y conjuntos funcionales con los controladores REST y servicios del *backend*, mostrando en qué microservicio se ejecuta la lógica de negocio asociada.

| Flujo / Caso de Uso | Microservicio | Clases Java (`Controller` y `Service`) |
|---|---|---|
| **CU-01.** Subir y procesar vídeo | *streaming-service* *catalog-service* | `VideoController`, `VideoService` `CatalogController`, `CatalogService` |
| **CU-02.** Reproducir contenido con calidad adaptativa | *catalog-service* *streaming-service* *users-service* | `CatalogController` `StreamingController`, `StreamingService` `PlaybackHistoryController`, `PlaybackHistoryService` |
| Gestión de identidad y cuentas | *users-service* | `UserController`, `UserService` `AdminUserController`, `AdminUserService` |
| Gestión de favoritos (*Watchlist*) | *users-service* | `WatchlistController`, `WatchlistService` |
| Reseñas y valoraciones | *catalog-service* | `ReviewController`, `ReviewService` |
| Extracción de metadatos (TMDB/OMDb) | *catalog-service* | `CatalogController`, `MovieMetadataService` |
| Búsqueda y filtrado del catálogo | *catalog-service* | `CatalogController`, `CatalogService` |

Tabla 5.13: Trazabilidad entre casos de uso y microservicios del backend.

Capítulo 6

# Implementación y pruebas

El Capítulo 5 ya dice qué componentes existen y cómo se relacionan. Aquí se entra en cómo funciona por dentro los puntos más complicados, y cómo se comprueba que ese comportamiento es correcto.

## 6.1 Implementación

### 6.1.1 Entrega fragmentada de vídeo (*Chunked Streaming*)

Servir un archivo de vídeo completo en una única respuesta HTTP obligaría al cliente a descargarlo entero antes de empezar a verlo, y a tener todo el archivo cargado en la memoria del servidor mientras dura la respuesta. `StreamingService.getVideoRegion` evita ambos problemas apoyándose en la clase `ResourceRegion` de Spring, que permite devolver únicamente el fragmento de bytes solicitado.

El método distingue dos casos: si el cliente no envía cabecera `Range` (la primera petición del reproductor), se sirve el primer megabyte del archivo. Si la envía, se respeta el punto de inicio solicitado, pero se limita igualmente la longitud devuelta a un megabyte mediante `Math.min`. Así el navegador va pidiendo el vídeo poco a poco. `StreamingController` devuelve siempre el código 206 (*Partial Content*), tanto si hay `Range` como si no.

```java
private static final long CHUNK_SIZE = 1000000L;

public ResourceRegion getVideoRegion(String filename, HttpHeaders
    headers) throws IOException {
    Path filePath = Paths.get(storagePath, filename);
    if (!Files.exists(filePath)) {
        throw new IOException("Archivo de vídeo no encontrado en el
    servidor");
    }

    Resource video = new FileSystemResource(filePath);
    long contentLength = video.contentLength();

    List<HttpRange> ranges = headers.getRange();
    if (ranges.isEmpty()) {
        long rangeLength = Math.min(CHUNK_SIZE, contentLength);
        return new ResourceRegion(video, 0, rangeLength);
    }

    HttpRange range = ranges.get(0);
    long start = range.getRangeStart(contentLength);
    long end = range.getRangeEnd(contentLength);
    long rangeLength = Math.min(CHUNK_SIZE, end - start + 1);

    return new ResourceRegion(video, start, rangeLength);
}
```

Listado 6.1: Cálculo del fragmento de vídeo a partir de la cabecera `Range` (`StreamingService.java`).

### 6.1.2 Autenticación JWT en las peticiones de `hls.js`

Como se explicó en el Capítulo 5, el elemento nativo `<video>` de HTML5 no permite configurar cabeceras HTTP personalizadas, por lo que la cabecera `Authorization` nunca llegaría al *streaming-service* si el reproductor dependiera únicamente de esa etiqueta. La solución final distingue dos casos según el navegador:

```
// Si el navegador es Safari tiene soporte nativo
  if (!Hls.isSupported()) {
    if (video.canPlayType('application/vnd.apple.mpegurl')) {
      video.src = `${m3u8Url}?token=${token}`;
      video.addEventListener('loadedmetadata', () =>
    setHlsStatus("Listo"), { once: true });
    }
    return;
  }

  const hls = new Hls({
    debug: false,
    xhrSetup: (xhr) => {
      xhr.setRequestHeader('Authorization', `Bearer ${token}`);
    }
  });

  hlsRef.current = hls;

  hls.attachMedia(video);

  hls.on(Hls.Events.MEDIA_ATTACHED, () => {
    hls.loadSource(m3u8Url);
  });
```

Listado 6.2: Inicialización de `hls.js` con inyección de cabecera JWT vía `xhrSetup` (`Player.jsx`).

En casi todos los navegadores `hls.js` está activo, y su opción `xhrSetup` permite inyectar una cabecera `Authorization: Bearer` real en cada petición XHR que la librería hace por dentro, tanto para el manifiesto (`master.m3u8`) como para cada segmento `.ts`. El JWT nunca llega a aparecer en la URL. Solo en el caso de Safari, que reproduce HLS de forma nativa sin pasar por `hls.js`, no existe ningún mecanismo para adjuntar cabeceras a la petición que genera la propia etiqueta `<video>`, por lo que ahí sí se recurre al parámetro de consulta `?token=` descrito en el diseño. El `JwtAuthFilter` de *streaming-service* acepta las dos vías (cabecera y parámetro) para cubrir este segundo caso, aunque en la práctica solo se usa cuando el reproductor detecta que `hls.js` no está soportado.

### 6.1.3 Corrección de consultas N+1 y `MultipleBagFetchException`

Por defecto, Hibernate resuelve las colecciones `@ManyToMany` (los géneros de una película o serie) de forma perezosa, lo que provoca el problema de rendimiento conocido como *N+1 queries*: al listar el catálogo, se ejecuta una consulta para obtener

las películas y, después, una consulta adicional por cada película para traer sus géneros. `MovieRepository` y `SeriesRepository` evitan esto anotando sus métodos de consulta con `@EntityGraph(attributePaths = {"genres"})`. Así Hibernate resuelve esa colección mediante `JOIN FETCH` en una única sentencia SQL, sin importar cuántos elementos se devuelvan.

Al introducir la jerarquía `Series` → `Season` → `Episode` apareció un problema nuevo: pedir con `@EntityGraph` el fetch simultáneo de `genres` y `seasons` sobre `Series` lanza la excepción `MultipleBagFetchException`, porque Hibernate no puede combinar en una sola consulta SQL el producto cartesiano de dos colecciones de tipo `List` sin generar filas duplicadas. La solución fue hacer que `SeriesRepository` solo declare el grafo de entidades sobre `genres`:

```
@EntityGraph(attributePaths = { "genres" })
Optional<Series> findById(Long id);

@Query(value = "SELECT * FROM series WHERE similarity(title,
    :title) > 0.15 OR title ILIKE '%' || :title || '%' ORDER BY
    similarity(title, :title) DESC", countQuery = "SELECT count(*)
    FROM series WHERE similarity(title, :title) > 0.15 OR title
    ILIKE '%' || :title || '%'", nativeQuery = true)
Page<Series> findByTitleContainingIgnoreCase(@Param("title") String
    title, Pageable pageable);
```

Listado 6.3: `@EntityGraph` aplicado solo sobre `genres` (`SeriesRepository.java`).

Las temporadas y episodios se cargan aparte, bajo demanda, con `SeasonRepository` (`findBySeriesIdOrderBySeasonNumberAsc`) y `EpisodeRepository`, que solo se ejecutan cuando el cliente accede al detalle de una serie concreta. Esto evita la excepción y la sobrecarga de cargar el árbol completo de temporadas y episodios al listar simplemente el catálogo.

### 6.1.4 Búsqueda difusa con `pg_trgm`

Para que la búsqueda por título admita erratas y coincidencias parciales, se combina la extensión `pg_trgm` de PostgreSQL (compara cadenas por trozos de tres caracteres y da un grado de similitud) con el operador `ILIKE` como respaldo (Listado 6.3, método `findByTitleContainingIgnoreCase`, presente también en `MovieRepository`). El umbral de similitud (`0.15`) se fijó para admitir pequeñas erratas sin devolver resultados irrelevantes, y los resultados se ordenan por `similarity(title, :title) DESC` para que las coincidencias más parecidas aparezcan primero.

La combinación de la consulta SQL con la paginación de Spring Data JPA da un problema: cuando una consulta se deriva automáticamente del nombre del método, Spring puede generar por sí solo la consulta de conteo necesaria para calcular el número total de páginas, pero al ser una consulta nativa con una función no estándar en la cláusula ORDER BY, Spring no puede calcular ese conteo automáticamente. Omitir el atributo `countQuery` produce un error en tiempo de ejecución al construir el objeto `Page`, por lo que ambas consultas deben declararse por separado.

### 6.1.5 Transcodificación adaptativa y control del subproceso FFmpeg

El *transcoding-service* genera las tres variantes de calidad HLS (1080p, 720p y 480p) con una única llamada a **FFmpeg**. Así no hace falta decodificar tres veces el mismo vídeo de entrada. El comando se construye en `VideoTranscodingWorker.buildAbsCommand`, utilizando el filtro `filter_complex` para dividir el flujo decodificado en tres ramas de escalado independientes:

```
private List<String> buildAbsCommand(String input, String
    segmentPattern, String playlistPattern) {
    return List.of(
            "ffmpeg", "-i", input,

            // UN decode -> TRES pipelines de escalado
            "-filter_complex",
            "[0:v]split=3[v1][v2][v3];" +
                    "[v1]scale=1920:1080[v1out];" +
                    "[v2]scale=1280:720[v2out];" +
                    "[v3]scale=854:480[v3out]",

            // Stream 0: 1080p
            "-map", "[v1out]", "-map", "0:a",
            "-c:v:0", "libx264", "-preset", "fast",
            "-profile:v:0", "main",
            "-b:v:0", "5000k", "-maxrate:v:0", "5350k",
            "-bufsize:v:0", "7500k",

            // Stream 1: 720p
            "-map", "[v2out]", "-map", "0:a",
            "-c:v:1", "libx264", "-preset", "fast",
            "-profile:v:1", "main",
            "-b:v:1", "2800k", "-maxrate:v:1", "2996k",
            "-bufsize:v:1", "4200k",

            // Stream 2: 480p
            "-map", "[v3out]", "-map", "0:a",
```

```
            "-c:v:2", "libx264", "-preset", "fast",
            "-profile:v:2", "main",
            "-b:v:2", "1400k", "-maxrate:v:2", "1498k",
            "-bufsize:v:2", "2100k",

            // Audio: AAC estereo compartido
            "-c:a", "aac", "-b:a", "128k", "-ac", "2",

            // HLS output
            "-var_stream_map", "v:0,a:0 v:1,a:1 v:2,a:2",
            "-master_pl_name", "master.m3u8",
            "-hls_time", "10",
            "-hls_playlist_type", "vod",
            "-hls_segment_filename", segmentPattern,
            playlistPattern);
}
```

Listado 6.4: Construcción del comando *multi-bitrate* (`VideoTranscodingWorker.java`).

El resultado se empaqueta como HLS con segmentos de 10 segundos (`-hls_time 10`) y un índice maestro que enumera las tres variantes mediante `-var_stream_map`. Así `hls.js` puede cambiar de calidad en tiempo de reproducción sin reiniciar la descarga.

`ProcessBuilder` ejecuta este comando como un subproceso del sistema operativo, y ese subproceso tiene un búfer de salida estándar con capacidad limitada. Si el proceso Java que lo lanza no consume esa salida, el búfer se llena y FFmpeg queda bloqueado, porque no puede seguir escribiendo su propio *log* de progreso. La transcodificación se cuelga sin lanzar ninguna excepción.

```
ProcessBuilder pb = new ProcessBuilder(command);
pb.redirectErrorStream(true);
pb.directory(outputDir);

Process process = pb.start();

// Consumir el output de FFmpeg en el hilo actual para no bloquear
    el buffer del SO
try (BufferedReader reader = new BufferedReader(
        new InputStreamReader(process.getInputStream()))) {
    reader.lines().forEach(line -> log.info("[FFmpeg] {}", line));
    // Info para verlo en consola
}

int exitCode = process.waitFor();

```

```
15 if (exitCode == 0) {
16     log.info("[TranscodingWorker] Transcodificacion ABS completada
       con exito. Salida en: {}",
17             outputDirPath);
18
19     // Borrar el archivo original
20     if (inputFile.delete()) {
21         log.info("[TranscodingWorker] Archivo de origen crudo
       eliminado correctamente: {}", filename);
22     }
23 } else {
24     log.error("[TranscodingWorker] El proceso FFmpeg finalizo con
       codigo de error: {}", exitCode);
25 }
```

Listado 6.5: Consumo activo de la salida del proceso para evitar el bloqueo del búfer (`VideoTranscodingWorker.java`).

Por eso hay que redirigir la salida de error al flujo estándar (`redirectErrorStream(true)`) y leerla línea a línea con un `BufferedReader` *antes* de invocar `process.waitFor()`. Así el búfer del sistema operativo se va vaciando a medida que se genera, y no llega a bloquearse.

### 6.1.6 Caché distribuida con Redis: patrón *Cache-Aside*

Las operaciones de lectura del catálogo (listados paginados, búsquedas y consultas por lote) son las más frecuentes de la aplicación. `CatalogService` implementa el patrón *Cache-Aside* con `@Cacheable` de Spring Cache, usando claves compuestas que incluyen los parámetros de cada consulta (por ejemplo, `'page_' + #page + '_size_' + #size` para el listado paginado, o `'id_' + #id` para la consulta por identificador). Cada combinación distinta de parámetros se guarda como una entrada independiente. El resultado de `getNextEpisode` (Apartado 6.1.7) también se cachea del mismo modo, porque es una consulta que se repite cada vez que un usuario avanza por la misma serie.

Una caché también puede servir datos obsoletos tras un cambio (*stale data*). Para evitarlo, cada método de `CatalogService` que crea, actualiza o elimina contenido se anota con `@CacheEvict(allEntries = true)`, pero solo invalida las cachés que ese cambio puede afectar: las escrituras de películas invalidan `movies` y `genres`; las de series invalidan `series` y `genres`; y las de temporadas y episodios invalidan solo `series`. El borrado de un subtítulo es la excepción que invalida las tres a la vez, porque en ese punto solo se tiene el identificador del subtítulo y no hay forma de saber si era de una película o de un episodio sin una consulta adicional. Se descarta toda la caché del dominio afectado en lugar de solo la entrada concreta, para no tener que mantener sincronizadas manualmente decenas de

claves compuestas. Para que Spring pueda serializar estos resultados hacia Redis, los DTO de respuesta del catálogo (`MovieResponse`, `SeriesResponse`, `EpisodeResponse`) implementan la interfaz `Serializable`.

### 6.1.7 Reproducción continua entre episodios (*Autoplay*)

Calcular el siguiente episodio no es solo sumar uno al número de episodio. Hay que tener en cuenta el caso en que el episodio actual sea el último de su temporada (hay que pasar al primer episodio de la temporada siguiente), y el caso en que no exista ninguna temporada posterior. `CatalogService.getNextEpisode` resuelve esta lógica en tres pasos, usando dos consultas derivadas de Spring Data JPA que hacen la búsqueda directamente en SQL:

```
@Cacheable(value = "series", key = "'next_ep_' + #currentEpisodeId")
public EpisodeResponse getNextEpisode(Long currentEpisodeId) {
  Episode currentEp = episodeRepository.findById(currentEpisodeId)
      .orElseThrow(() -> new ResourceNotFoundException("Episode not found"));

  // Intentar buscar el siguiente episodio en la MISMA temporada
  Optional<Episode> nextInSeason = episodeRepository
      .findFirstBySeasonIdAndEpisodeNumberGreaterThanOrderByEpisodeNumberAsc(
          currentEp.getSeason().getId(), currentEp.getEpisodeNumber());

  if (nextInSeason.isPresent()) {
    return toEpisodeResponse(nextInSeason.get());
  }

  // Si no hay más en esta temporada, buscar la SIGUIENTE temporada de la
    serie
  Optional<Season> nextSeason = seasonRepository
      .findFirstBySeriesIdAndSeasonNumberGreaterThanOrderBySeasonNumberAsc(
          currentEp.getSeason().getSeries().getId(),
    currentEp.getSeason().getSeasonNumber());

  if (nextSeason.isPresent()) {
    // Devolver el primer episodio de esa nueva temporada
    Optional<Episode> firstEpNextSeason = episodeRepository

    .findFirstBySeasonIdOrderByEpisodeNumberAsc(nextSeason.get().getId());
    if (firstEpNextSeason.isPresent()) {
      return toEpisodeResponse(firstEpNextSeason.get());
    }
  }

  // Ha visto el último episodio de la última temporada por lo que fin de la
    serie
  return null;
}
```

Listado 6.6: Algoritmo de cálculo del siguiente episodio (`CatalogService.java`).

El método devuelve `null` cuando no queda contenido disponible. El *frontend* usa ese `null` para no ofrecer la reproducción automática al terminar la serie. En el cliente, `Player.jsx` escucha el evento `onEnded` del elemento `<video>`. Si existe un episodio siguiente, muestra una interfaz con una cuenta atrás de diez segundos y navega mediante React Router con la opción `{ replace: true }`. Así no se acumula una entrada por episodio en el historial del navegador, y el botón de retroceso vuelve directamente al catálogo. Además, el evento `onLoadedMetadata` del episodio siguiente comprueba si el progreso guardado del historial supera el 95 % de la duración total. En ese caso, reinicia el contador a cero antes de continuar. Así un episodio marcado casi como completo no vuelve a disparar el salto automático en bucle.

### 6.1.8 Composición y post-procesado de datos en el cliente

Como ya se explicó en el Capítulo 5, *users-service* y *catalog-service* no comparten datos entre sí. Por eso, algunas vistas del *frontend* tienen que juntar en el cliente información que viene de los dos microservicios. Dos componentes van un paso más allá de solo juntar los datos: además los procesan antes de mostrarlos.

`Recommendations.jsx` calcula recomendaciones de contenido sin tocar el *backend*, todo el cálculo se hace en el propio componente de React. Recupera las últimas 50 entradas del historial de reproducción, cuenta cuántas veces aparece cada género y filtra el catálogo para sugerir contenido de ese género más visto, quitando lo que el usuario ya ha visto.

```
let genreCounts = {};
const watchedParentIds = new Set(); // Para no recomendar cosas que ya hemos visto

// Obtener metadatos de lo que ha visto
if (type === 'movie') {
  const watchedRes = await catalogApi.get(`/movies/batch?ids=${watchedIds.join(',')}`);
  watchedRes.data.forEach(movie => {
    watchedParentIds.add(movie.id);
    if (movie.genres) {
      movie.genres.forEach(genre => {
        genreCounts[genre] = (genreCounts[genre] || 0) + 1;
      });
    }
  });
} else {
  // Si es serie obtenemos los episodios y leemos la informacion
  const episodesData = await Promise.all(
    watchedIds.map(id => catalogApi.get(`/episodes/${id}`).then(res => res.data).catch(() => null))
  );
```

```
20   episodesData.forEach(ep => {
21     if (ep) {
22       watchedParentIds.add(ep.seriesId); // Marcamos la SERIE como
     vista
23       if (ep.seriesGenres) {
24         ep.seriesGenres.forEach(genre => {
25           genreCounts[genre] = (genreCounts[genre] || 0) + 1;
26         });
27       }
28     }
29   });
30 }
31
32 // Calcular el genero favorito por frecuencia
33 const sortedGenres = Object.keys(genreCounts).sort((a, b) =>
     genreCounts[b] - genreCounts[a]);
34 const favoriteGenre = sortedGenres[0];
35
36 if (!favoriteGenre) {
37   setLoading(false);
38   return;
39 }
40
41 setTopGenre(favoriteGenre);
42
43 // Descargar catalogo para buscar coincidencias
44 const catalogRes = await catalogApi.get(`/${type === 'movie' ?
     'movies' : 'series'}?page=0&size=100`);
45 const allItems = catalogRes.data.content;
46
47 // Filtrar que tengan el genero favorito y que NO las haya visto
     todavia
48 const suggestions = allItems.filter(item =>
49   item.genres?.includes(favoriteGenre) &&
     !watchedParentIds.has(item.id)
50 );
51
52 // Ordenar por la mejor nota de TMDB
53 suggestions.sort((a, b) => (b.tmdbRating || 0) - (a.tmdbRating ||
     0));
54
55 setRecommended(suggestions.slice(0, 10)); // Guardar el Top 10
```

Listado 6.7: Cálculo del género favorito y filtrado de sugerencias (`Recommendations.jsx`).

El algoritmo trata distinto las películas y las series porque el género de una serie no viene directamente en el historial de un episodio. Primero hay que saber a qué serie pertenece cada episodio visto (`ep.seriesId`) y leer sus géneros, que ya vienen incluidos en el propio DTO del episodio (`seriesGenres`), para no tener que hacer una segunda petición al catálogo.

`ContinueWatching.jsx` resuelve un problema distinto: si un usuario ve varios episodios seguidos de la misma serie, aparecería una entrada por cada episodio en el carrusel de *Continuar viendo*. El historial ya llega ordenado del más reciente al más antiguo desde el *backend*, así que el componente lo recorre una sola vez con un `Set` de los títulos de serie que ya ha visto:

```
const seenSeries = new Set();
const deduplicatedItems = [];

for (const item of validItems) {
  if (item.mediaType === 'episode') {
    const seriesName = item.mediaDetails.seriesTitle;
    // Si ya tenemos un episodio más reciente de esta serie en la
    lista, se salta
    if (seenSeries.has(seriesName)) {
      continue;
    }
    seenSeries.add(seriesName);
  }
  deduplicatedItems.push(item);
}
```

Listado 6.8: Quedarse con el episodio más reciente de cada serie (`ContinueWatching.jsx`).

Como el historial ya viene ordenado del episodio más reciente al más antiguo, el primer episodio de cada serie que encuentra el bucle es siempre el más reciente. Cualquier episodio de esa misma serie que aparezca después se descarta con el `continue`. Antes de este paso, el componente también quita cualquier entrada cuyo progreso pase del 95 % de su duración, por considerarla ya terminada.

### 6.1.9 Propagación de errores internos con Spring Security 6

Por defecto, Spring Security 6 bloquea el *forward* interno que hace Spring Boot hacia `/error` cuando la petición no lleva un JWT válido. Esto tapaba cualquier error real del servidor (`500 Internal Server Error`) detrás de una respuesta de acceso denegado (`403 Forbidden`), lo que hacía difícil depurar: un fallo dentro de un método de servicio se veía como un problema de permisos, y no como el error que era en realidad.

La solución fue añadir una excepción para el tipo de *dispatch* de error en la cadena de filtros de `SecurityConfig`, en cada microservicio:

```
.authorizeHttpRequests(auth -> auth
        .dispatcherTypeMatchers(DispatcherType.ERROR).permitAll()
        // ... resto de reglas de autorizacion
)
```

Listado 6.9: Excepción para el *forward* interno hacia `/error` (`SecurityConfig.java`).

Con esta línea, el *forward* hacia `/error` se completa bien y el `GlobalExceptionHandler` de cada microservicio recibe el error real, en vez de un `403` que escondía el problema de fondo.

### 6.1.10 Despliegue

Todo el sistema se despliega con **Docker Compose**, que orquesta los cinco microservicios de *backend*, el *frontend* y la infraestructura auxiliar (PostgreSQL, RabbitMQ y Redis) como contenedores independientes sobre una red interna común. Para la persistencia de los servicios de infraestructura se usan volúmenes gestionados por Docker (*named volumes*), sin exponer su almacenamiento interno al equipo donde corre Docker.

El almacenamiento de vídeo se gestiona con un *bind mount* sobre una ruta relativa del proyecto. Así el *streaming-service* y el *transcoding-service* acceden al mismo contenido físico sin tener que sincronizarlo entre contenedores.

Para que el proyecto se pueda ejecutar tanto en contenedores como directamente en local durante el desarrollo, varias rutas y direcciones se inyectan por variable de entorno en `docker-compose.yml`: la ruta de almacenamiento de vídeo (`VIDEOS_STORAGE_PATH`), la URL de PostgreSQL y los hosts de RabbitMQ y Redis. Cada variable sobrescribe automáticamente la propiedad equivalente del `application.yaml` de cada microservicio, sin tener que tocar el fichero. Fuera de los contenedores, cada propiedad usa el valor por defecto que tiene puesto para ejecutarlo en local (`localhost`).

Por último, la activación de la extensión `pg_trgm` de PostgreSQL (Apartado 6.1.4) se automatizó con el mecanismo de inicialización nativo de Docker (`/docker-entrypoint-initdb.d`), así que queda disponible desde el primer arranque del contenedor.

## 6.2 Pruebas

La validación de Vodeo se organizó en cuatro niveles: pruebas unitarias, pruebas de integración automatizadas, pruebas manuales de API con Postman y pruebas de aceptación.

### 6.2.1 Pruebas unitarias

La lógica de negocio se validó con pruebas unitarias en JUnit 5 y Mockito, siguiendo un enfoque de *Behavior-Driven Development* (BDD). Cada método de prueba lleva una anotación `@DisplayName` que describe en lenguaje natural qué comportamiento comprueba (por ejemplo, *"register - email duplicado → lanza ConflictException"*), en vez de quedarse solo con el nombre técnico del método.

Cada clase de servicio se prueba sola, sin depender de nada externo: ni de la base de datos, ni de RabbitMQ, ni de las llamadas HTTP reales a otros servicios. Estas dependencias se sustituyen por *Test Doubles* de Mockito con `@Mock` e `@InjectMocks`. El comportamiento se define con `when(...).thenReturn(...)`, y después se comprueban las llamadas relevantes con `verify(...)`. Así todas las pruebas se ejecutan en milisegundos, sin tener que levantar ni PostgreSQL, ni Redis, ni RabbitMQ de verdad.

El Listado 6.10 muestra un caso real de `UserServiceTest`: cuando el correo ya existe en la base de datos, `register` tiene que lanzar `ConflictException` y no debe llegar a llamar a `save`.

```java
@Test
@DisplayName("register — email duplicado → lanza ConflictException")
void testRegister_EmailAlreadyExists() {
    RegisterRequest request = new RegisterRequest();
    request.setEmail("test@vodeo.com");

    when(userRepository.findByEmail(request.getEmail()))
        .thenReturn(Optional.of(mockUser));

    assertThrows(ConflictException.class, () ->
    userService.register(request));
    verify(userRepository, never()).save(any());
}
```

Listado 6.10: Prueba unitaria del registro con email duplicado (`UserServiceTest.java`).

La tabla 6.1 resume las ocho clases de prueba y lo que comprueba cada una.

**Un caso aparte: probar un método privado por *reflection*.** `buildAbsCommand` es un método `private` de `VideoTranscodingWorker`, así que no se puede llamar directamente desde fuera de la clase. Tampoco sirve probar el único método público que sí es accesible (`processVideo`), porque ese lanza un proceso real de FFmpeg contra el sistema de archivos. Para comprobar solo la construcción del comando, sin llegar a ejecutar FFmpeg de verdad, `VideoTranscodingWorkerTest` usa `ReflectionTestUtils.invokeMethod` de Spring Test. Este método invoca `buildAbsCommand` saltándose la restricción de `private`, y comprueba después que la lista de argumentos resultante trae las resoluciones y *bitrates* correctos, junto con el nombre de la *playlist* maestra.

| Clase de prueba | Microservicio | Comportamiento validado |
|---|---|---|
| `UserServiceTest` | *users-service* | Registro (éxito y conflicto por email duplicado), autenticación (éxito, usuario inexistente, contraseña incorrecta), consulta y edición de perfil. |
| `JwtServiceTest` | *users-service* | Generación de tokens, validez de la firma, detección de manipulación, extracción de los datos guardados en el token (correo y rol) y detección de expiración. |
| `WatchlistServiceTest` | *users-service* | Alternar favorito (*toggle*): añadir cuando no existe, eliminar cuando ya existe, y paginación de la lista guardada. |
| `PlaybackHistoryServiceTest` | *users-service* | Guardado de un nuevo progreso, actualización de uno existente (*Upsert*) y validación de usuario inexistente. |
| `AdminUserServiceTest` | *users-service* | Borrado en cascada de un usuario normal, protección frente al borrado de administradores y usuario inexistente. |
| `CatalogServiceTest` | *catalog-service* | Consulta por id (éxito y no encontrado), consulta por lote, *Upsert* de géneros al crear contenido, borrado con evento asíncrono y cálculo del episodio siguiente. |
| `VideoServiceTest` | *streaming-service* | La subida de un vídeo `.mp4` publica el evento en RabbitMQ. La subida de un subtítulo no lo hace. |
| `VideoTranscodingWorkerTest` | *transcoding-service* | Construcción correcta del comando FFmpeg *multi-bitrate* (resoluciones, *bitrates* y nombre del índice maestro). |

Tabla 6.1: Clases de prueba unitaria desarrolladas para el backend de Vodeo.

```
[INFO] Results:
[INFO]
[INFO] Tests run: 24, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
```

Figura 6.1: Resultado de la ejecución de las pruebas unitarias del backend.

### 6.2.2 Pruebas de integración automatizadas

Las pruebas unitarias aíslan los servicios con *mocks* de Mockito, sin tocar la base de datos. Para comprobar que la capa de persistencia también funciona, se utiliza `@DataJpaTest` de Spring Boot, que levanta los repositorios de Spring Data JPA contra una base de datos H2 en memoria sin necesidad de cargar toda la aplicación. Se hizo una clase de prueba por cada uno de los repositorios de *catalog-service* y *users-service* (los dos microservicios que usan JPA). La tabla 6.2 las resume.

| Clase de prueba | Microservicio | Comportamiento validado |
|---|---|---|
| `MovieRepositoryTest` | *catalog-service* | El `@EntityGraph` carga la película con sus géneros en una sola consulta. |
| `GenreRepositoryTest` | *catalog-service* | `findByName`, base del patrón *Upsert* de géneros. |
| `EpisodeRepositoryTest` | *catalog-service* | Cálculo del siguiente episodio para el *autoplay*, incluyendo el caso sin siguiente. |
| `SeasonRepositoryTest` | *catalog-service* | Cálculo de la siguiente temporada al cruzar el límite de la actual. |
| `SeriesRepositoryTest` | *catalog-service* | `findByIdIn`, usado en la Composición de API de la *Watchlist*. |
| `ReviewRepositoryTest` | *catalog-service* | Detección de una reseña ya existente del mismo usuario. |
| `SubtitleRepositoryTest` | *catalog-service* | Borrado en cascada de los subtítulos al borrar su episodio. |
| `UserRepositoryTest` | *users-service* | `findByEmail` y la restricción `unique` del email en la base de datos. |
| `WatchlistRepositoryTest` | *users-service* | Consulta que sostiene el *toggle* de favoritos. |
| `PlaybackHistoryRepositoryTest` | *users-service* | Recuperación del progreso guardado para ”Seguir viendo”. |

Tabla 6.2: Pruebas de integración automatizadas, una clase por cada repositorio del proyecto.

### 6.2.3 Pruebas manuales de API con Postman

Se probó la API con **Postman** haciendo peticiones HTTP directas contra los microservicios en marcha, para ver tanto la respuesta como el código de estado devuelto. A continuación se muestran dos ejemplos representativos.

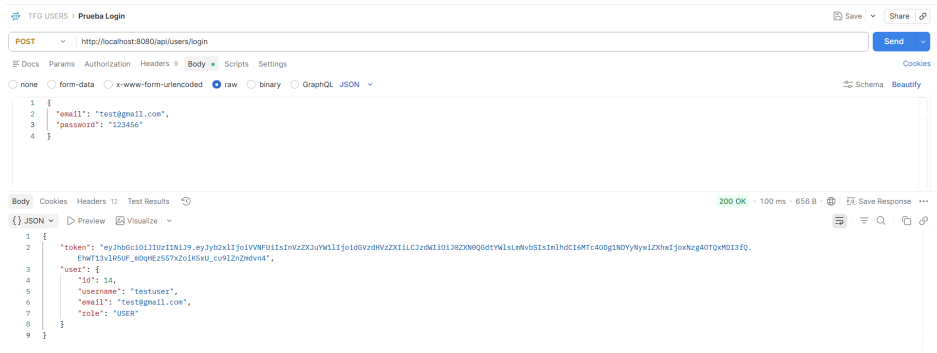



Figura 6.2: Autenticación de un usuario: la respuesta incluye el `AuthResponse` con el JWT.

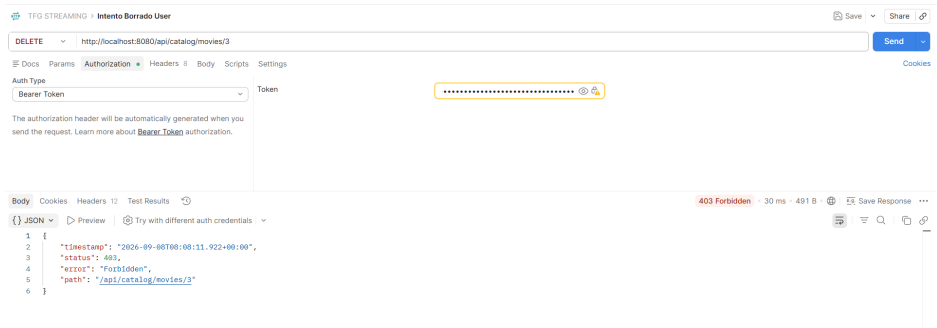



Figura 6.3: Intento de borrado de una película con un token de rol USER: el servidor responde `403 Forbidden`.

### 6.2.4 Pruebas de aceptación

Por último, se comprobó manualmente que la aplicación cumple los requisitos funcionales desde el punto de vista de un usuario final, navegando por la SPA: registro e inicio de sesión, exploración y búsqueda del catálogo, reproducción de una película y de una serie con cambio de calidad, guardado y borrado de favoritos, escritura de una reseña, uso del panel de administración para dar de alta contenido nuevo, etc. Estas pruebas de aceptación las realizó el autor.

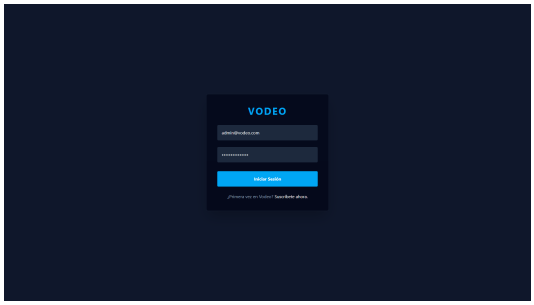
Capítulo 7

# Solución desarrollada

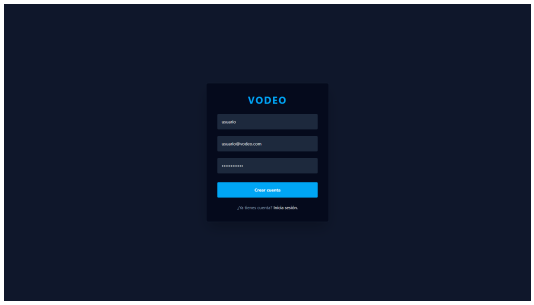
En este capítulo vamos a mostrar las características principales de la aplicación desarrollada, realizando una visita guiada por las principales funcionalidades acompañada de capturas de pantalla de la aplicación final.

## 7.1 Registro e inicio de sesión

El primer contacto de cualquier usuario con Vodeo es la pantalla de registro o de inicio de sesión. Ambas vistas tienen el mismo diseño oscuro que el resto de la aplicación y validan los datos introducidos. Si las credenciales no son correctas o el correo ya está registrado, aparece un aviso indicando el error.




(a) Inicio de sesión.



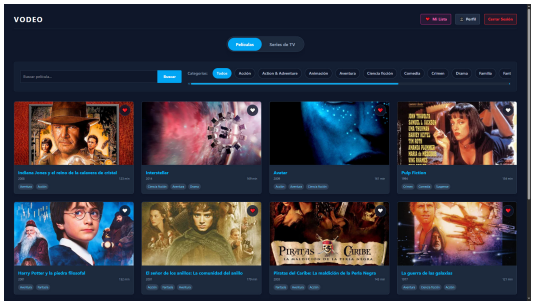

(b) Registro de un nuevo usuario.

Figura 7.1: Pantallas de registro e inicio de sesión de Vodeo.

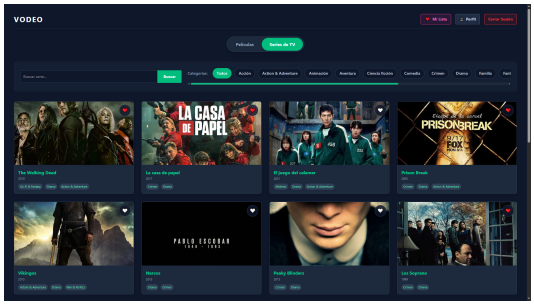
## 7.2 Catálogo

Una vez el usuario inicia sesión, llega al catálogo, la vista principal de la aplicación. Las películas y series se muestran en pestañas separadas, en una cuadrícula con su póster, título, año y géneros. Desde aquí se puede buscar por título, aunque no se escriba exactamente igual,

y moverse por los distintos géneros con un carrusel horizontal.




(a) Pestaña de películas.




(b) Pestaña de series de TV.

Figura 7.2: Vista principal del catálogo.

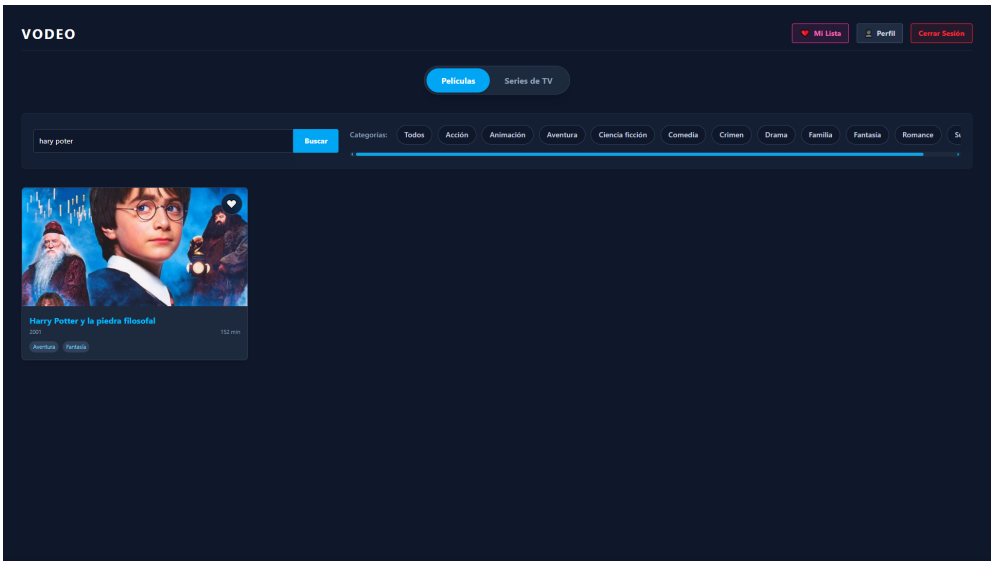



Figura 7.3: Resultado de una búsqueda de contenido en el catálogo.

## 7.3 Ficha de contenido y reproducción

Al hacer clic en una película o una serie del catálogo, el usuario llega a su ficha, donde se muestra el póster, la sinopsis, el director, el reparto principal y los géneros, además de si ya se ha visto ese contenido. Desde esta pantalla se puede añadir a favoritos, gestionar los subtítulos y dejar una reseña. Al pulsar el botón “Reproducir” se abre el vídeo, que va cambiando de calidad según la conexión disponible. En el caso de una serie, al terminar un episodio pasa al siguiente automáticamente tras una cuenta atrás.

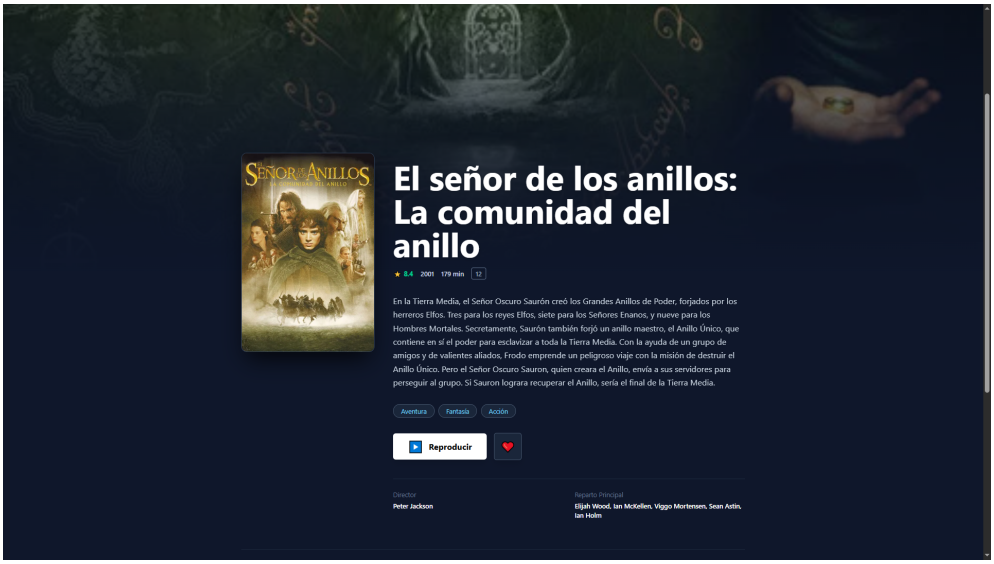



Figura 7.4: Ficha de una película, con su información y el botón de reproducción.

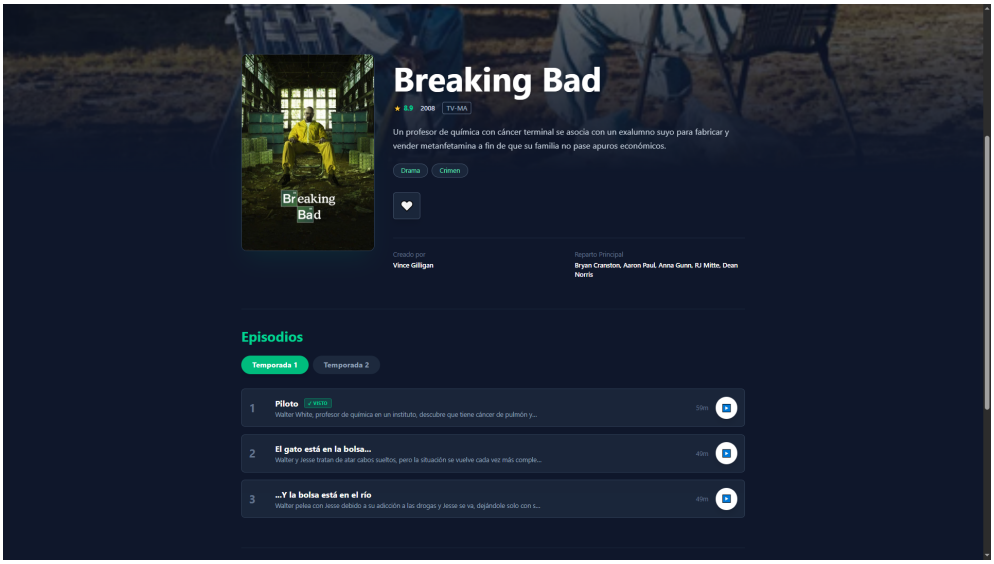



Figura 7.5: Ficha de una serie con la navegación por temporadas y episodios.




Figura 7.6: Reproducción de una película.

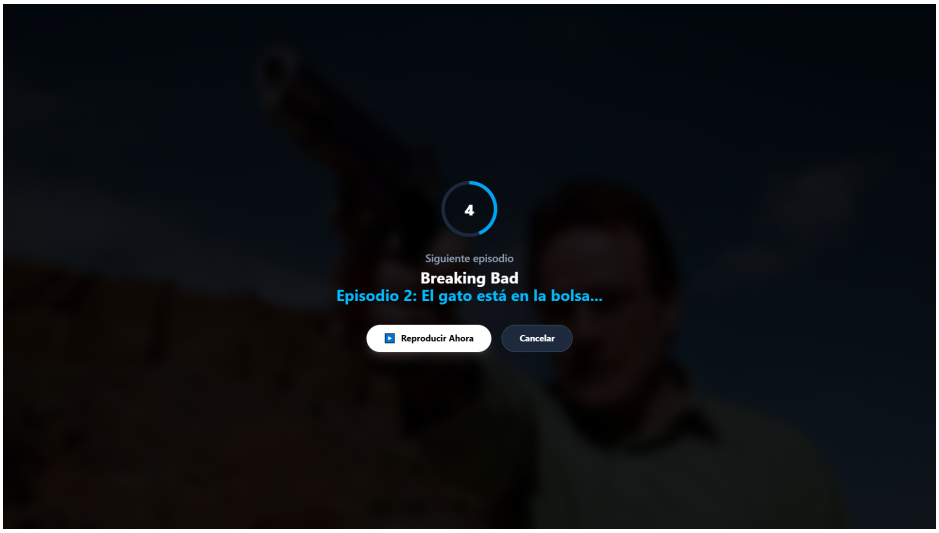



Figura 7.7: Aviso de reproducción automática del siguiente episodio.

## 7.4 Reseñas

Cada ficha de contenido tiene una sección de reseñas donde los usuarios pueden valorar películas y series con estrellas y dejar un comentario. Un administrador puede borrar cualquier reseña de la plataforma para moderar el contenido.

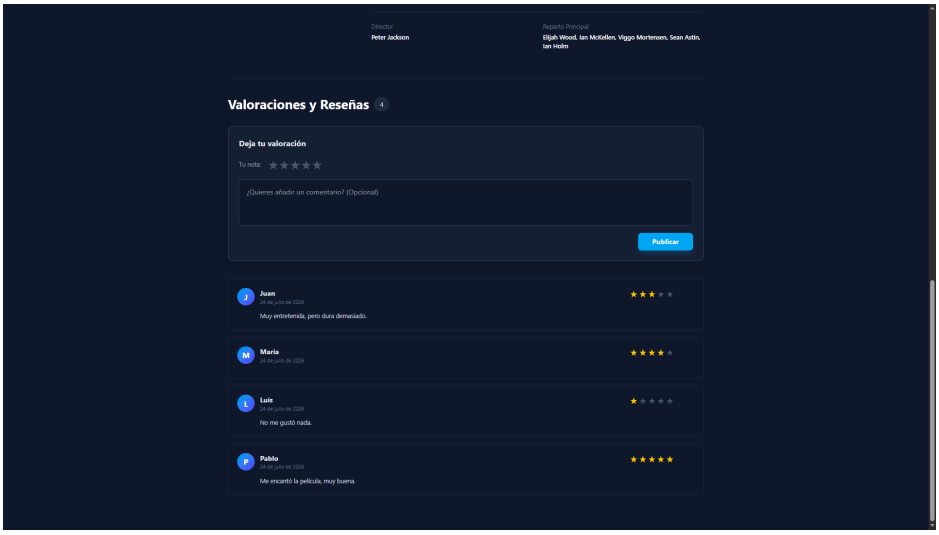



Figura 7.8: Sección de reseñas de un contenido.

## 7.5 Mi Lista

Desde el catálogo o desde la ficha de contenido, el usuario puede añadir películas y series a su lista de favoritos, llamada “Mi Lista”. En esta vista aparecen tanto las películas como las series guardadas, marcando de qué tipo es cada una, y se pueden borrar directamente desde ahí.

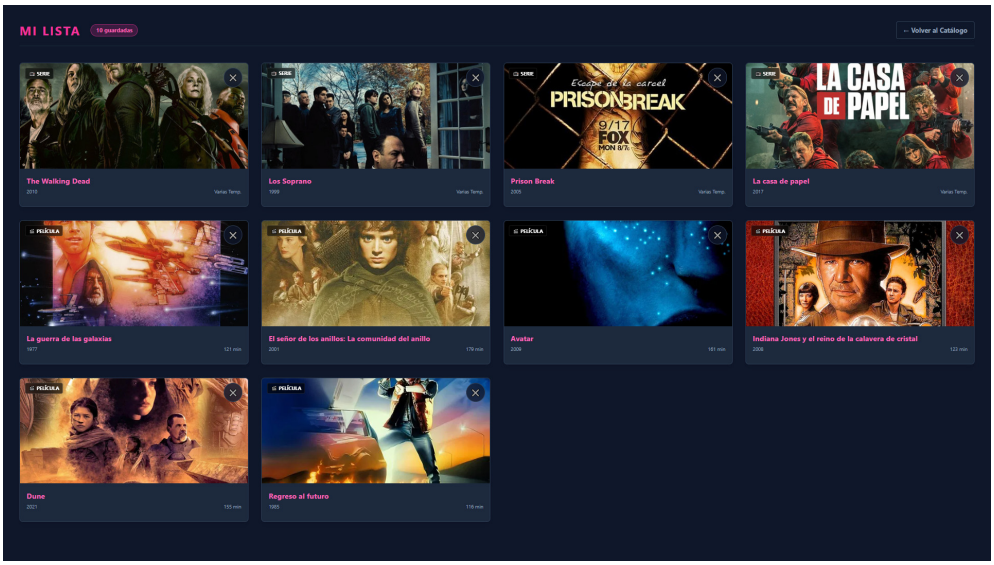



Figura 7.9: Vista de “Mi Lista” con contenido guardado por el usuario.

## 7.6 Seguir viendo y recomendaciones

En la parte de abajo del catálogo, Vodeo muestra un carrusel de "Seguir viendo" con el contenido que el usuario dejó a medias, para poder retomarlo justo por donde lo dejó. Debajo, un carrusel de recomendaciones sugiere contenido según los géneros que más ha visto el usuario, sin repetir lo que ya ha visto.

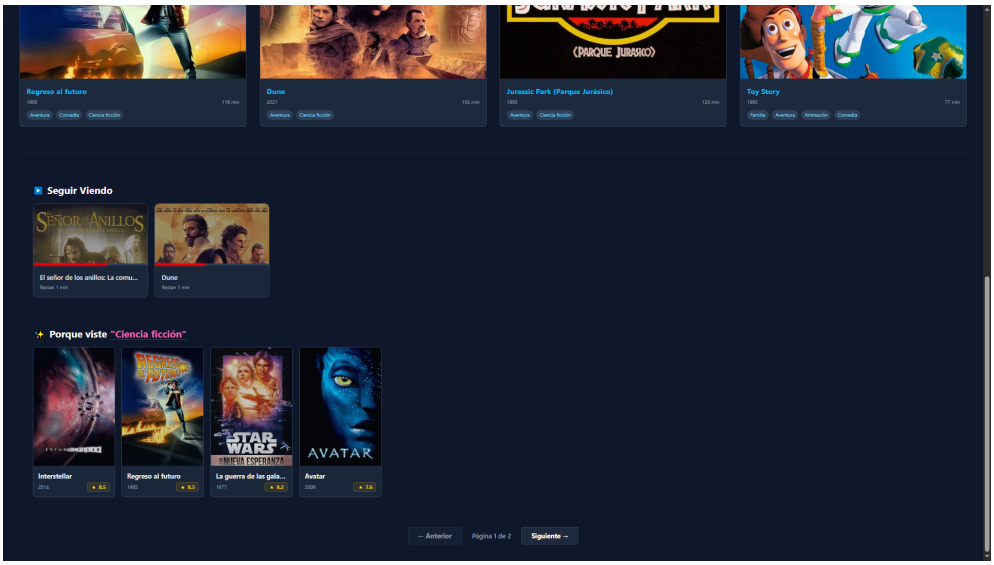



Figura 7.10: Carrusel de "Seguir viendo" y de recomendaciones en la vista principal del catálogo.

## 7.7 Panel de administración

Los usuarios con rol de administrador tienen acceso a un panel desde el que pueden dar de alta nuevo contenido. El formulario permite importar automáticamente los datos de una película o serie desde TMDB u OMDb con solo escribir el título, y cambiar a mano cualquier campo (póster, sinopsis, géneros, etc.). En el caso de una serie, las temporadas y los episodios se gestionan desde la propia ficha de la serie, donde el administrador puede crear nuevas temporadas y subir el vídeo de cada episodio.

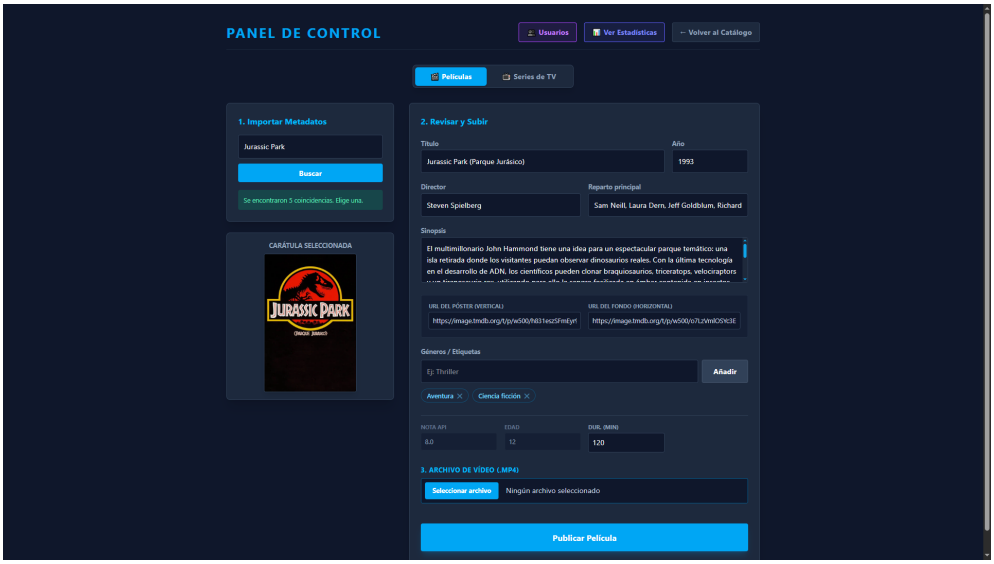



Figura 7.11: Formulario de alta de una película, con importación automática de metadatos.

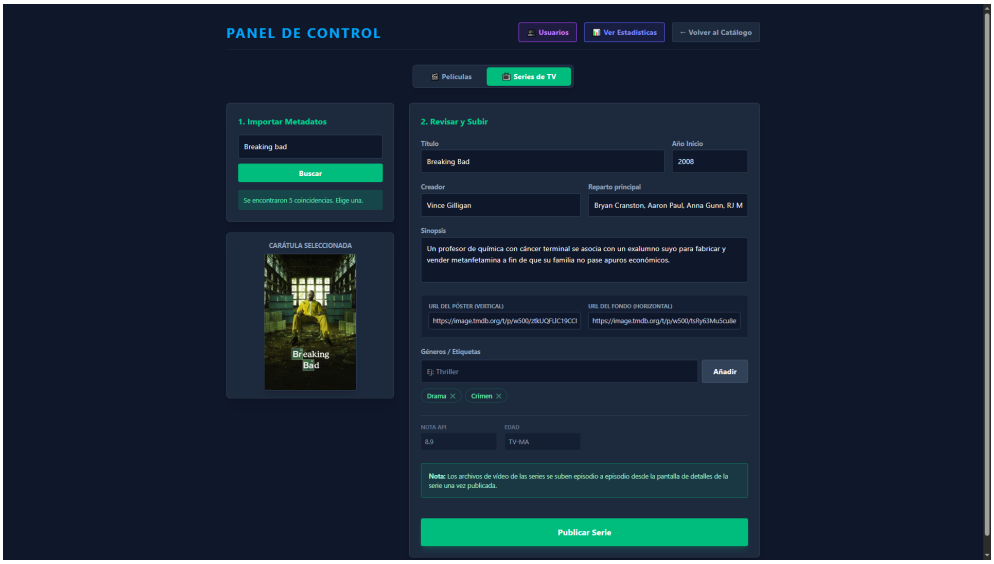



Figura 7.12: Formulario de alta de una serie, con importación automática de metadatos.

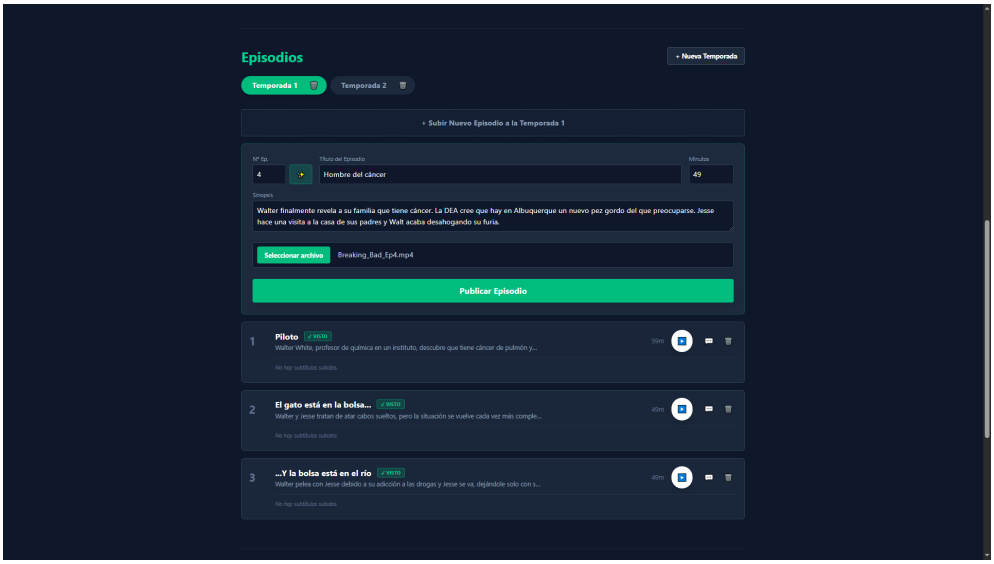



Figura 7.13: Gestión de temporadas y episodios desde la ficha de una serie.

## 7.8 Panel de gestión de usuarios y analítica

Además, Vodeo tiene un panel de gestión de usuarios desde el que el administrador puede ver y moderar las cuentas registradas, y un panel con estadísticas de la plataforma, como el número total de usuarios registrados.

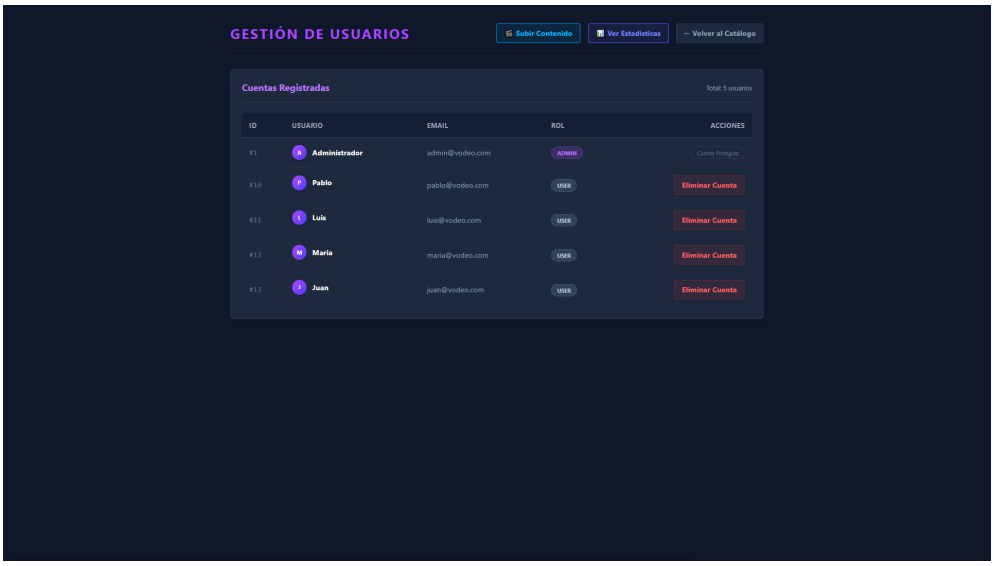



Figura 7.14: Panel de gestión de usuarios.

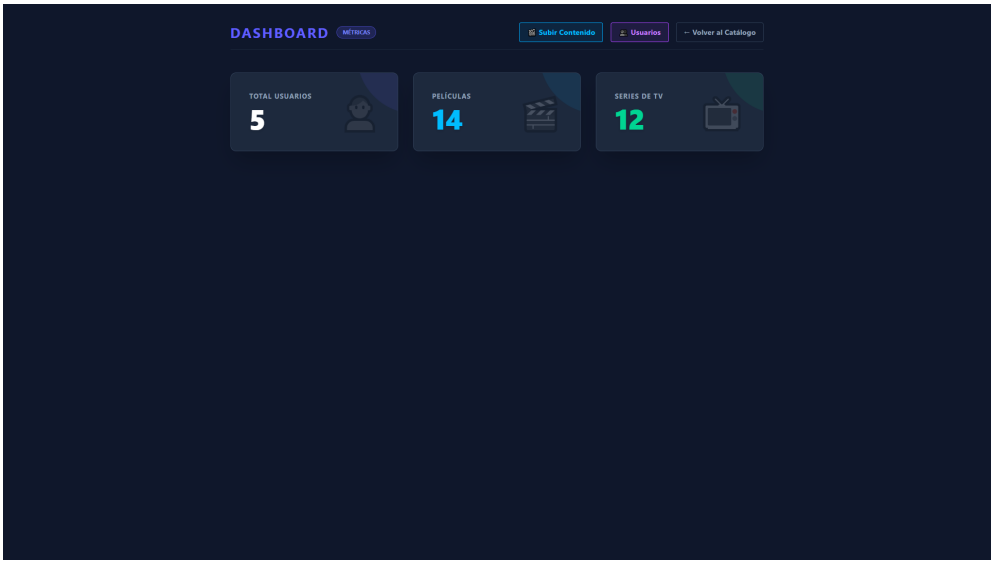



Figura 7.15: Panel analítico con métricas de la plataforma.

## 7.9 Gestión del perfil

Por último, cualquier usuario puede entrar en su perfil para cambiar su nombre de usuario o su contraseña. Para cambiar la contraseña hace falta escribir la contraseña actual antes de aplicar el cambio, pero para cambiar el nombre de usuario no.

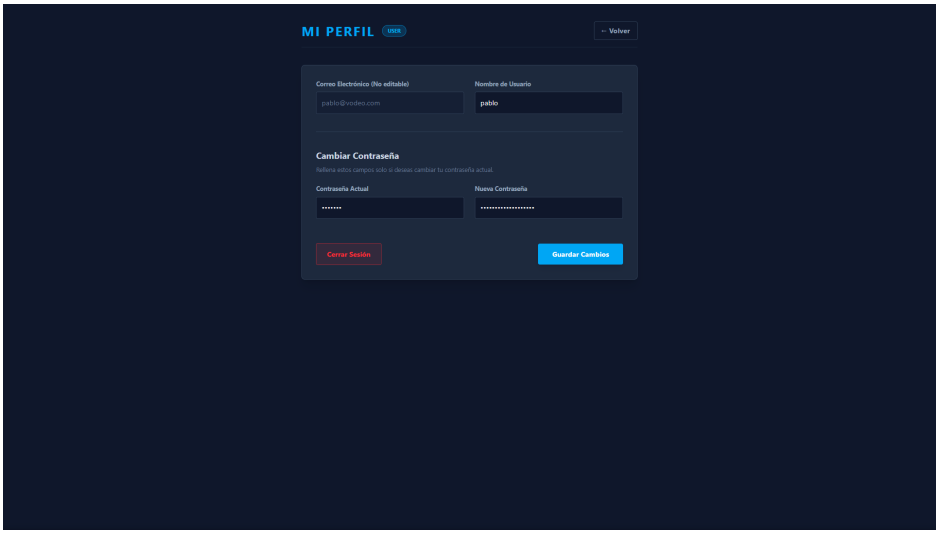



Figura 7.16: Panel de gestión del perfil de usuario.

Capítulo 8

# Conclusiones y trabajo futuro

## 8.1 Grado de cumplimiento de los objetivos

Al comenzar este trabajo, el objetivo era construir Vodeo: una plataforma de streaming propia, capaz de sostener por sí misma todo lo que normalmente damos por hecho al usar Netflix o Plex sin pensar en ello. No se trataba solo de que la aplicación funcionase, sino de entender y resolver por mi cuenta los problemas de ingeniería que esas plataformas esconden tan bien que ni siquiera nos planteamos que existen. Terminado el desarrollo, puedo decir que ese objetivo se cumplió, y que en el camino el proyecto acabó creciendo más de lo que preveía al principio, tanto en funcionalidades como en la cantidad de decisiones de diseño que tuve que ir tomando sobre la marcha.

La arquitectura de microservicios planteada al inicio se materializó tal como estaba pensada: los servicios quedaron efectivamente aislados entre sí, de forma que un problema puntual en uno no compromete al resto del sistema, que era precisamente la limitación que motivó este proyecto frente a las soluciones monolíticas. Sobre esa base se fueron incorporando la gestión de usuarios y permisos, un catálogo que se rellena solo a partir de fuentes externas en lugar de a mano, y un motor de vídeo que, siendo la parte que al principio más me preocupaba, terminó siendo de lo más satisfactorio de ver funcionando, adaptando la calidad de reproducción en tiempo real según la conexión de quien lo esté viendo. El resto de funcionalidades, como favoritos, continuar viendo donde uno lo dejó, reseñas o recomendaciones, no eran estrictamente necesarias para que el sistema funcionase, pero son las que hacen que Vodeo se sienta como un producto real.

## 8.2 Aportación personal y profesional

Este proyecto me ha permitido acercarme de verdad a algo que me interesaba desde antes de empezarlo: entender cómo funciona el streaming de vídeo por dentro, más allá de darle al

play y que simplemente funcione. Siempre me ha llamado la atención el mundo del multimedia, y Vodeo me dio la excusa perfecta para dejar de usar plataformas como espectador y empezar a construir una desde el otro lado.

La parte que más me hizo aprender fue la transcodificación con FFmpeg. Antes de este trabajo no tenía ni idea de cómo funcionaba HLS por dentro, ni de qué hace realmente un códec cuando convierte un archivo de una resolución a varias, y tuve que entenderlo a base de ir probando, equivocándome y volviendo a intentarlo hasta que funcionase. No es solo que la tecnología fuera nueva para mí, es que trabajar con FFmpeg significa lanzar y controlar un proceso externo del sistema operativo desde Java, algo bastante distinto a programar un servicio o un controlador REST. Hay que construir el comando, leer su salida sin bloquear el hilo que lo lanza, y comprobar el código con el que termina para saber si algo ha ido mal.

El resto del proyecto también dejó su huella. La mensajería asíncrona con RabbitMQ, la caché con Redis, la orquestación de todo el sistema con Docker Compose. Fueron piezas que aprendí porque el propio proyecto las pedía, y que en conjunto han sido probablemente la parte del grado en la que más he tenido que resolver cosas por mi cuenta, sin nadie que me dijera de antemano cómo se hacía.

## 8.3 Trabajo futuro

Vodeo cubre el recorrido completo de una plataforma VOD, pero según iba avanzando el desarrollo fueron apareciendo ideas, algunas de las cuales, por falta de tiempo, se quedaron fuera de este trabajo.

Una posible mejora sería automatizar parte de las pruebas que ahora mismo se hacen a mano con Postman. Herramientas como Testcontainers permitirían levantar PostgreSQL, RabbitMQ y Redis reales dentro de los propios tests de integración, y añadir pruebas end to end del frontend con algo como Playwright o Cypress permitiría terminar con la validación manual.

También queda camino por recorrer en la parte de infraestructura. El despliegue actual con Docker Compose ejecuta una única instancia de cada servicio, así que migrar a un orquestador como Kubernetes permitiría escalar de forma independiente los servicios que más lo necesiten, sobre todo el de transcodificación en los momentos en los que se suba más contenido a la vez. En un entorno de producción real, además, tendría sentido servir los segmentos de vídeo a través de una CDN en lugar de directamente desde el servidor.

Además sería interesante mejorar el sistema de recomendaciones, que hoy en día solo se fija en los géneros que más ha visto cada usuario. Sería interesante que también tuviera en cuenta lo que ven otros usuarios con gustos parecidos, para que las sugerencias fueran más acertadas. También quedaría pendiente sacar la plataforma de la web y llevarla a una

aplicación móvil, reutilizando toda la lógica que ya existe en el backend, y añadir la posibilidad de tener varios perfiles dentro de una misma cuenta, como hacen las plataformas comerciales, cada uno con su propio historial y sus propias recomendaciones.

Otra funcionalidad que quedaría bien es avisar al usuario cuando se añade contenido nuevo relacionado con algo que ya sigue, por ejemplo un episodio nuevo de una serie que tiene guardada en su lista. La estructura de datos necesaria ya existe, así que sería cuestión de detectar ese evento en el catalog-service y decidir cómo hacérselo llegar al usuario, ya sea con una notificación dentro de la propia aplicación o por correo.

Otra idea que me parece atractiva es la de las watch parties, es decir, que dos o más usuarios puedan ver el mismo contenido a la vez de forma sincronizada aunque estén en dispositivos distintos con acceso a un chat para hablar. Técnicamente encajaría bien con la infraestructura de mensajería que ya existe en el sistema, usándola para propagar en tiempo real las acciones de reproducción (pausar, reanudar, saltar) entre todos los participantes de la sesión.

# Apéndices

Apéndice A

# Material adicional

Este apéndice recoge los prototipos de interfaz de usuario (*mockups*) elaborados durante la fase de análisis de la plataforma Vodeo, previos al desarrollo de la interfaz final. Estos diseños sirvieron como referencia visual para definir la estructura y navegación de cada pantalla del sistema, y fueron utilizados como guía durante la implementación del frontend.

## A.1 Prototipos de interfaz de usuario

### Inicio de Sesión

Como se muestra en la Figura A.1, el usuario inicia sesión ingresando su correo electrónico y su contraseña. Desde esta pantalla también se proporciona un enlace a la página de registro para nuevos usuarios.

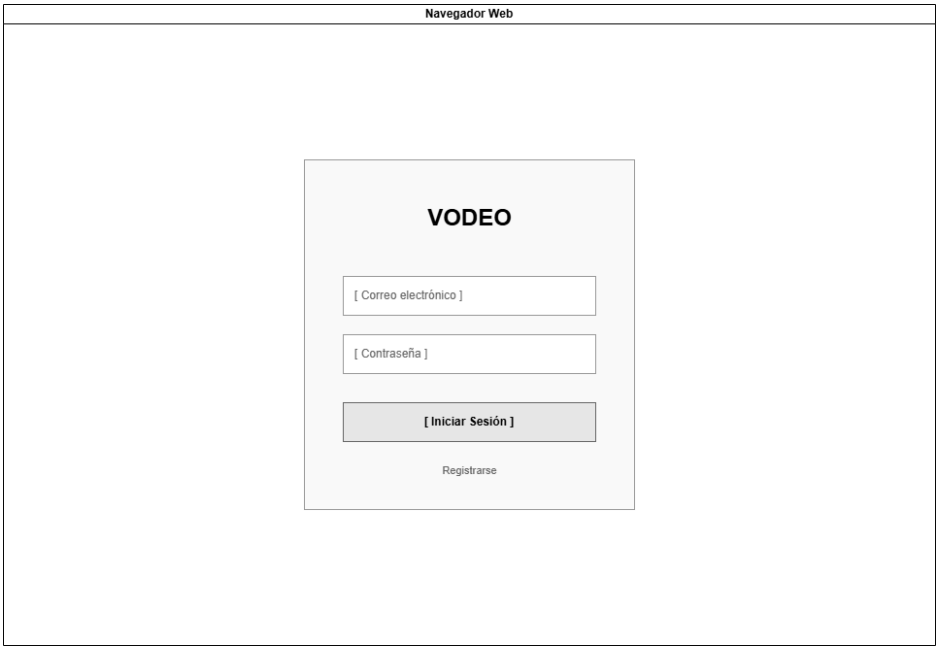



Figura A.1: Prototipo de la pantalla de inicio de sesión.

## Registro

En la Figura A.2 se muestra la pantalla de registro, donde el usuario completa un formulario con su nombre, correo y contraseña para crear una nueva cuenta en la plataforma.

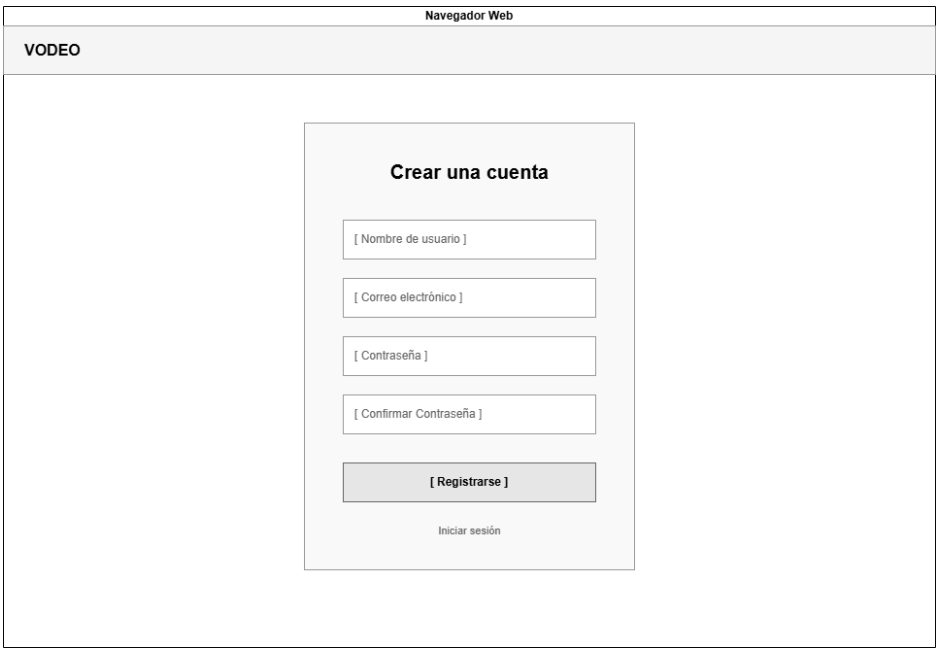



Figura A.2: Prototipo de la pantalla de registro.

### Catálogo

Como se observa en la Figura A.3, esta es la pantalla principal a la que accede el usuario tras iniciar sesión. Presenta el contenido organizado en cuadrículas y filas (para contenido recomendado o en curso), e incluye controles para la búsqueda directa y el filtrado por categorías.

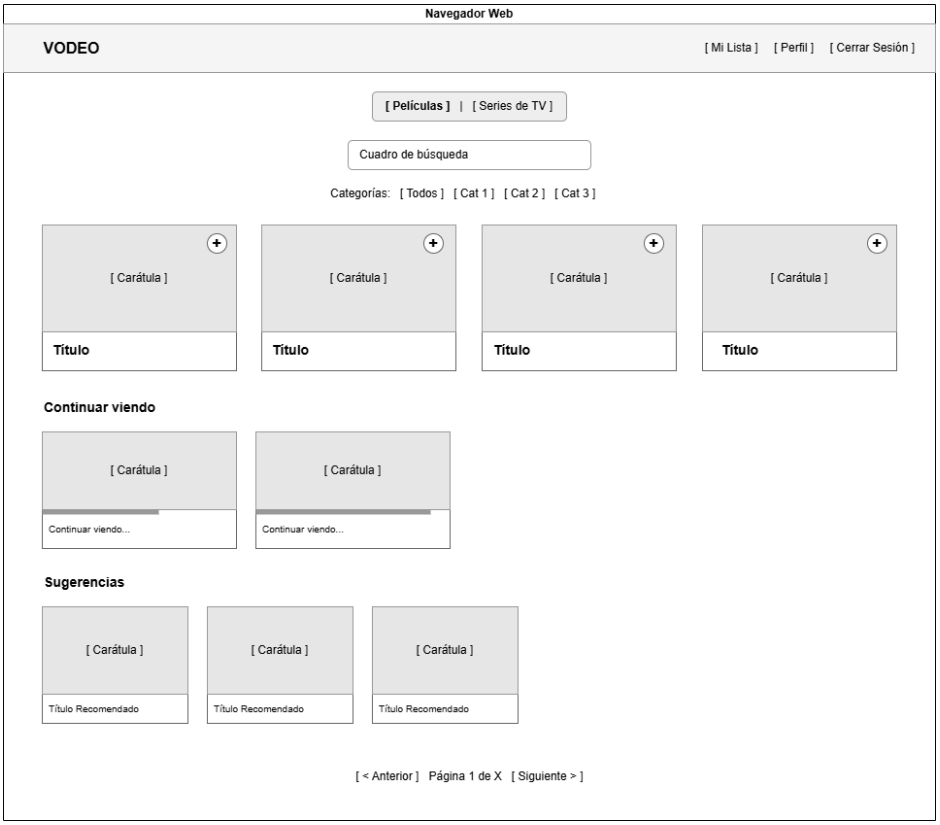



Figura A.3: Prototipo de la pantalla principal del catálogo.

### Detalle de Película

La Figura A.4 enseña la ficha de información ampliada para películas. Muestra la carátula, los metadatos principales (año, duración, géneros), la sinopsis, un botón de acción principal para iniciar la reproducción y una sección destinada a la lectura y publicación de reseñas.

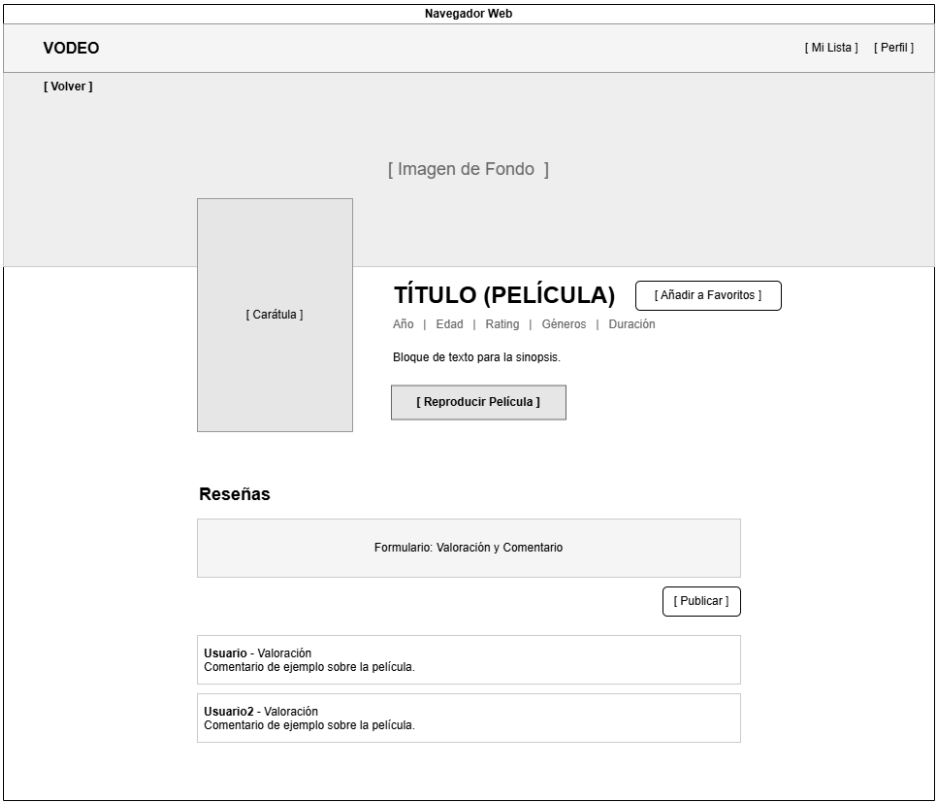



Figura A.4: Prototipo de la pantalla de detalle de película.

### Detalle de Serie

Como se detalla en la Figura A.5, la ficha correspondiente a las series mantiene una estructura similar a la de las películas en cuanto a presentación de metadatos y reseñas, pero añade un selector de temporadas con su lista de episodios disponibles.

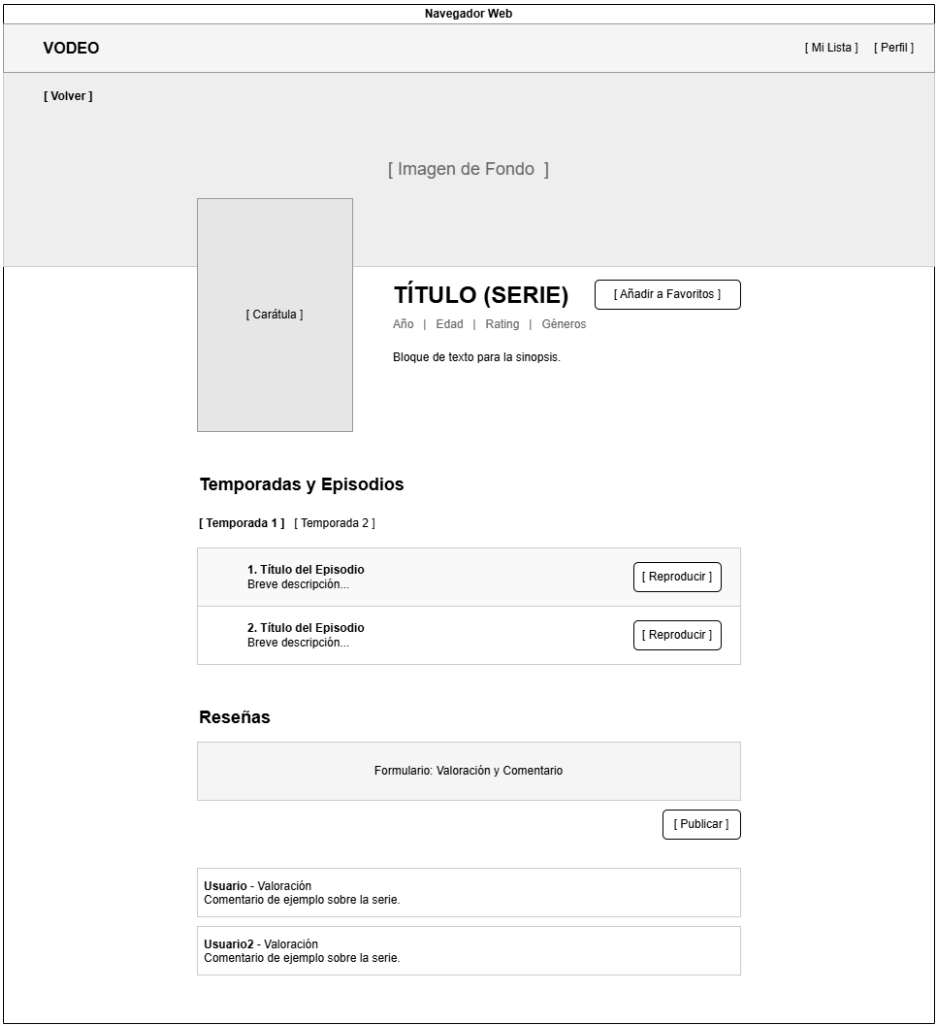



Figura A.5: Prototipo de la pantalla de detalle de serie.

### Reproductor Multimedia

La Figura A.6 muestra el diseño del reproductor de vídeo. Incluye una barra de controles inferior que gestiona la reproducción, el volumen, el progreso temporal y los subtítulos.




Figura A.6: Prototipo de la pantalla del reproductor multimedia.

### Mi Lista

En esta página, como se muestra en la Figura A.7, el usuario accede a su colección personal de películas y series guardadas. La interfaz permite visualizar el contenido mediante una cuadrícula de carátulas y ofrece la posibilidad de eliminar elementos específicos de la lista.

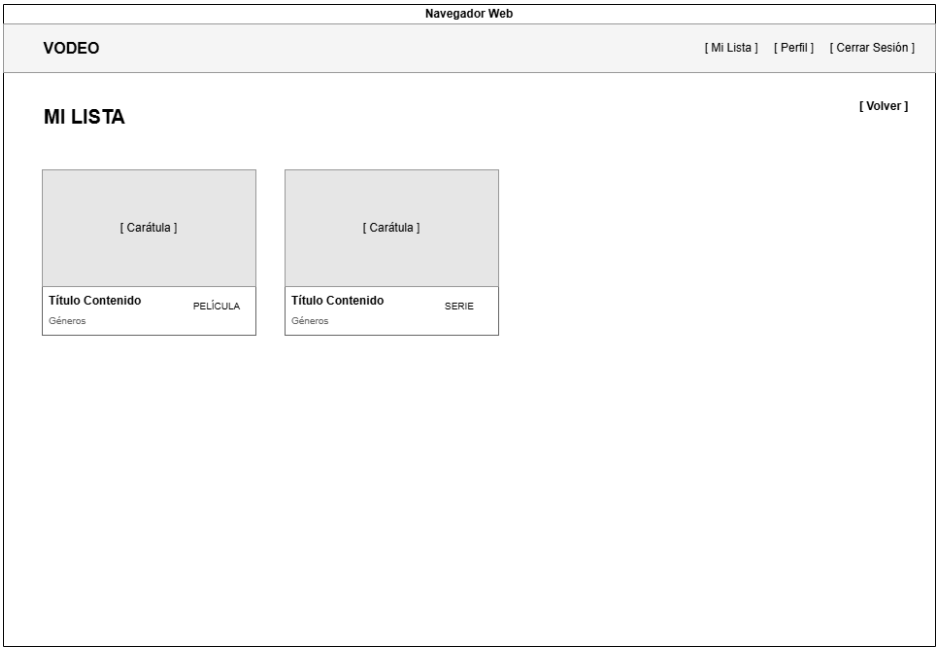



Figura A.7: Prototipo de la pantalla de lista de favoritos.

### Ajustes de Perfil

Como se observa en la Figura A.8, esta pantalla muestra la información del usuario autenticado. Se permite modificar los parámetros.

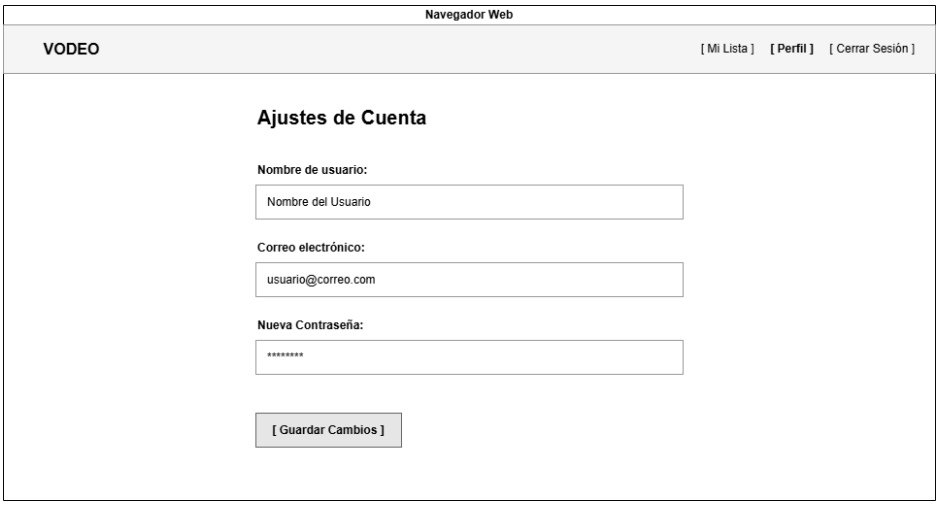



Figura A.8: Prototipo de la pantalla de perfil de usuario.

### Añadir Nuevo Contenido

En la Figura A.9 se muestra el formulario para que el administrador publique nuevas películas o series. Dispone de un bloque superior que permite extraer metadatos automáticamente, agilizando el relleno de campos como el título o la sinopsis. En la parte inferior, se ubica el campo para seleccionar el archivo de vídeo asociado.




Figura A.9: Prototipo del panel de administración: subida de contenido.

### Panel de Estadísticas (Dashboard)

Como se observa en la Figura A.10, la pantalla muestra un resumen de métricas clave del sistema mediante tarjetas. En este prototipo inicial se contempló mostrar el volumen total de usuarios, películas y series registradas.

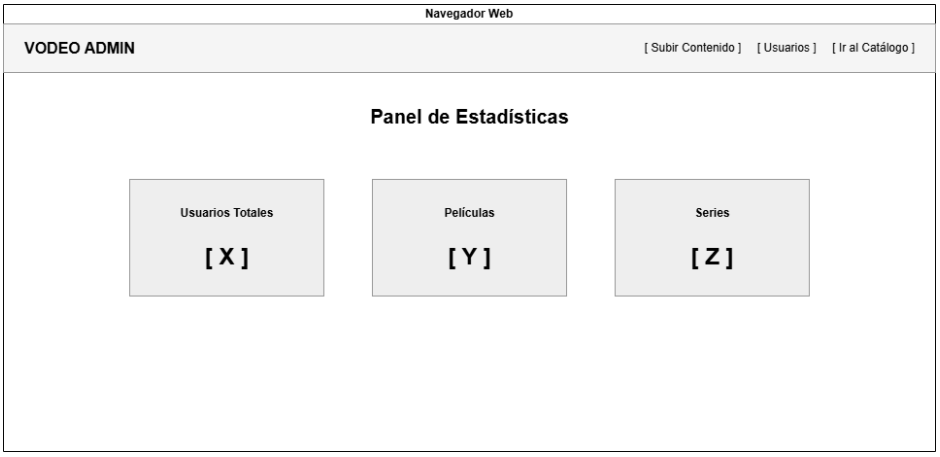



Figura A.10: Prototipo del panel de administración: estadísticas.

### Gestión de Usuarios

En esta pantalla el administrador visualiza una tabla con todos los usuarios registrados en la plataforma. Como se muestra en la Figura A.11, el listado incluye información identificativa, el rol asignado y un botón para eliminar cuentas, restringiendo la eliminación del rol de administrador.




Figura A.11: Prototipo del panel de administración: gestión de usuarios.

# Bibliografía


[1] Oracle, "Java Programming Language," 2026, consultado el 2026-09-09. [En línea]. Disponible en: https://www.java.com/

[2] VMware, "Spring Boot Framework," 2026, consultado el 2026-09-09. [En línea]. Disponible en: https://spring.io/projects/spring-boot

[3] ——, "RabbitMQ: Messaging that just works," 2026, consultado el 2026-09-09. [En línea]. Disponible en: https://www.rabbitmq.com/

[4] FFmpeg Developers, "FFmpeg: A complete, cross-platform solution to record, convert and stream audio and video," 2026, consultado el 2026-09-09. [En línea]. Disponible en: https://ffmpeg.org/

[5] Meta Platforms, "React: The library for web and native user interfaces," 2026, consultado el 2026-09-09. [En línea]. Disponible en: https://react.dev/

[6] Tailwind Labs, "Tailwind CSS: Rapidly build modern websites without ever leaving your HTML," 2026, consultado el 2026-09-09. [En línea]. Disponible en: https://tailwindcss.com/

[7] PostgreSQL Global Development Group, "PostgreSQL: The World's Most Advanced Open Source Relational Database," 2026, consultado el 2026-09-09. [En línea]. Disponible en: https://www.postgresql.org/

[8] Redis Ltd., "Redis: The open source, in-memory data store," 2026, consultado el 2026-09-09. [En línea]. Disponible en: https://redis.io/

[9] Docker Inc., "Docker: Accelerated Container Application Development," 2026, consultado el 2026-09-09. [En línea]. Disponible en: https://www.docker.com/

[10] L. Torvalds, "Git: Distributed Version Control System," 2026, consultado el 2026-09-09. [En línea]. Disponible en: https://git-scm.com/

[11] Postman Inc., "Postman API Platform," 2026, consultado el 2026-09-09. [En línea]. Disponible en: https://www.postman.com/

[12] Microsoft, "Visual Studio Code," 2026, consultado el 2026-09-09. [En línea]. Disponible en: https://code.visualstudio.com/

[13] JUnit Team, "JUnit 5: The next generation of JUnit," 2026, consultado el 2026-09-09. [En línea]. Disponible en: https://junit.org/junit5/

[14] Atlassian, "What is Kanban?" 2026, consultado el 2026-09-09. [En línea]. Disponible en: https://www.atlassian.com/agile/kanban

[15] Conventional Commits, "Conventional Commits: A specification for adding human and machine readable meaning to commit messages," 2026, consultado el 2026-09-09. [En línea]. Disponible en: https://www.conventionalcommits.org/

[16] Talent.com, "Salario medio de un programador junior en España," 2026, consultado el 2026-09-09. [En línea]. Disponible en: https://es.talent.com/salary?job=programador+junior

[17] E. You, "Vite: Next Generation Frontend Tooling," 2026, consultado el 2026-09-09. [En línea]. Disponible en: https://vitejs.dev/